

REALTEK

RTL87x2E Peripheral Sample Project User Manual

Realtek Confidential

V1.4
2024/03/20



REALTEK

Realtek Semiconductor Corp.
No.2, Innovation Road II, Hsinchu Science Park, Hsinchu 300, Taiwan
Tel.: +886-3-578-0211. Fax: +886-3-577-6047
www.realtek.com

COPYRIGHT

©2024 Realtek Semiconductor Corp. All rights reserved. No part of this document may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language in any form or by any means without the written permission of Realtek Semiconductor Corp.

DISCLAIMER

Realtek provides this document ‘as is’, without warranty of any kind. Realtek may make improvements and/or changes in this document or in the product described in this document at any time. This document could include technical inaccuracies or typographical errors.

TRADEMARKS

Realtek is a trademark of Realtek Semiconductor Corporation. Other names mentioned in this document are trademarks/registered trademarks of their respective owners.

USING THIS DOCUMENT

This document is intended for the software engineer’s reference and provides detailed programming information. Though every effort has been made to ensure that this document is current and accurate, more information may have become available subsequent to the production of this guide.

ELECTROSTATIC DISCHARGE (ESD) WARNING

This product can be damaged by Electrostatic Discharge (ESD). When handling, care must be taken. Damage due to inappropriate handling is not covered by warranty.

Do not open the protective conductive packaging until you have read the following, and are at an approved anti-static workstation.

- Use an approved anti-static mat to cover your work surface
- Use a conductive wrist strap attached to a good earth ground
- Always discharge yourself by touching a grounded bare metal surface or approved anti-static mat before picking up an ESD-sensitive electronic component
- If working on a prototyping board, use a soldering iron or station that is marked as ESD-safe
- Always disconnect the microcontroller from the prototyping board when it is being worked on

修订历史

日期	版本	修改	作者	审阅
2021/06/23	V1.0	初稿	Echo_Gao	
2023/03/16	V1.1	修订格式	Echo_Gao	Bert_Wang
2023/06/12	V1.2	核对内容	Grace_Yan	
2023/09/06	V1.3	添加申明	Grace_Yan	
2024/03/20	V1.4	新增三线 SPI	Pacman_Chen	

Realtek Confidential

目 录

1 外设固件概述	1
2 测试环境	2
2.1 RTL87x2E 评估板	2
2.2 Keil	2
2.3 下载（JLINK 仿真器或 MPTool）	4
2.4 LogTool	5
3 外设初始化流程	6
3.1 PAD 和 PINMUX	8
3.2 进出 DLPS 流程	9
4 通用输入/输出（GPIO）	10
4.1 输出控制闪灯	10
4.2 输入检测——轮询方式	13
4.3 按键检测——中断方式	15
5 通用异步接收发送端（UART）	18
5.1 轮询收发	18
5.2 中断接收	21
5.3 UART 唤醒深度低功耗状态（DLPS）	25
5.4 UART 通过 GDMA 接收和发送定长数据	29
5.5 UART 通过 GDMA 接收不定长数据	34
6 定时器和脉冲宽度调制（TIM&PWM）	40
6.1 脉冲宽度调制输出（PWM）	40
6.2 定时器中断	43
6.3 数字信号任意波形输出	45
7 增强定时器（ENHTIM）	48
7.1 GPIO 触发锁存	48
8 直接内存存取控制器（GDMA）	51
8.1 单 Block 传输（Single Block）	51
8.2 多 Block 传输（Multi Block）	54
9 模数转换器（ADC）	59

9.1 单次采样模式——轮询	59
9.2 单次采样模式——中断	63
9.3 单次、差分采样模式	66
9.4 单次采样模式——唤醒深度低功耗状态（DLPS）	69
9.5 连续采样模式	75
9.6 连续采样模式——GDMA 搬运采样数据	78
10 串型外设接口（SPI）	82
10.1 EEPROM 模式读取 ID	82
10.2 FullDuplex 模式读取 ID	85
10.3 中断读取 ID	88
10.4 SPI 与闪存（Flash）通信	91
10.5 GDMA 搬运 SPI 数据	93
10.6 伪静态随机存储（PSRAM）	97
11 内部集成电路（I2C）	103
11.1 I2C 自发自收	103
11.2 I2C 通过 GDMA 实现主机发送从机接收	109
11.3 I2C 通过 GDMA 实现从机发送主机接收	114
11.4 I2C 与 STK8321 通信	120
12 按键扫描（KEYSCAN）	123
12.1 按键扫描	123
13 红外（IR）	128
13.1 红外发送	128
13.2 红外接收	132
13.3 红外编码协议发送	136
13.4 红外学习	141
13.5 红外 GDMA 发送	145
13.6 红外 GDMA 接收	148
14 集成电路内置音频总线（I2S）	151
14.1 音频数据传输	151
14.2 从设备接收数据	154
15 编解码器（CODEC）	157

15.1 模拟麦克	157
15.2 数字麦克	160
16 正交解调 (QDEC)	163
16.1 相位检测	163
17 电容触摸感应按键 (CTC)	167
17.1 按键感应	167
18 实时时钟 (RTC)	170
18.1 滴答定时器	170
18.2 计数溢出	172
18.3 定时比较器 (闹钟)	173
18.4 分频比较器	175
19 低功耗比较器 (LPC)	178
19.1 电压检测	178
19.2 电压检测唤醒低功耗状态 (DLPS)	181
20 三线 SPI (SPI3WIRE)	185
20.1 读取鼠标 ID	185

表目录

表 1-1 外设缩写定义.....	1
-------------------	---

Realtek Confidential

图目录

图 2-1 Keil 工具链版本信息	2
图 2-2 Keil 设置流程示意图	3
图 2-3 UART 跳线	4
图 2-4 MPTool 操作流程示意图	4
图 2-5 LOG 跳线	5
图 2-6 DebugAnalyser 操作流程示意图	5
图 3-1 外设初始化流程	6
图 3-2 PINMUX 和 PAD 电路示意图	8
图 3-3 进出 DLPS 流程	9
图 4-1 LED 驱动电路	10
图 4-2 按键驱动电路	15
图 6-1 GPIO 输出 PWM 波形	47
图 6-2 GPIO 输出 PWM 波形	47
图 12-1 外接矩阵键盘	123
图 13-1 IR 发送数据编码	131
图 13-2 IR 发送数据编码	140
图 13-3 IR-GDMA 发送数据编码	147
图 14-1 I2S 输出波形	153
图 16-1 QDEC 连接图	163
图 16-2 QDEC 相位转移图	163
图 17-1 CTC 原理图	167

词汇表

缩写	含义
EVb	评估板 (Evaluation Board)
FIFO	先进先出 (First In First Out)
GND	地 (ground)
LED	发光二极管 (Light Emitting Diode)
SW	软件模式
LPC	低功耗比较器

Realtek Confidential

1 外设固件概述

表 1-1 外设缩写定义

缩写	外设/单元
GPIO	通用输入输出
UART	通用异步接收发送端
TIM(PWM)	定时器（脉冲宽度调制）
GDMA	直接内存存取控制器
ADC	模数转换器
SPI	串型外设接口
I2C	内部集成电路
KEYSCAN	按键扫描
IR	红外
I2S	集成电路内置音频总线
CODEC	编解码器
QDEC	正交解调
CTC	电容触摸感应按键
RTC	实时时钟
LPC	低功耗比较器

2 测试环境

Peripheral Sample Project 的工程目录：\HoneyComb\sdk\board\evb\io_sample\...

在演示、测试 Peripheral Sample Project 过程中公用的硬件设备、测试软件如下。

➤ 公用硬件环境

RTL87x2E 评估板

JLINK 仿真器

MPTool

➤ 公用测试软件

Keil MDK-ARM v5

LogTool

某些工程使用到专用的硬件设备、测试软件，将在具体章节介绍。

2.1 RTL87x2E 评估板

Peripheral Sample Project 是在 RTL87x2E 评估板上进行演示、测试的，RTL87x2E 评估板由母板和子板组成，本测试使用的母版：EVB MotherBoard 2V0，子板 RTL8762E QFN48 EVB V1.0。

RTL87x2E 评估板的使用请参考 *RTL8762E Evaluation Board User Manual*^[1]。

2.2 Keil

Peripheral Sample Project 中所有工程能够通过 Keil Microcontroller Development Kit(MDK)编译、使用。本测试使用的工具链版本信息如下，建议使用以下或更高版本。



图 2-1 Keil 工具链版本信息

使用 Keil 前，需要进行设置，设置流程如图 2-2 所示。

- 点击工具栏上的，或在菜单中进入 Project→Options，点击 Debug 页面。选择 J-LINK/J-TRACE Cortex，然后点击“Settings”。
- J-Link Port 选择 SW。如果硬件连接是正确的，CPU 就能在 SW Device 列表中识别出来。
- 选择“Flash Download”页面，修改“RAM for Algorithm”的起始地址为 0x00200000，大小为 0x4000。

如果已经有下载算法就先删除。点击“Add”添加 Bee3_SPI_FLASH.FLM（在 sdk\tool\flash 目录下，需将这个文件拷贝到 Keil 安装目录）。

Keil 的使用请参考 *RTL8762E SDK User Guide*^[2]。

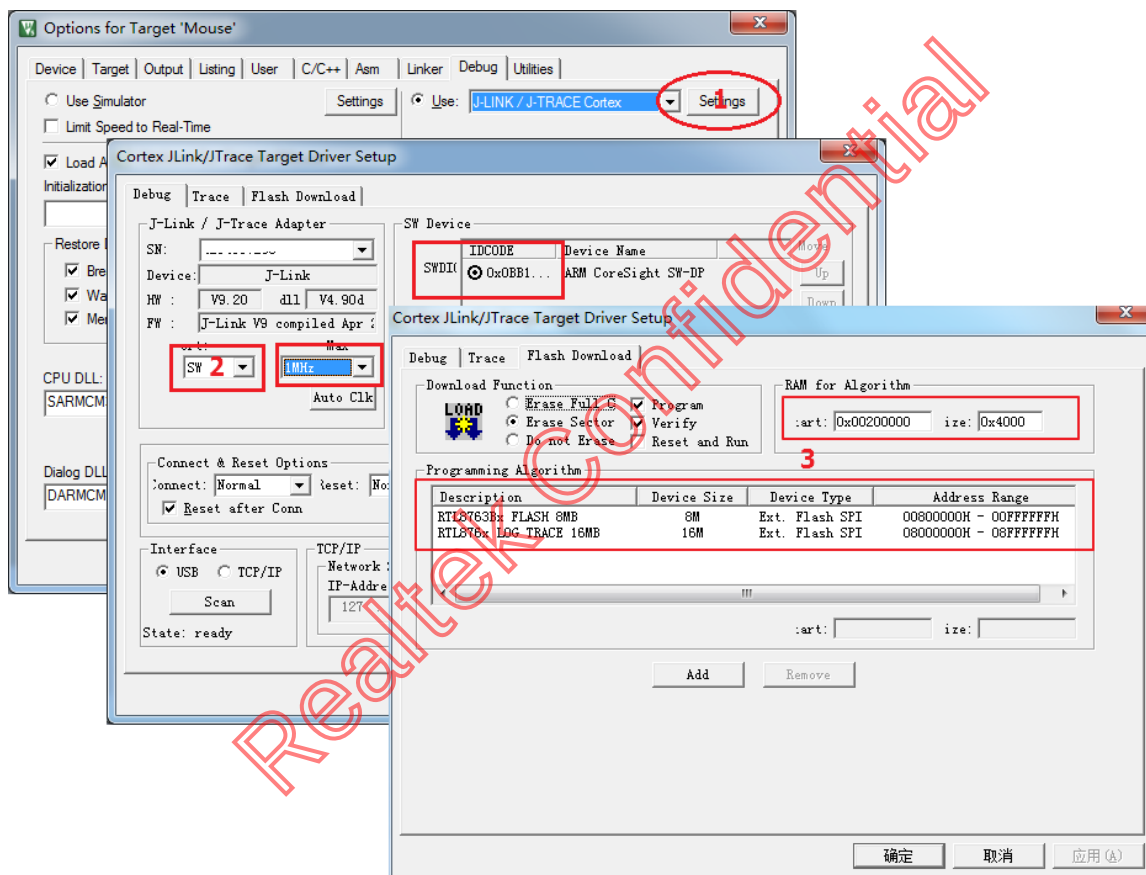


图 2-2 Keil 设置流程示意图

2.3 下载（JLINK 仿真器或 MPTool）

可以使用 J-Link 仿真器搭配 Keil MDK 进行下载调试，详情请参考 *RTL8762E SDK User Guide*^[2]。
除了 J-Link 仿真器，还可以使用 MPTool 进行下载。

- 连接 EVB 的 UART 到 PC 端，跳线连接方式如下图所示。

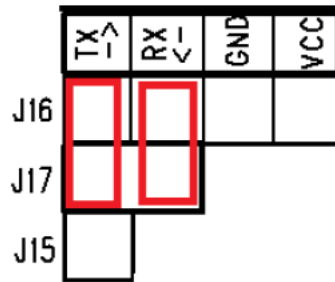


图 2-3 UART 跳线

- 导入 flash map.ini 文件，设置 Config 参数、选择 Secure boot、OTA Header、Patch 等文件。
- 导入编译生成的 APP 镜像文件。
- 探测并打开 UART 端口。
- 点击“Download”按钮开始下载。

MP Tool 操作流程如下图所示。详情请参考 *RTL8762x MP Tool User Guide*^[3]。

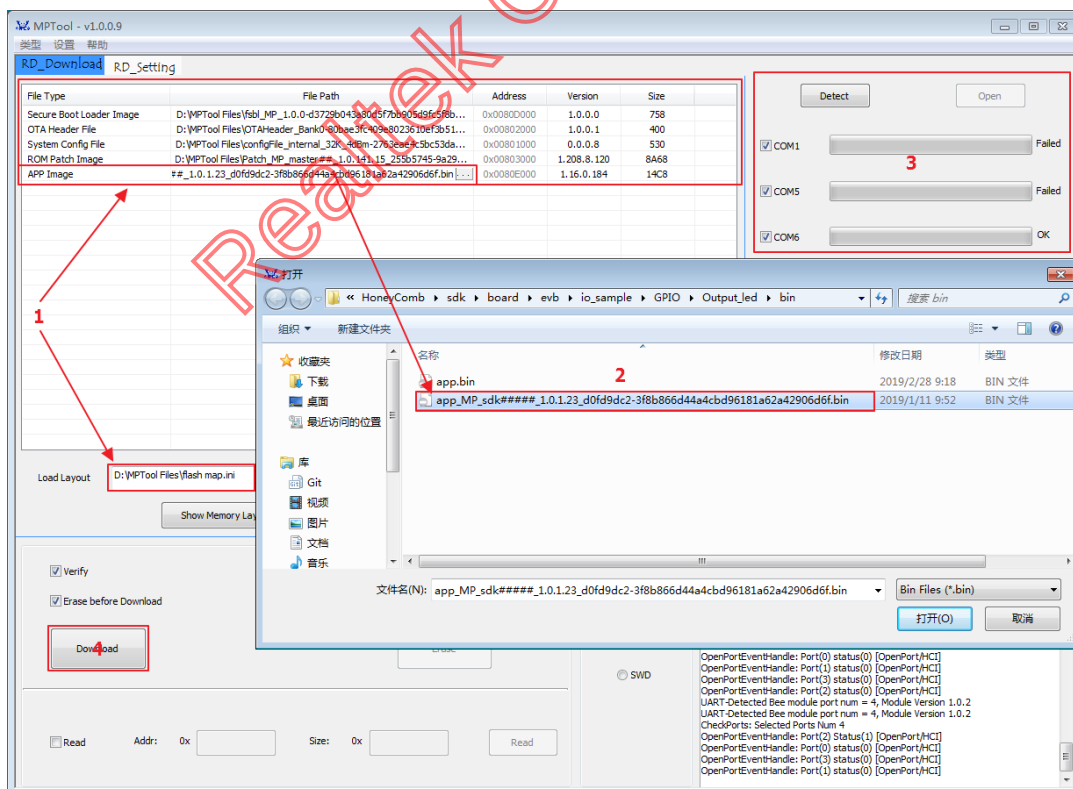


图 2-4 MPTool 操作流程示意图

2.4 LogTool

代码下载到 EVB 上，可以使用 LogTool: DebugAnalyser 软件打印 LOG 信息。

- 连接 EVB 的 LOG 引脚到 PC 端，跳线连接方式如下图所示。

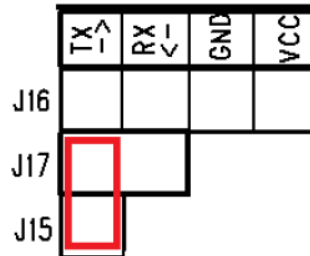


图 2-5 LOG 跳线

- 设置解析输入、解析输出、log 等。
- 配置 UART 参数、加载并导入编译生成的 Trace 文件（加载错误的 Trace 文件，log 信息会出错）
- Start（开始解析），在 log 窗口会打印 log 信息。

DebugAnalyser 的操作流程如下图所示。LogTool 的使用请参考 *Debug Analyzer User Guide*^[4]。

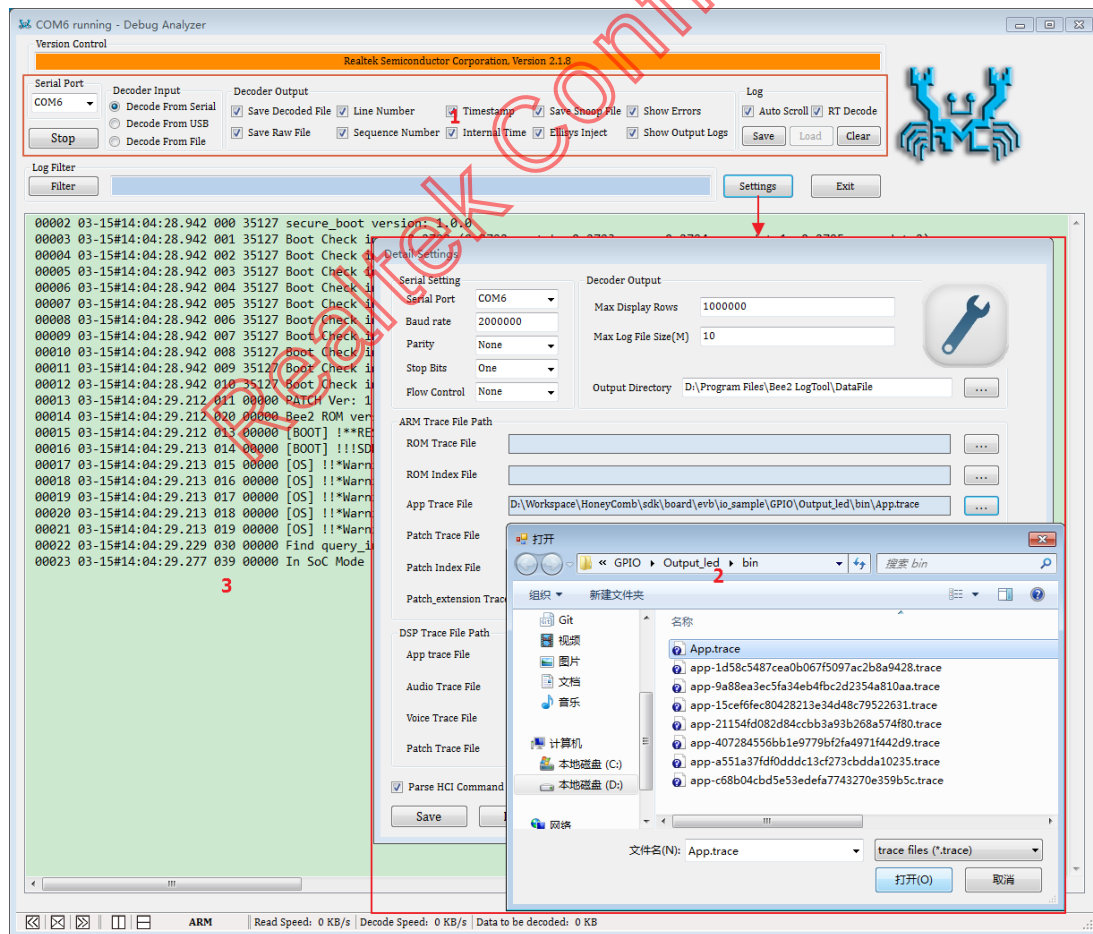


图 2-6 DebugAnalyser 操作流程示意图

3 外设初始化流程

IO 外设的初始化流程主要分为以下几个部分。

- 配置 PAD，即管脚分配定义
- 配置 Pinmux，即引脚复用
- 使能外设的时钟信号
- 设置外设的初始化参数
- 若需要开启中断，则使能相应中断
- 使能外设

流程图如下图所示，具体实现可参考相应外设章节。

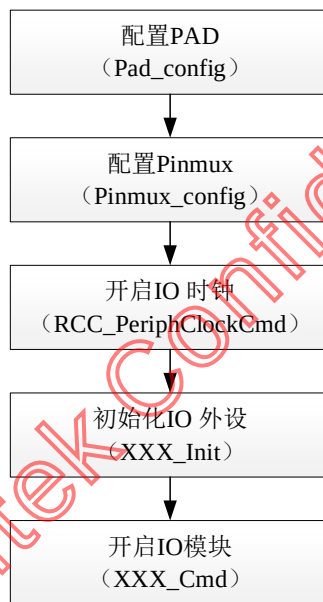


图 3-1 外设初始化流程

➤ 示例：IR-Send

1. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、无内部上拉、输出使能、输出低。
2. 配置 PINMUX：分配引脚为 IRDA_TX 功能。

```

1.  void board_ir_init(void)
2.  {
3.      Pad_Config(IR_TX_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
                  PAD_PULL_NONE, PAD_OUT_ENABLE, PAD_OUT_LOW);
4.      Pinmux_Config(IR_TX_PIN, IRDA_TX);
5.  }
  
```

3. 使能 IR 时钟。
4. 初始化 IR 外设。
 - 1) IR_Freq 设置为 vFreq，即 IR 发送频率为 38KHz。
 - 2)
5. 使能 IR 外设发送功能。

```
1. void driver_ir_init(uint32_t vFreq)
2. {
3.     RCC_PeriphClockCmd(APBPeriph_IR, APBPeriph_IR_CLOCK, ENABLE);
4.     IR_InitTypeDef IR_InitStruct;
5.     IR_StructInit(&IR_InitStruct);
6.     IR_InitStruct.IR_Freq = vFreq;
7.     .....
8.     IR_Cmd(IR_MODE_TX, ENABLE);
9. }
```

Realtek Confidential

3.1 PAD 和 PINMUX

PINMUX 是 Pin Multiplexing 缩写，即引脚复用。通过 PINMUX，可以对指定管脚配置成特定功能，例如 GPIO、I2C、SPI 等功能。PAD 用于控制管脚的工作模式、输入/输出状态以及唤醒系统等。

PINMUX 电路和所有 IO 模块（除 RTC 和 LPC）都在 OFF 区，进入 DLPS 后会掉电，寄存器也会被复位，所以在 DLPS 时无法工作。PAD 电路 ON 区，DLPS 时不掉电，PAD 寄存器设定和功能依然有效。如果在进入 DLPS 后，管脚需要维持在特定状态或者唤醒系统，需要对 PAD 进行相关配置。

下图是 PINMUX 和 PAD 电路示意图。

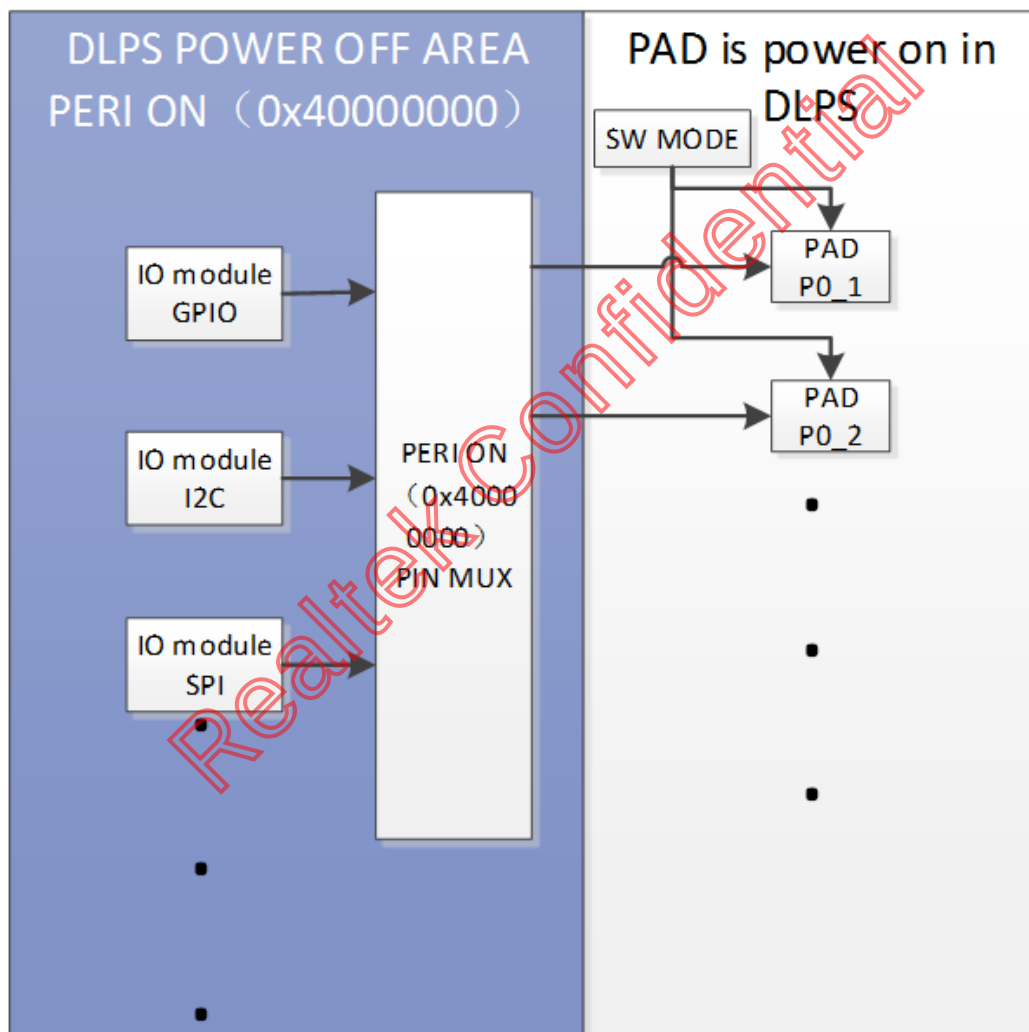


图 3-2 PINMUX 和 PAD 电路示意图

3.2 进出 DLPS 流程

进入 DLPS 后，PINMUX 与 IO 模块（除 RTC 和 LPC）会掉电，而 PAD 可以正常工作。为了控制管脚电平状态，在进入 DLPS 前，相关管脚必须设置成软件控制模式。而退出 DLPS 后，PINMUX 部分会重新上电并恢复寄存器。这时候为了使用 IO 相关功能，管脚必须重新设置为 PINMUX 模式。

设定流程如下。

- APP 初始化时注册进入 DLPS 前的回调函数以及退出 DLPS 后的回调函数，使能外设的时钟信号。
- 在进入 DLPS 前的回调函数中通过 PAD 设定管脚为 SW Mode。
- 在退出 DLPS 后的回调函数中通过 PAD 设定管脚为 PINMUX Mode。

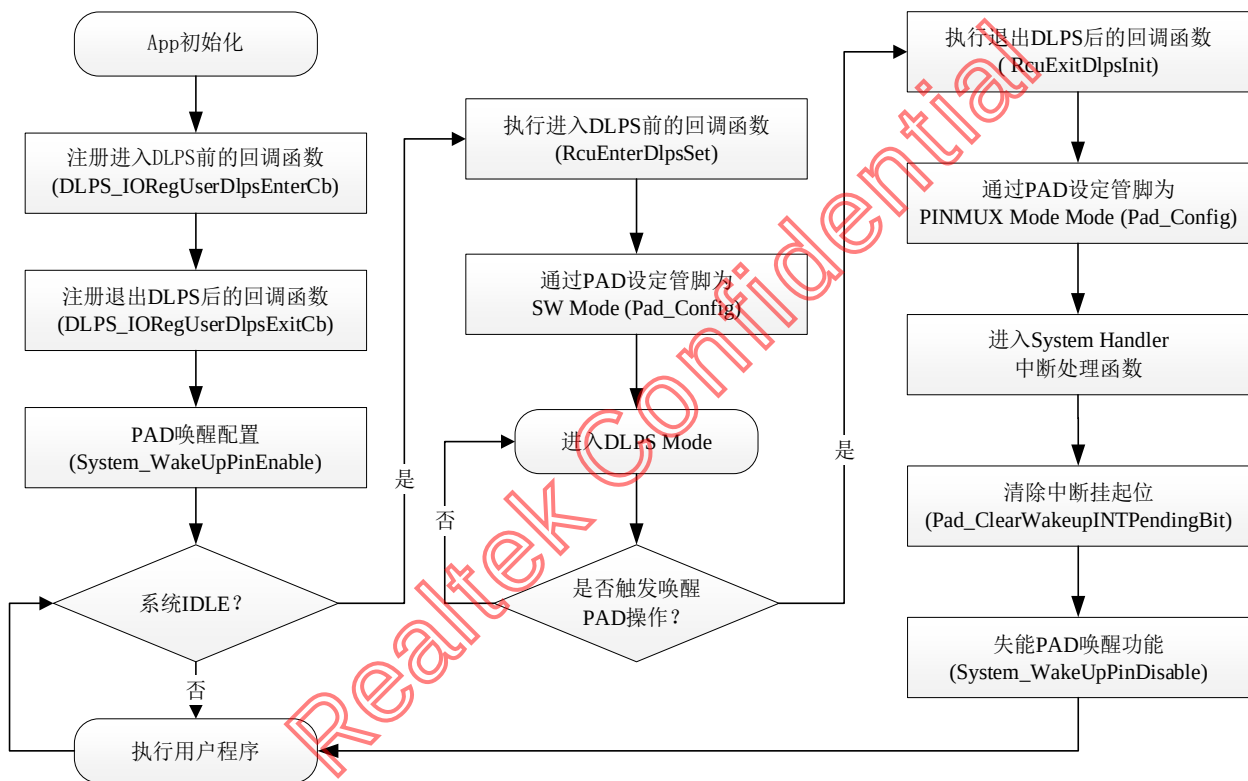


图 3-3 进出 DLPS 流程

4 通用输入/输出（GPIO）

GPIO 集成核心是可编程通用输入输出控制器，每个 GPIO 引脚都可以由软件配置成输出（推挽或开漏）、输入或中断的外设功能端口。

其功能如下：

- 32 个 GPIO 信号，独立 bit 位控制
- 独立的输入，输出控制
- 支持电平、边沿中断
- 支持 1ms-64ms 输入消抖
- 32 路独立中断信号

4.1 输出控制闪灯

通过控制 GPIO 输出高低电平，实现 LED 闪烁功能。

工程目录：\HoneyComb\sdk\board\evb\io_sample\GPIO\Output_led

4.1.1 硬件设计

硬件连接：P0_1→LED0。

在 EVB 上，使用跳帽短接 J24，连接 P0_1 和 LED0，LED 驱动电路如下图所示。

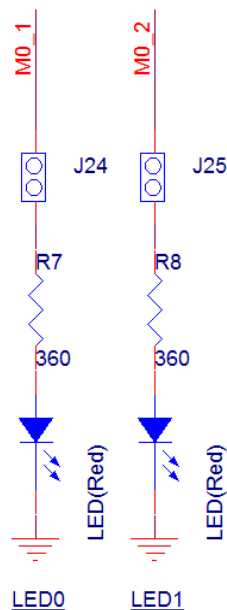


图 4-1 LED 驱动电路

4.1.2 软件流程

1. 引脚定义，GPIO_GetPin 用于得到 PAD 对应的 GPIO。

```
1. #define GPIO_OUTPUT_PIN_0    P0_1
2. #define GPIO_PIN_OUTPUT      GPIO_GetPin(GPIO_OUTPUT_PIN_0)
```

2. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、无内部上拉、输出使能、输出高。
3. 配置 PINMUX：分配引脚为 GPIO 功能。

```
1. board_gpio_init(void)
2. {
3.     Pad_Config(GPIO_OUTPUT_PIN_0, PAD_PINMUX_MODE, PAD_IS_PWRON,
4.               PAD_PULL_NONE, PAD_OUT_ENABLE, PAD_OUT_HIGH);
5.     Pinmux_Config(GPIO_OUTPUT_PIN_0, DWGPIO);
6. }
```

4. 使能 GPIO 时钟。
5. 初始化 GPIO 外设。
 - 1) GPIO_Pin 设置为 GPIO_PIN_OUTPUT。
 - 2) GPIO_Mode 设置为 GPIO_Mode_OUT。
 - 3) GPIO_ITCmd 设置为 DISABLE，即不配置 GPIO 中断模式。

```
1. void driver_gpio_init(void)
2. {
3.     RCC_PeriphClockCmd(APBPeriph_GPIO, APBPeriph_GPIO_CLOCK, ENABLE);
4.     GPIO_InitTypeDef GPIO_InitStructure;
5.     GPIO_StructInit(&GPIO_InitStructure);
6.     GPIO_InitStructure.GPIO_Pin = GPIO_PIN_OUTPUT;
7.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
8.     GPIO_InitStructure.GPIO_ITCmd = DISABLE;
9.     GPIO_Init(&GPIO_InitStructure);
10. }
```

6. 循环控制 GPIO 输出高低电平。

```
1. while (1)
2. {
3.     /* Light up LED0 */
4.     GPIO_WriteBit(GPIO_PIN_OUTPUT, (BitAction)(1));
5.     for (uint32_t i = 0; i < 100000; i++);
6. }
```

```
6.      /* Light off LED0 */
7.      GPIO_WriteBit(GPIO_PIN_OUTPUT, (BitAction)(0));
8.      for (uint32_t i = 0; i < 100000; i++);
9.  }
```

4.1.3 实验现象

LED0 一直闪烁。

Realtek Confidential

4.2 输入检测——轮询方式

实现 GPIO 输入功能，检测 GPIO 输入的高低电平信号。

工程目录：\HoneyComb\sdk\board\evb\io_sample\GPIO\Input_polling

4.2.1 硬件设计

硬件连接：P2_3→输入引脚。

在 EVB 上，使用杜邦线短接 P2_3 和 VCC 检测输入高电平；短接 P2_3 和 GND 检测输入低电平。

4.2.2 软件流程

1. 引脚定义。

```
1. #define GPIO_INPUT_PIN_0    P2_3
2. #define GPIO_PIN_INPUT      GPIO_GetPin(GPIO_INPUT_PIN_0)
```

2. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、内部上拉、输出失能、输出高。

3. 配置 PINMUX：分配引脚为 GPIO 功能。

```
1. void board_gpio_init(void)
2. {
3.     Pad_Config(GPIO_INPUT_PIN_0, PAD_PINMUX_MODE, PAD_IS_PWRON,
4.               PAD_PULL_UP, PAD_OUT_DISABLE, PAD_OUT_HIGH);
5.     Pinmux_Config(GPIO_INPUT_PIN_0, DWGPIO);
6. }
```

4. 使能 GPIO 时钟。

5. 初始化 GPIO 外设。

- 1) GPIO_Pin 设置为 GPIO_PIN_INPUT。
- 2) GPIO_Mode 设置为 GPIO_Mode_IN。
- 3) GPIO_ITCmd 设置为 DISABLE，即不配置 GPIO 中断模式。

```
1. void driver_gpio_init(void)
2. {
3.     RCC_PeriphClockCmd(APBPeriph_GPIO, APBPeriph_GPIO_CLOCK, ENABLE);
4.     GPIO_InitTypeDef GPIO_InitStructure;
5.     GPIO_StructInit(&GPIO_InitStructure);
```

```
6.     GPIO_InitStruct.GPIO_Pin   = GPIO_PIN_INPUT;
7.     GPIO_InitStruct.GPIO_Mode  = GPIO_Mode_IN;
8.     GPIO_InitStruct.GPIO_ITCmd = DISABLE;
9.     GPIO_Init(&GPIO_InitStruct);
10. }
```

6. 循环检测 P2_3 电平状态。

```
1.  while (1)
2.  {
3.      /* Read GPIO input value: gpio_input_data */
4.      uint8_t gpio_input_data = GPIO_ReadInputDataBit(GPIO_PIN_INPUT);
5.      for (uint32_t i = 0; i < 100000; i++);
6.      /** User code here.
7.       * for example: print the value of gpio_input_data;
8.       */
9.  }
```

4.2.3 实验现象

在 DebugAnalyser 工具上，循环打印检测电压高低电平信息：当输入引脚连接高电平时，输出数据显示为 1，当输入引脚连接低电平时，输出数据显示为 0。

4.3 按键检测——中断方式

实现检测 GPIO 输入，通过 GPIO 中断向 app_task 发送 GPIO 消息事件，app_task 检测到消息事件，在 app 层解析 GPIO 消息，执行用户程序：在 DebugAnalyser 工具上，打印按键信息。

工程目录：\HoneyComb\sdk\board\evb\io_sample\GPIO\Input_key

4.3.1 硬件设计

硬件连接：P4_0→KEY0。

在 EVB 上，按键驱动电路如下图所示。

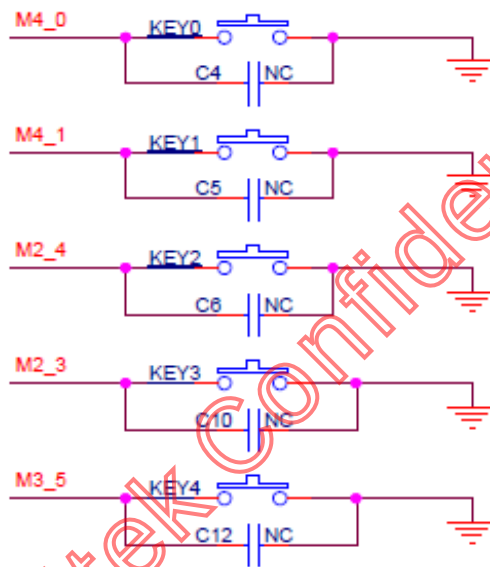


图 4-2 按键驱动电路

4.3.2 软件流程

1. 引脚定义：P4_0 对应 GPIO13，定义 GPIO13_IRQn 中断通道和 GPIO13_Handler 中断处理函数。

```

1.  #define KEY0                P4_0
2.  #define GPIO_INPUT_PIN_0    KEY0
3.  #define GPIO_PIN_INPUT      GPIO_GetPin(GPIO_INPUT_PIN_0)
4.  #define GPIO_PIN_INPUT_IRQN GPIO13_IRQn
5.  #define GPIO_Input_Handler  GPIO13_Handler
    
```

2. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、内部上拉、输出失能、输出高。
3. 配置 PINMUX：分配引脚为 GPIO 功能。


```

1. void board_gpio_init(void)
2. {
3.     Pad_Config(GPIO_INPUT_PIN_0, PAD_PINMUX_MODE, PAD_IS_PWRON,
4.               PAD_PULL_UP, PAD_OUT_DISABLE, PAD_OUT_HIGH);
5.     Pinmux_Config(GPIO_INPUT_PIN_0, DWGPIO);
6. }

```

4. app_task 初始化。

```

1. os_task_create(&app_task_handle, "app", app_main_task, 0,
2.               APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);

```

5. 在 app_main_task 中执行 driver_init 函数，对 GPIO 外设进行初始化。

- 1) GPIO_ITCmd 设置为 ENABLE，即配置 GPIO 中断模式。
- 2) GPIO_ITTrigger 设置为 GPIO_INT_Trigger_EDGE，即边沿触发中断。
- 3) GPIO_ITPolarity 设置为 GPIO_INT_POLARITY_ACTIVE_LOW，即下降沿触发。
- 4) ITDebounce 设置为 GPIO_INT_DEBOUNCE_ENABLE，即使能 GPIO 中断去抖动。
- 5) GPIO_DebounceTime 设置为 10，即去抖时间设置为 10ms。

6. 初始化中断：设置中断优先级，并使能 GPIO_PIN_INPUT_IRQN 通道。

7. 取消屏蔽 GPIO 外部中断，使能 GPIO 中断。

```

1. void driver_gpio_init(void)
2. {
3.     .....
4.     GPIO_InitTypeDef GPIO_InitStructure;
5.     GPIO_StructInit(&GPIO_InitStructure);
6.     GPIO_InitStructure.GPIO_Pin = GPIO_PIN_INPUT;
7.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN;
8.     GPIO_InitStructure.GPIO_ITTrigger = GPIO_INT_Trigger_EDGE;
9.     GPIO_InitStructure.GPIO_ITPolarity = GPIO_INT_POLARITY_ACTIVE_LOW;
10.    GPIO_InitStructure.GPIO_ITDebounce = GPIO_INT_DEBOUNCE_ENABLE;
11.    GPIO_InitStructure.GPIO_DebounceTime = 10; /* unit:ms , can be 1~64 ms */
12.    GPIO_Init(&GPIO_InitStructure);
13.    .....
14.    GPIO_MaskINTConfig(GPIO_PIN_INPUT, DISABLE);
15.    GPIO_INTConfig(GPIO_PIN_INPUT, ENABLE);
16. }

```

8. 开始任务调度。

```

1. os_sched_start();

```

9. 当按键按下（P4_0 由高电平变为低电平）时，检测到 GPIO 下降沿，触发 GPIO 中断，进入中断处理函数 GPIO_Input_Handler。
 - 1) 关 GPIO 中断，屏蔽 GPIO 中断。
 - 2) 定义消息类型 IO_MSG_TYPE_GPIO，发送 msg 给 task。
 - 3) 清除 GPIO 中断挂起位，取消屏蔽 GPIO 中断，使能 GPIO 中断。

```

1.  void GPIO_Input_Handler(void)
2.  {
3.      GPIO_INTConfig(GPIO_PIN_INPUT, DISABLE);
4.      GPIO_MaskINTConfig(GPIO_PIN_INPUT, ENABLE);
5.      T_IO_MSG int_gpio_msg;
6.      int_gpio_msg.type = IO_MSG_TYPE_GPIO;
7.      int_gpio_msg.subtype = 0;
8.      if (false == app_send_msg_to_apptask(&int_gpio_msg))
9.          .....
10.     GPIO_ClearINTPendingBit(GPIO_PIN_INPUT);
11.     GPIO_MaskINTConfig(GPIO_PIN_INPUT, DISABLE);
12.     GPIO_INTConfig(GPIO_PIN_INPUT, ENABLE);
13. }

```

10. app_main_task 循环检测 msg queue，当有 msg 时，执行 app_handle_io_msg(io_msg)函数。
11. 在 app_handle_io_msg 函数中，判断消息类型为 IO_MSG_TYPE_GPIO，执行用户程序。

```

1.  void app_handle_io_msg(T_IO_MSG io_msg)
2.  {
3.      uint16_t msg_type = io_msg.type;
4.      switch (msg_type)
5.      {
6.          .....
7.          case IO_MSG_TYPE_GPIO:
8.              {
9.                  APP_PRINT_INFO0("[app] app_handle_io_msg: GPIO input msg.");
10.             }
11.         }
12.     }

```

4.3.3 实验现象

按下 KEY0，在 DebugAnalyser 工具上，打印按键信息。

5 通用异步接收发送端（UART）

UART 是一种通用串行数据总线，用于异步通信。

其功能如下：

- 波特率可调节，最大波特率至 4M
- 支持自动硬件流控制（Hareware Flow Ctrol）
- 数据长度为 7bit 或 8it
- 停止位为 1 位或 2 位
- 可设置是否进行奇偶校验，在开启奇偶校验后波特率参数可能发生改变
- TX FIFO 大小为 16 字节，Rx FIFO 大小为 32 字节
- 可配置 Rx 触发中断阈值范围：1~29

专用硬件设备：FT232 usb 转串口

专用测试软件：串口调试工具

5.1 轮询收发

通过 UART 实现 RTL87x2E 与 PC 端的数据通信。PC 端发送数据，芯片收到数据并回相同数据给 PC 端，PC 端收到芯片回的数据。

工程目录：\HoneyComb\sdk\board\evb\io_sample\UART\Polling

5.1.1 硬件设计

硬件连接：P3_0→RX，P3_1→TX。

EVB 外接 FT232 模块，连接 P3_0 和 FT232 的 RX，P3_1 和 FT232 的 TX。

5.1.2 软件流程

1. 引脚定义。

```
1.  #define UART_TX_PIN      P3_0
2.  #define UART_RX_PIN      P3_1
```

2. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、内部上拉、输出失能、输出高。

3. 配置 PINMUX：分配引脚分别为 UART0_TX、UART0_RX 功能。

```

1. void board_uart_init(void)
2. {
3.     Pad_Config(UART_TX_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
4.               PAD_PULL_UP, PAD_OUT_DISABLE, PAD_OUT_HIGH);
5.     Pad_Config(UART_RX_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
6.               PAD_PULL_UP, PAD_OUT_DISABLE, PAD_OUT_HIGH);
7.     Pinmux_Config(UART_TX_PIN, UART0_TX);
8.     Pinmux_Config(UART_RX_PIN, UART0_RX);
9. }

```

4. 使能 UART0 时钟。

5. 初始化 UART0 外设。

- 1) 设置波特率为 115200。
- 2) 校验位设置为无奇偶校验。
- 3) 停止位设置为 1 位。
- 4) 数据长度为 8bits。
- 5) Rx FIFO 阈值设置为 16。
- 6) 空闲时间设置为 2Byte，即当前波特率下接收 2 个字节的时间。

```

1. void driver_uart_init(void)
2. {
3.     RCC_PeriphClockCmd(APBPeriph_UART0, APBPeriph_UART0_CLOCK, ENABLE);
4.     UART_InitTypeDef UART_InitStruct;
5.     UART_StructInit(&UART_InitStruct);
6.     UART_InitStruct.div      = BaudRate_Table[BAUD_RATE_115200].div;
7.     UART_InitStruct.ovsr     = BaudRate_Table[BAUD_RATE_115200].ovsr;
8.     UART_InitStruct.ovsr_adj = BaudRate_Table[BAUD_RATE_115200].ovsr_adj;
9.     UART_InitStruct.parity   = UART_PARITY_NO_PARTY;
10.    UART_InitStruct.stopBits  = UART_STOP_BITS_1;
11.    UART_InitStruct.wordLen   = UART_WROD_LENGTH_8BIT;
12.    UART_InitStruct.rxTriggerLevel = 16; //1~29
13.    UART_InitStruct.idle_time  = UART_RX_IDLE_2BYTE; //idle interrupt wait time
14.    UART_Init(UART, &UART_InitStruct);
15. }

```

6. 发送字符串："### Uart demo polling read uart data ###\r\n"，实现多字节连续发送。

```

1. void uart_senddata_continuous(
2.     UART_TypeDef *UARTx, const uint8_t *pSend_Buf, uint16_t vCount)
3. {

```

```

3.     uint8_t count;
4.     while (vCount / UART_TX_FIFO_SIZE > 0)
5.     {
6.         while (UART_GetFlagState(UARTx, UART_FLAG_THR_EMPTY) == 0);
7.         for (count = UART_TX_FIFO_SIZE; count > 0; count--)
8.         {
9.             UARTx->RB_THR = *pSend_Buf++;
10.        }
11.        vCount -= UART_TX_FIFO_SIZE;
12.    }
13.    while (UART_GetFlagState(UARTx, UART_FLAG_THR_EMPTY) == 0);
14.    while (vCount-->0)
15.    {
16.        UARTx->RB_THR = *pSend_Buf++;
17.    }
18. }

```

7. 检测接收到数据状态，状态为 SET 时，接收数据，并发送接收的数据。

```

1.     while (1)
2.     {
3.         if (UART_GetFlagState(UART, UART_FLAG_RX_DATA_RDY) == SET)
4.         {
5.             rx_byte = UART_ReceiveByte(UART);
6.             UART_SendByte(UART, rx_byte);
7.         }
8.     }

```

5.1.3 实验现象

➤ 发送数据

在串口调试工具上接收到芯片发送的字符串"### Uart demo polling read uart data ###\r\n"信息。

➤ 接收数据

使用串口调试助手，发送数据到芯片，RTL87x2E 收到数据并回相同的数据到 PC 端，通过串口调试工具收到与发送数据相同的数据信息。

5.2 中断接收

通过 UART 实现 RTL87x2E 与 PC 端的数据通信。PC 端发送数据，芯片收到数据触发中断，在中断处理函数中接收数据，并传输相同数据给 PC 端，PC 端收到芯片回的数据。

工程目录：\HoneyComb\sdk\board\evb\io_sample\UART\Interrupt

5.2.1 硬件设计

硬件连接：P3_0→RX，P3_1→TX。

EVB 外接 FT232 模块，连接 P3_0 和 FT232 的 RX，P3_1 和 FT232 的 TX。

5.2.2 软件流程

1. 引脚定义。

```
1.  #define UART_TX_PIN      P3_0
2.  #define UART_RX_PIN      P3_1
```

2. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、内部上拉、输出失能、输出高。
3. 配置 PINMUX：分配引脚分别为 UART0_TX、UART0_RX 功能。

```
1.  void board_uart_init(void)
2.  {
3.      Pad_Config(UART_TX_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
4.                 PAD_PULL_UP, PAD_OUT_DISABLE, PAD_OUT_HIGH);
5.      Pad_Config(UART_RX_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
6.                 PAD_PULL_UP, PAD_OUT_DISABLE, PAD_OUT_HIGH);
7.      Pinmux_Config(UART_TX_PIN, UART0_TX);
8.      Pinmux_Config(UART_RX_PIN, UART0_RX);
9.  }
```

4. 使能 UART0 时钟。
5. 初始化 UART0 外设。
 - 1) 设置 div、ovsr、ovsr_adj，波特率 115200。
 - 2) 使能 UART 接收中断(UART_INT_RD_AVA)和 UART 接收空闲中断(UART_INT_RX_IDLE)。
6. 初始化中断：设置中断优先级，并使能 UART0_IRQn 通道。

```

1.  void driver_uart_init(void)
2.  {
3.      RCC_PeriphClockCmd(APBPeriph_UART0, APBPeriph_UART0_CLOCK, ENABLE);
4.      UART_InitTypeDef UART_InitStruct;
5.      UART_StructInit(&UART_InitStruct);
6.      .....
7.      UART_InitStruct.UART_Div    = BaudRate_Table[BAUD_RATE_115200].div;
8.      UART_InitStruct.UART_Ovsr   = BaudRate_Table[BAUD_RATE_115200].ovsr;
9.      UART_InitStruct.UART_OvsrAdj = BaudRate_Table[BAUD_RATE_115200].ovsr_adj;
10.     UART_Init(UART0, &UART_InitStruct);
11.
12.     UART_INTConfig(UART, UART_INT_RD_AVA, ENABLE);
13.     UART_INTConfig(UART, UART_INT_RX_IDLE);
14.     .....
15.     NVIC_InitTypeDef NVIC_InitStruct;
16.     NVIC_InitStruct.NVIC_IRQChannel    = UART0_IRQn;
17.     NVIC_InitStruct.NVIC_IRQChannelCmd = (FunctionalState)ENABLE;
18.     NVIC_InitStruct.NVIC_IRQChannelPriority = 3;
19.     NVIC_Init(&NVIC_InitStruct);
20. }

```

7. 发送字符串: "### Uart demo--Auto Hardware Flow Contrl ###\r\n", 实现多字节连续发送。
(详情参考: UART→Polling)

```

1.  void uart_demo(void)
2.  {
3.      .....
4.      char *demoStr = "### Uart demo--Auto Hardware Flow Contrl ###\r\n";
5.      uart_senddata_continuous(UART0, String_Buf, demoStrLen);
6.      .....
7.  }

```

8. 当 UART 收到数据时触发 UART_INT_RD_AVA 中断, 或当 UART 空闲时触发 UART_INT_RX_IDLE 中断, 进入中断处理函数 UART0_Handler。
9. 失能 UART_INT_RD_AVA 中断。
10. 判断 UART_FLAG_RX_IDLE 状态为 SET 时, 即 UART 空闲。(UART 接收数据结束, 进入空闲状态)
- 1) 失能 UART_INT_RX_IDLE 中断。
 - 2) 清除 UART0 接收 FIFO。
 - 3) 重新使能 UART_INT_RX_IDLE 中断。
 - 4) 接收判断标志 receive_flag 置位。

11. 判断 Interrupt Identifier。

- 1) 当 id 为 UART_INT_ID_RX_LEVEL_REACH 时（数据长度达到 rxTriggerLevel，产生该类型中断），接收 FIFO 数据，保存到 UART_Recv_Buf 中。
- 2) 当 id 为 UART_INT_ID_RX_DATA_TIMEOUT 时（数据长度未达到 rxTriggerLevel，产生该类型中断），接收 FIFO 数据，保存到 UART_Recv_Buf 中。

12. 使能 UART_INT_RD_AVA 中断。

```

1.  void UART0_Handler()
2.  {
3.      .....
4.      UART_INTConfig(UART, UART_INT_RD_AVA, DISABLE);
5.      if (UART_GetFlagState(UART, UART_FLAG_RX_IDLE) == SET)
6.      {
7.          UART_INTConfig(UART0, UART_INT_RX_IDLE, DISABLE);
8.          UART_ClearRx_FIFO(UART0);
9.          UART_INTConfig(UART0, UART_INT_RX_IDLE, ENABLE);
10.         receive_flag = true;
11.     }
12.     switch (int_status & 0x0E)
13.     {
14.     case UART_INT_ID_RX_DATA_TIMEOUT:
15.         {
16.             lenth = UART_GetRx_FIFODataLen(UART0);
17.             UART_ReceiveData(UART0, UART_Recv_Buf, lenth);
18.             for (uint8_t i = 0; i < lenth; i++)
19.             {
20.                 UART_Send_Buf[UART_Recv_Buf_Lenth + i] = UART_Recv_Buf[i];
21.             }
22.             UART_Recv_Buf_Lenth += lenth;
23.             break;
24.         }
25.     case UART_INT_ID_RX_LEVEL_REACH:
26.         {
27.             lenth = UART_GetRx_FIFODataLen(UART0);
28.             UART_ReceiveData(UART0, UART_Recv_Buf, lenth);
29.             for (uint8_t i = 0; i < lenth; i++)
30.             {
31.                 UART_Send_Buf[UART_Recv_Buf_Lenth + i] = UART_Recv_Buf[i];
32.             }
33.             UART_Recv_Buf_Lenth += lenth;
34.             break;
35.         }
36.     .....

```



```

37.     }
38.     UART_INTConfig(UART, UART_INT_RD_AVA, ENABLE);
39.     }

```

13. 循环检测接收数据标志 `receive_flag`，若其值为 `true`，将接收数据标志复位，发送接收的数据到 PC 端。

```

1.  while (1)
2.  {
3.      if (receive_flag == true)
4.      {
5.          receive_flag = false;
6.          uart_senddata_continuous(UART0, UART_Send_Buf, UART_Recv_Buf_Lenth);
7.          .....
8.      }
9.  }

```

5.2.3 实验现象

运行程序，在串口调试工具上收到 RTL87x2E 发送的字符串数据"### Uart demo ###\r\n"；使用串口调试助手，发送数据到芯片，芯片收到数据触发中断，在中断处理函数中接收数据；接收数据完成后回相同数据到 PC 端。

在 DebugAnalyser 工具上，打印 UART 接收数据以及中断信息，在串口调试助手上收到与 PC 端发送到 RTL87x2E 相同的数据信息。

5.3 UART 唤醒深度低功耗状态（DLPS）

通过 UART 实现 RTL87x2E 与 PC 端的数据通信，该程序实现了 UART 接受发送数据以及系统进入 DLPS 状态后，通过接受数据唤醒 DLPS 状态等功能。

系统处于 IDLE 状态时，会自动进入 DLPS；当通过串口调试助手发送数据给 RTL87x2E 时，UART_RX_PIN 出现低电平，唤醒系统，与 PC 端进行数据的通信。

工程目录：\HoneyComb\sdk\board\evb\io_sample\UART\DLPS

5.3.1 硬件设计

硬件连接：P3_0→RX，P3_1→TX。

EVB 外接 FT232 模块，连接 P3_0 和 FT232 的 RX，P3_1 和 FT232 的 TX。

5.3.2 软件流程

1. 开启 DLPS 功能。

```
1.  #define DLPS_EN 1
2.  #define USE_UART_DLPS 1
3.  #define USE_USER_DEFINE_DLPS_EXIT_CB 1
4.  #define USE_USER_DEFINE_DLPS_ENTER_CB 1
```

2. 初始化 UART 全局数据。

```
1.  void global_data_uart_init(void)
2.  {
3.      IO_UART_DLPS_Enter_Allowed = true;
4.      UART_RX_Count = 0;
5.      memset(UART_RX_Buffer, 0, sizeof(UART_RX_Buffer));
6.  }
```

3. 配置 PAD、PINMUX，设置 P3_0、P3_1 为 UART0 功能。

4. 执行 pwr_mgr_init 函数，初始化电源管理。

- 1) 注册用户进入 DLPS 回调函数 app_enter_dlps_config，注册用户退出 DLPS 回调函数 app_exit_dlps_config。
- 2) 注册硬件控制回调函数 DLPS_IO_EnterDlpsCb 和 DLPS_IO_ExitDlpsCb，进入 DLPS 会保存 CPU、PINMUX、Peripheral 等，退出 DLPS 会恢复 CPU、PINMUX、Peripheral 等。

- 3) 设置 DLPS 模式。
- 4) 设置唤醒 DLPS 方式为 UART_RX_PIN 低电平唤醒。

```

1.  void pwr_mgr_init(void)
2.  {
3.      #if DLPS_EN
4.          if (false == dlps_check_cb_reg(app_dlps_check_cb))
5.              .....
6.          DLPS_IORegUserDlpsEnterCb(app_enter_dlps_config);
7.          DLPS_IORegUserDlpsExitCb(app_exit_dlps_config);
8.          DLPS_IORegister();
9.          lps_mode_set(LPM_DLPS_MODE);
10.         System_WakeUpPinEnable(UART_RX_PIN, PAD_WAKEUP_POL_LOW, 0);
11.         .....
12.     #endif
13. }

```

5. 在 app_enter_dlps_config 中，执行 io_uart_dlps_enter 函数，设置 UART_TX_PIN 和 UART_RX_PIN 为 SW 模式，设置唤醒 DLPS 方式为 UART_RX_PIN 低电平唤醒。

```

1.  void io_uart_dlps_enter(void)
2.  {
3.      Pad_ControlSelectValue(UART_TX_PIN, PAD_SW_MODE);
4.      Pad_ControlSelectValue(UART_RX_PIN, PAD_SW_MODE);
5.      System_WakeUpPinEnable(UART_RX_PIN, PAD_WAKEUP_POL_LOW, 0);
6.  }

```

6. 在 app_exit_dlps_config 中，执行 io_uart_dlps_exit 函数，设置 UART_TX_PIN 和 UART_RX_PIN 为 PINMUX 模式。

```

1.  void io_uart_dlps_exit(void)
2.  {
3.      Pad_ControlSelectValue(UART_TX_PIN, PAD_PINMUX_MODE);
4.      Pad_ControlSelectValue(UART_RX_PIN, PAD_PINMUX_MODE);
5.  }

```

7. app_task 初始化。

```

1.  os_task_create(&app_task_handle, "app", app_main_task, 0,
                APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);

```

8. 在 app_main_task 中执行 driver_init 函数，执行 driver_uart_init 函数，对 UART 外设进行初始化，开启 UART 接受数据相关中断。（详情参考：UART→Interrupt）

9. 在 `app_handle_io_msg` 函数中，判断消息类型为 `IO_MSG_TYPE_UART`，执行 `io_handle_uart_msg` 函数，执行 `io_uart_handle_msg`。

10. 判断子类型为 `IO_MSG_UART_RX` 时

- 1) 将 `UART_RX_Buffer` 中的数据通过 UART 发送到 PC 端。
- 2) 初始化 UART 全局数据。
- 3) 判断 UART 发送缓冲区为空的状态为 0 时，发送完成，`IO_UART_DLPS_Enter_Allowed` 置位。

```

1.  void io_uart_handle_msg(T_IO_MSG *io_uart_msg)
2.  {
3.      .....
4.      if (IO_MSG_UART_RX == subtype)
5.      {
6.          uart_senddata_continuous(UART0, UART_RX_Buffer, UART_RX_Count);
7.          global_data_uart_init();
8.          while (UART_GetFlagState(UART0, UART_FLAG_THR_EMPTY) == 0)
9.          {
10.             IO_UART_DLPS_Enter_Allowed = true;
11.          }
12.      }
13.  }
```

11. 系统处于 IDLE 状态时，会自动进入 DLPS。

12. 通过串口调试助手发送数据给 RTL87x2E 时，`UART_RX_PIN` 出现低电平，唤醒 DLPS，执行 `System_Handler`。

- 1) 清除 `UART_RX_PIN` 的唤醒中断挂起位。
- 2) 失能 `UART_RX_PIN` 的唤醒功能。
- 3) `IO_UART_DLPS_Enter_Allowed` 设置为 False。

```

1.  void System_Handler(void)
2.  {
3.      APP_PRINT_INFO0("[main] System_Handler");
4.      if (System_WakeUpInterruptValue(UART_RX_PIN) == SET)
5.      {
6.          Pad_ClearWakeUpINTPendingBit(UART_RX_PIN);
7.          System_WakeUpPinDisable(UART_RX_PIN);
8.          IO_UART_DLPS_Enter_Allowed = false;
9.      }
10. }
```

5.3.3 实验现象

系统处于 IDLE 状态时，会自动进入 DLPS。

在 DLPS 状态下，使用串口调试助手发送的第一笔数据唤醒系统，再发送数据，芯片收到数据并回相同数据到 PC 端，串口调试助手收到芯片回的数据。

Realtek Confidential

5.4 UART 通过 GDMA 接收和发送定长数据

通过 GDMA 外设和 UART 外设实现 RTL87x2E 芯片与 PC 端进行数据通信。

➤ 芯片发送数据到 PC 端时

芯片发送数据并通过 GDMA 外设搬运数据到 PC 端。

➤ PC 端发送数据到芯片时

芯片收到数据并通过 GDMA 外设搬运数据，并回相同数据给 PC 端，PC 端收到芯片端回的数据。

工程目录：\HoneyComb\sdk\board\evb\io_sample\UART\GDMA

5.4.1 硬件设计

硬件连接：P3_0→RX，P3_1→TX。

EVB 外接 FT232 模块，连接 P3_0 和 FT232 的 RX，P3_1 和 FT232 的 TX。

5.4.2 软件流程

1. 引脚定义。

```
1.  #define UART_TX_PIN      P3_0
2.  #define UART_RX_PIN      P3_1
```

2. 开启 UART_GDMA 功能。

```
1.  #define UART_GDMA_TX_ENABLE      EVB_ENABLE
2.  #define UART_GDMA_RX_ENABLE      EVB_ENABLE
```

3. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、内部上拉、输出失能、输出高。

4. 配置 PINMUX：分配引脚 P3_0、P3_1 分别为 UART0_TX、UART0_RX 功能。

5. app_task 初始化。

```
1.  os_task_create(&app_task_handle,"app",app_main_task,0,
                APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);
```

6. 在 app_main_task 中执行 driver_init 函数，执行 driver_uart_init 函数，对 UART 外设进行初始化。

- 1) 设置 div、ovsr、ovsr_adj，波特率 115200。
- 2) rxThdLevel 设置为 16，FIFO 阈值为 16。
- 3) idle_time 设置为 UART_RX_IDLE_2BYTE，即当前波特率下接收 2 个字节的时间。
- 4) dmaEn 设置为 UART_DMA_ENABLE，即使能 UART_DMA 传输。
- 5) txWaterLevel 设置为 15，其值一般设置为 TX_FIFO_SIZE-GDMA_MSize。
- 6) rxWaterLevel 设置为 1，其值一般设置为 GDMA_MSize。
- 7) 使能 UART_TxDmaEn 和 UART_RxDmaEn。

```

1.  void driver_uart_init(void)
2.  {
3.      .....
4.      UART_InitStruct.UART_Div      = CBaudrateTable[UART_BAUDRATE].div;
5.      UART_InitStruct.UART_Ovsr     = CBaudrateTable[UART_BAUDRATE].ovsr;
6.      UART_InitStruct.UART_OvsrAdj  = CBaudrateTable[UART_BAUDRATE].ovsr_adj;
7.      UART_InitStruct.rxTriggerLevel = 16; //1~29
8.      UART_InitStruct.idle_time     = UART_RX_IDLE_2BYTE; //idle interrupt wait time
9.      UART_InitStruct.UART_DmaEn    = UART_DMA_ENABLE;
10.     UART_InitStruct.UART_TxDmaEn   = ENABLE;
11.     UART_InitStruct.UART_RxDmaEn   = ENABLE;
12.     UART_InitStruct.UART_TxWaterLevel = 15;
        //Better to equal TX_FIFO_SIZE(16)-GDMA_MSize
13.     UART_InitStruct.UART_RxWaterLevel = 1; //Better to equal GDMA_MSize
14.     UART_InitStruct.UART_TxDmaEn    = ENABLE;
15.     UART_InitStruct.UART_RxDmaEn    = ENABLE;
16.     UART_Init(UART, &UART_InitStruct);
17. }

```

7. 开始任务调度。

```

1.  os_sched_start();

```

8. stack 准备好时执行 app_handle_dev_state_evt 函数，执行 io_uart_demo 函数，初始化 GDMA 外设。

- 1) 执行 UART_SendDataByGDMA，初始化 GDMA 发送数据外设。
- 2) 执行 UART_ReceiveDataByGDMA，初始化 GDMA 接收数据外设。

```

1.  void io_uart_demo(void)
2.  {
3.      .....
4.      UART_SendDataByGDMA();
5.      UART_ReceiveDataByGDMA();
6.  }

```

9. 在 UART_SendDataByGDMA 中，对 GDMA 通道 0 外设进行初始化。

- 1) 使用 GDMA 通道 0。
- 2) GDMA 的传输方向为内存到外设传输。
- 3) GDMA_SourceAddr 设置为 GDMA_SendData_Buffer。
- 4) GDMA_DestinationAddr 设置为(&(UART0->RB_THR))。
- 5) 使能 GDMA 通道 0 总传输完成中断（GDMA_INT_Transfer）。
- 6) 执行 GDMA_Cmd 函数：使能 GDMA 传输。

```

1.  void UART_SendDataByGDMA(void)
2.  {
3.      .....
4.      GDMA_InitStruct.GDMA_ChannelNum    = UART_TX_GDMA_CHANNEL_NUM;
5.      GDMA_InitStruct.GDMA_DIR            = GDMA_DIR_MemoryToPeripheral;
6.      GDMA_InitStruct.GDMA_BufferSize     = UART_TX_GDMA_BUFFER_SIZE;
7.      //determine total transfer size
8.      .....
9.      GDMA_InitStruct.GDMA_SourceAddr     = (uint32_t)GDMA_SendData_Buffer;
10.     GDMA_InitStruct.GDMA_DestinationAddr = (uint32_t)(&(UART0->RB_THR));
11.     .....
12.     GDMA_INTConfig(UART_TX_GDMA_CHANNEL_NUM, GDMA_INT_Transfer, ENABLE);
13.     GDMA_Cmd(UART_TX_GDMA_CHANNEL_NUM, ENABLE);
14. }

```

10. 在 UART_ReceiveDataByGDMA 中，对 GDMA 通道 1 外设进行初始化。

- 1) 使用 GDMA 通道 1。
- 2) GDMA 的传输方向为外设到内存传输。
- 3) GDMA_SourceAddr 设置为(&(UART0->RB_THR))。
- 4) GDMA_DestinationAddr 设置为 GDMA_ReceiveData_Buffer。
- 5) 使能 GDMA 通道 1 总传输完成中断（GDMA_INT_Transfer）。
- 6) 执行 GDMA_Cmd 函数：使能 GDMA 传输。

```

1.  void UART_ReceiveDataByGDMA (void)
2.  {
3.      .....
4.      GDMA_InitStruct.GDMA_ChannelNum    = UART_RX_GDMA_CHANNEL_NUM;
5.      GDMA_InitStruct.GDMA_DIR            = GDMA_DIR_PeripheralToMemory;
6.      .....
7.      GDMA_InitStruct.GDMA_SourceAddr     = (uint32_t)(&(UART0->RB_THR));
8.      GDMA_InitStruct.GDMA_DestinationAddr = (uint32_t)GDMA_ReceiveData_Buffer;
9.      .....
10.     GDMA_INTConfig(UART_RX_GDMA_CHANNEL_NUM, GDMA_INT_Transfer, ENABLE);
11.     GDMA_Cmd(UART_RX_GDMA_CHANNEL_NUM, ENABLE);

```


12. }

11. 当 UART 通过 GDMA 传输发送数据时, GDMA 搬运数据到目的端地址(&(UART0->RB_THR)), 当 GDMA 发送数据总传输完成时, 触发 GDMA_INT_Transfer 中断, 进入中断处理函数 UART_TX_GDMA_Handler。

- 1) 清除 GDMA 通道 0 GDMA_INT_Transfer 中断挂起位。
- 2) 定义消息类型 IO_MSG_TYPE_GDMA, 子类型为 0, 保存发送数据, 发送 msg 给 task。

```

1.  void UART_TX_GDMA_Handler (void)
2.  {
3.      GDMA_ClearINTPendingBit(UART_TX_GDMA_CHANNEL_NUM, GDMA_INT_Transfer);
4.
5.      T_IO_MSG int_uart_msg;
6.      int_uart_msg.type = IO_MSG_TYPE_GDMA;
7.      int_uart_msg.subtype = 0;
8.      int_uart_msg.u.buf = (void *) (GDMA_SendData_Buffer);
9.      .....
10. }
```

12. 当 UART 通过 GDMA 搬运接收数据时, GDMA 搬运数据到内存, 当 GDMA 接收数据总传输完成时, 触发 GDMA_INT_Transfer 中断, 进入中断处理函数 UART_RX_GDMA_Handler。

- 1) 清除 GDMA 通道 1GDMA_INT_Transfer 中断挂起位。
- 2) 定义消息类型 IO_MSG_TYPE_GDMA, 子类型为 1, 发送 msg 给 task。

```

1.  void UART_RX_GDMA_Handler (void)
2.  {
3.      GDMA_ClearINTPendingBit(UART_RX_GDMA_CHANNEL_NUM, GDMA_INT_Transfer);
4.
5.      T_IO_MSG int_uart_msg;
6.      int_uart_msg.type = IO_MSG_TYPE_GDMA;
7.      int_uart_msg.subtype = 1;
8.      int_uart_msg.u.buf = (void *) (GDMA_SendData_Buffer);
9.      .....
10. }
```

13. app_main_task 循环检测 msg queue, 当有 msg 时, 执行 app_handle_io_msg(io_msg)函数。

14. 在 app_handle_io_msg 函数中, 判断消息类型为 IO_MSG_TYPE_GDMA, 执行 io_handle_gdma_msg(&io_msg)函数, 执行 io_handle_gdma_msg。

15. 判断 subtype 类型

- 1) 如果 subtype == 0, 即 TX 传输, 打印 GDMA 搬运发送数据完成信息。

- 2) 如果 subtype == 1, 即 RX 传输, 打印 GDMA 搬运接收数据完成信息, 芯片收到数据并回相同数据给 PC 端。

```

1. void io_handle_gdma_msg(T_IO_MSG *io_gdma_msg)
2. {
3.     uint16_t subtype = io_gdma_msg->subtype;
4.     if (subtype == 0)
5.     {
6.         APP_PRINT_INFO0("io_handle_gdma_msg: uart tx done ");
7.     }
8.     else if (subtype == 1)
9.     {
10.        APP_PRINT_INFO0("io_handle_gdma_msg: uart rx done ");
11.        UART_SendData_Continuous(
12.            UART0, GDMA_ReceiveData_Buffer, UART_RX_GDMA_BUFFER_SIZE);
13.    }
14. }

```

5.4.3 实验现象

程序运行后, 芯片发送数据, GDMA 搬运发送数据完成后产生中断, 在 DebugAnalyser 工具上, 打印 GDMA 搬运发送数据完成信息, 可以通过串口调试助手收到芯片发送的数据。

使用串口调试助手发送数据到芯片, 发送长度要大于或者等于设置的 GDMA 搬运数据总长度, 通过 GDMA 外设搬运接收数据完成后产生中断, 在 DebugAnalyser 工具上, 打印接收数据完成信息, 芯片收到数据并回相同数据, 串口调试助手收到芯片回的数据。

5.5 UART 通过 GDMA 接收不定长数据

通过 UART 和 GDMA 外设实现 RTL87x2E 芯片与 PC 端的数据通信。PC 端发送数据给芯片时，程序通过 UART 中断和 GDMA 中断搬运接收数据；接受数据完成后，通过 GDMA 搬运数据到 PC 端，PC 端收到芯片回的数据。

工程目录：\HoneyComb\sdk\board\evb\io_sample\UART\DMA_unfixedlen

5.5.1 硬件设计

硬件连接：P3_0→RX，P3_1→TX。

EVB 外接 FT232 模块，连接 P3_0 和 FT232 的 RX，P3_1 和 FT232 的 TX。

5.5.2 软件流程

1. 引脚定义。

```
1.  #define UART_TX_PIN      P3_0
2.  #define UART_RX_PIN      P3_1
```

2. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、内部上拉、输出失能、输出高。

3. 配置 PINMUX：分配引脚 P3_0、P3_1 分别为 UART0_TX、UART0_RX 功能。

4. 使能 UART0 时钟。

5. 初始化 UART0 外设。

- 1) 设置波特率 115200。
- 2) FIFO 阈值为 8。
- 3) dle_time 设置为 UART_RX_IDLE_2BYTE。
- 4) 使能 UART_TxDmaEn 和 UART_RxDmaEn。
- 5) UART_DmaEn 设置为 UART_DMA_ENABLE，即使能 UART_DMA 传输。

6. 使能 UART 接收空闲中断（UART_INT_RX_IDLE）以及线接收状态中断（UART_INT_RX_LINE_STS）。

```
1.  void driver_uart_init(void)
2.  {
3.      .....
4.      RCC_PeriphClockCmd(APBPeriph_UART0, APBPeriph_UART0_CLOCK, ENABLE);
```

```

5.      .....
6.      UART_InitStruct.UART_RxDmaEn      = ENABLE;
7.      UART_InitStruct.UART_TxDmaEn      = ENABLE;
8.      UART_InitStruct.UART_DmaEn        = UART_DMA_ENABLE;
9.      UART_Init(UART0, &UART_InitStruct);
10.     .....
11.     UART_INTConfig(UART0, UART_INT_RX_IDLE | UART_INT_RX_LINE_STS, ENABLE);
12.     .....
13.     }

```

7. 执行 UART_ReceiveDataByGDMA 函数，对 GDMA 接收数据外设进行初始化。

- 1) 使用 GDMA 通道 2。
- 2) GDMA 的传输方向为外设到内存传输。
- 3) GDMA_SourceAddr 设置为(&(UART0->RB_THR))。
- 4) GDMA_DestinationAddr 设置为 GDMA_Rx_Buf。
- 5) 使能 GDMA 通道 2 总传输完成中断（GDMA_INT_Transfer）。
- 6) 执行 GDMA_Cmd 函数：使能 GDMA 传输。

```

1.  void driver_gdma2_init (void)
2.  {
3.      .....
4.      GDMA_InitStruct.GDMA_ChannelNum      = DMA_RX_CHANNEL_NUM;
5.      GDMA_InitStruct.GDMA_DIR              = GDMA_DIR_PeripheralToMemory;
6.      .....
7.      GDMA_InitStruct.GDMA_SourceAddr      = (uint32_t)(&(UART0->RB_THR));
8.      GDMA_InitStruct.GDMA_DestinationAddr  = (uint32_t)GDMA_Rx_Buf;
9.      .....
10.     GDMA_INTConfig(UART_RX_GDMA_CHANNEL_NUM, GDMA_INT_Transfer, ENABLE);
11.     GDMA_Cmd(UART_RX_GDMA_CHANNEL_NUM, ENABLE);
12.     }

```

8. 执行 UART_SendDataByGDMA 函数，对 GDMA 发送数据外设进行初始化。

- 1) 使用 GDMA 通道 3。
- 2) GDMA 的传输方向为内存到外设传输。
- 3) GDMA_SourceAddr 设置为 GDMA_Tx_Buf。
- 4) GDMA_DestinationAddr 设置为(&(UART0->RB_THR))。
- 5) 使能 GDMA 通道 3 总传输完成中断（GDMA_INT_Transfer）。

```

1.  void driver_gdma3_init (void)
2.  {
3.      .....
4.      GDMA_InitStruct.GDMA_ChannelNum      = DMA_TX_CHANNEL_NUM;

```

```

5.      GDMA_InitStruct.GDMA_DIR                = GDMA_DIR_MemoryToPeripheral;
6.      .....
7.      GDMA_InitStruct.GDMA_SourceAddr          = (uint32_t)GDMA_Tx_Buf;
8.      GDMA_InitStruct.GDMA_DestinationAddr     = (uint32_t)(amp;(UART0->RB_THR));
9.      .....
10.     GDMA_INTConfig(UART_RX_GDMA_CHANNEL_NUM, GDMA_INT_Transfer, ENABLE);
11.     }

```

9. 当 GDMA 接收数据总传输完成时，触发 GDMA_INT_Transfer 中断，进入中断处理函数 UART_RX_GDMA_Handler。

- 1) 失能 GDMA 通道 2 传输。
- 2) 清除 GDMA 通道 2 全部中断挂起。
- 3) 保存 GDMA_Rx_Buf 数据。
- 4) 清除 GDMA 通道 2 GDMA_INT_Transfer 中断挂起位。
- 5) 重新设置目的端地址。
- 6) 使能 GDMA 通道 2 传输。

```

1.  void UART_RX_GDMA_Handler (void)
2.  {
3.      GDMA_Cmd(DMA_RX_CHANNEL_NUM, DISABLE);
4.      GDMA_ClearAllTypeINT(DMA_RX_CHANNEL_NUM);
5.      .....
6.      memcpy(GDMA_Tx_Buf + GDMA_BLOCK_SIZE * (count - 1),
              GDMA_Rx_Buf, GDMA_BLOCK_SIZE);
7.
8.      GDMA_ClearINTPendingBit(DMA_RX_CHANNEL_NUM, GDMA_INT_Transfer);
9.      GDMA_SetDestinationAddress(DMA_RX_CHANNEL, (uint32_t)GDMA_Rx_Buf);
10.     GDMA_Cmd(DMA_RX_CHANNEL_NUM, ENABLE);
11.     }

```

10. 当 UART 接收空闲时触发 UART_INT_RX_IDLE 中断，当 UART 线状态出错误时触发 UART_INT_RX_LINE_STS 中断，进入中断处理函数 UART0_Handler。

11. 获取中断标志。

12. 判断 UART_FLAG_RX_IDLE 状态为 SET 时，即 UART 空闲（UART 接收数据结束，进入空闲状态）

- 1) 暂停 GDMA 通道 2 传输。
- 2) 失能 UART_FLAG_RX_IDLE 中断。
- 3) 保存 GDMA 通道 2 中接收数据长度。
- 4) 打印接收数据信息。

5) 判断 GDMA 通道 2 中是否有数据

- (1) 若有数据，则将数据保存到 GDMA_Tx_Buf 中，失能 GDMA 通道 2 传输，清除 GDMA 通道 2 传输悬挂；执行 driver_gdma2_init 函数，将接收数据标志 receiveflg 置位。
- (2) 若无数据，则清除 GDMA 通道 2 传输悬挂，将接收数据标志 receiveflg 置位。

6) 清除 UART 的 RX FIFO 中的数据。

7) 重新使能 UART_INT_IDLE 中断。

13. 判断 Interrupt Identifier。

- 1) 当 id 为 UART_INT_ID_RX_TMEOUT 时(数据长度未达到 rxTriggerLevel, 产生该类型中断)，检测接收到数据状态，状态为 SET 时，接收并保存数据。
- 2) 当 id 为 UART_INT_ID_LINE_STATUS 时（线状态错误，产生该类型中断），打印线状态错误信息。

```

1.  void UART0_Handler()
2.  {
3.      /* read interrupt id */
4.      uint32_t int_status = UART_GetIID(UART0);
5.
6.      if (UART_GetFlagStatus(UART0, UART_FLAG_RX_IDLE) == SET)
7.      {
8.          GDMA_SuspendCmd(DMA_RX_CHANNEL, ENABLE);
9.          UART_INTConfig(UART0, UART_INT_RX_IDLE, DISABLE);
10.
11.         data_len = GDMA_GetTransferLen(DMA_RX_CHANNEL);
12.         for (uint32_t i = 0; i < data_len; i++)
13.         {
14.             DBG_DIRECT("value is 0x%x", GDMA_Rx_Buf[i]);
15.         }
16.
17.         if (data_len)
18.         {
19.             receive_offset += data_len;
20.             memcpy(GDMA_Tx_Buf + GDMA_BLOCK_SIZE * count,
21.                   GDMA_Rx_Buf, receive_offset);
22.             .....
23.             GDMA_Cmd(DMA_RX_CHANNEL, DISABLE);
24.             GDMA_SuspendCmd(DMA_RX_CHANNEL, DISABLE);
25.
26.             driver_gdma2_init ();
27.             receiveflg = true;
28.         }
29.     }

```

```

30.         GDMA_SuspendCmd(DMA_RX_CHANNEL, DISABLE);
31.         receiveflg = true;
32.     }
33.     UART_ClearRxFIFO(UART0);
34.     UART_INTConfig(UART0, UART_INT_RX_IDLE, ENABLE);
35. }
36. UART_INTConfig(UART0, UART_INT_RD_AVA | UART_INT_RX_LINE_STS, DISABLE);
37. switch (int_status & 0x0E)
38. {
39.     case UART_INT_ID_RX_DATA_TIMEOUT:
40.     {
41.         while (UART_GetFlagStatus(UART0, UART_FLAG_RX_DATA_AVA) == SET)
42.         {
43.             UART_ReceiveData(UART0, &tmp, 1);
44.         }
45.         break;
46.     }
47.     case UART_INT_ID_LINE_STATUS:
48.     {
49.         .....
50.     }
51. }

```

14. 检测接收数据标志 `receiveflg`，状态为 `TRUE` 时，设置 GDMA 通道 3 的 `BufferSize` 为接收数据总量，使能 GDMA 通道 3 传输，将 `receiveflg` 状态设置为 `false`。

```

1.     while (1)
2.     {
3.         .....
4.         if (receiveflg)
5.         {
6.             GDMA_SetBufferSize(DMA_TX_CHANNEL, receive_offset);
7.             GDMA_Cmd(DMA_TX_CHANNEL_NUM, ENABLE);
8.             .....
9.             receiveflg = false;
10.        }
11.    }

```

15. 当 UART 通过 GDMA 传输发送数据时，GDMA 搬运数据到(&(UART0->RB_THR))，当 GDMA 发送数据总传输完成时，触发 `GDMA_INT_Transfer` 中断，进入中断处理函数 `GDMA0_Channel3_Handler`。

- 1) 打印 GDMA 搬运 UART 接收数据信息。
- 2) 失能 GDMA 通道 3 传输。

3) 清除 GDMA 通道 3 全部中断挂起。

```
1. void GDMA0_Channel2_Handler (void)
2. {
3.     DBG_DIRECT("UART_TX_GDMA_Handler");
4.     GDMA_Cmd(DMA_TX_CHANNEL_NUM, DISABLE);
5.     GDMA_ClearAllTypeINT(DMA_TX_CHANNEL_NUM);
6. }
```

5.5.3 实验现象

使用串口调试助手发送不定长数据到芯片，通过 GDMA 中断和 UART 中断收到数据，再通过 GDMA 搬运接收数据到 PC 端，串口调试助手收到芯片回的数据，在 DebugAnalyser 工具上，打印接收数据以及 GDMA 搬运数据完成信息。

Realtek Confidential

6 定时器和脉冲宽度调制 (TIM&PWM)

TIM/PWM 功能以及特点如下：

- 一共有 6 个 Timer，提供给用户可编程的 Timer 为 TIM2-TIM5
- 每个 Timer 支持时钟分频
- 可设定为 Free run/user define 模式
- 每一路 Timer 都支持 PWM 模式
- PWM 互补输出以及四区设定仅 PWM2 支持
- 死区时钟可更改为 5MHz
- TIM2-TIM5 时钟源增加且 PLL CLK 选项

专用硬件设备：逻辑分析仪

专用测试软件：DSview V1.1.2

6.1 脉冲宽度调制输出 (PWM)

本程序实现了 TIM (PWM) 外设的几种功能，如下。

- 使用 TIM2，实现输出周期为 1s、占空比为 50% 的 PWM 功能，为了便于观察，使用 P0_1 (LED0) 作为 PWM 输出，实现 LED0 的闪烁。
- 实现 PWM 互补输出功能，使用 P0_2 作为 PWM_P 输出，使用 P2_2 作为 PWM_N 输出。当设置死区后，PWM_P 和 PWM_N 在输出高电平时会比 PWM 延迟一个死区时长，可切换死区时钟源到 5M。
- 实现 PWM 互补 Bypass 功能，使用 P0_2 作为 PWM_P 输出，使用 P2_2 作为 PWM_N 输出。当设置 Bypass 功能后，PWM_P 和 PWM_N 输出波形完全互补。

工程目录：\HoneyComb\sdk\board\evb\io_sample\TIM\PWM

6.1.1 硬件设计

硬件连接：P0_1→LED0，P0_2→LED1 或 P2_2→LED1。

在 EVB 上，连接 P0_1 和 LED0，观察 LED0 的输出；如需观察 P0_2 或 P2_2 的输出，连接 P0_2 和 LED1 或 P2_2 和 LED1，观察 LED1 的输出。

6.1.2 软件流程

1. 引脚定义。

```
1.  #define PWM_OUT_PIN          P0_1
2.  #define PWM_OUT_P_PIN        P0_2
3.  #define PWM_OUT_N_PIN        P2_2
```

2. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、无内部上拉、输出使能、输出高。
3. 配置 PINMUX：分配引脚分别为 timer_pwm2、PWM2_P、PWN2_N 功能。

```
1.  void board_pwm_init(void)
2.  {
3.      Pad_Config(PWM_OUT_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
4.                  PAD_PULL_NONE, PAD_OUT_ENABLE, PAD_OUT_HIGH);
5.      Pad_Config(PWM_OUT_P_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
6.                  PAD_PULL_NONE, PAD_OUT_ENABLE, PAD_OUT_HIGH);
7.      Pad_Config(PWM_OUT_N_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
8.                  PAD_PULL_NONE, PAD_OUT_ENABLE, PAD_OUT_HIGH);
9.      Pinmux_Config(PWM_OUT_PIN, PWM_OUT_PIN_PINMUX);
10.     Pinmux_Config(PWM_OUT_P_PIN, PWM_OUT_P_PIN_PINMUX);
11.     Pinmux_Config(PWM_OUT_N_PIN, PWM_OUT_N_PIN_PINMUX);
12. }
```

4. 使能 TIMER 时钟。
5. 初始化 TIMER 外设。
 - 1) TIM_PWM_En 设置为 PWM_ENABLE，即 PWM 模式。
 - 2) TIM_Mode 设置为 TIM_Mode_UserDefine，即用户定义模式。
 - 3) 设置 TIM_PWM_High_Count 和 TIM_PWM_Low_Count，用户可以根据需求分配高电平周期值和低电平周期值。
 - 4) PWM_Deazone_Size 设置为 PWM_DEAD_ZONE_SIZE，即配置双路互补输出死区的大小。
 - 5) PWMDeadZone_En 设置为 DEADZONE_ENABLE，使能 PWM 互补输出（带死区功能）功能。
 - 6) 可以根据需求设定是否开启切换死区时钟以及是否开启 Bypass 功能。
6. 使能 TIM2。

```
1.  void driver_pwm_init(void)
2.  {
3.      RCC_PeriphClockCmd(APBPeriph_TIMER, APBPeriph_TIMER_CLOCK, ENABLE);
4.      TIM_TimeBaseInitTypeDef TIM_InitStruct;
```

```
5.     TIM_StructInit(&TIM_InitStruct);
6.     TIM_InitStruct.TIM_PWM_En  = PWM_ENABLE;
7.     TIM_InitStruct.TIM_Mode    = TIM_Mode_UserDefine;
8.     TIM_InitStruct.TIM_PWM_High_Count  = PWM_HIGH_COUNT;
9.     TIM_InitStruct.TIM_PWM_Low_Count   = PWM_LOW_COUNT;
10.    TIM_InitStruct.PWM_Deazone_Size    = PWM_DEAD_ZONE_SIZE;
11.    TIM_InitStruct.PWMDeadZone_En      = DEADZONE_ENABLE;
12.    TIM_TimeBaseInit(TIM2, &TIM_InitStruct);
13.    TIM_Cmd(TIM2, ENABLE);
14. }
```

6.1.3 实验现象

LED0 每秒闪烁，当连接 P0_2 和 LED1 时，LED1 与 LED0 同相闪烁，当连接 P2_2 和 LED1，LED1 与 LED0 反相闪烁。

可以外接逻辑分析仪抓取 P0_1，P0_2，P2_2 引脚的输出波形。

Realtek Confidential

6.2 定时器中断

使用 TIM4, 实现定时 1s 的定时功能。定时时间到, 触发中断, 在中断处理函数中翻转 P0_1 (LED0), 实现 LED0 每秒闪烁。

工程目录: \HoneyComb\sdk\board\evb\io_sample\TIM\Timer_interrupt

6.2.1 硬件设计

硬件连接: P0_1→LED0。

在 EVB 上, 连接 P0_1 和 LED0, 观察 LED0 的输出。

6.2.2 软件流程

1. 配置 PAD、PINMUX, 设置 P0_1 为 GPIO 输出功能。(详情参考: GPIO→Output_led)
2. 初始化 TIMER 外设。
 - 1) TIM_PWM_En 设置为 PWM_DISABLE, 即 TIMER 模式。
 - 2) TIM_Period 设置为 TIMER_PERIOD (40000000-1), 即定时 1s。
 - 3) TIM_Mode 设置为 TIM_Mode_UserDefine, 即用户定义模式。
3. 清除 TIM4 中断; 使能 TIM4 中断; 使能 TIM4。

```

1. void driver_timer_init(void)
2. {
3.     .....
4.     TIM_InitStruct.TIM_PWM_En = PWM_DISABLE;
5.     TIM_InitStruct.TIM_Period = TIMER_PERIOD ;
6.     TIM_InitStruct.TIM_Mode = TIM_Mode_UserDefine;
7.     TIM_TimeBaseInit(TIMER_NUM, &TIM_InitStruct);
8.     .....
9.     TIM_ClearINT(TIMER_NUM);
10.    TIM_INTConfig(TIMER_NUM, ENABLE);
11.    TIM_Cmd(TIMER_NUM, ENABLE);
12. }
```

4. TIM4 定时时间到, 触发中断, 进入中断处理函数 Timer4_Handler。
 - 1) 清除 TIM4 中断, 失能 TIM4。
 - 2) 判断当前 LED 状态, 翻转 LED0 电平。
 - 3) 重新使能 TIM4。

```
1. void Timer4_Handler(void)
2. {
3.     TIM_ClearINT(TIM4);
4.     TIM_Cmd(TIM4, DISABLE);
5.     if (!LED_Status)
6.     {
7.         GPIO_SetBits(GPIO_PIN_OUTPUT);
8.         LED_Status = 1;
9.     }
10.    else
11.    {
12.        GPIO_ResetBits(GPIO_PIN_OUTPUT);
13.        LED_Status = 0;
14.    }
15.    TIM_Cmd(TIM4, ENABLE);
16. }
```

6.2.3 实验现象

在 EVB 上，连接 P0_1 和 LED0，LED0 每秒闪烁。

6.3 数字信号任意波形输出

使用 TIM2，定时周期 10us，定时产生 GDMA 请求，实现 GDMA 搬运数据到 GPIO 寄存器。通过改变 GPIO 输出高低电平时间(GDMA 源数据搬运次数*TIM 周期)，模拟动态改变 PWM 波形占空比的功能。

例如：如果定义数据：{0xFFFFFFFF, 0xFFFFFFFF, 0x0}，则 GPIO 输出高电平时间为 2*10us=20us，GPIO 输出低电平时间为 1*10us=10us，PWM 周期为 30us，占空比 2/3。0xFFFFFFFF 表示高电平，0x0 表示低电平。

工程目录：\HoneyComb\sdk\board\evb\io_sample\TIM\GDMA+PWM+GPIO

6.3.1 硬件设计

硬件连接：P2_2→逻辑分析仪 Pin。

EVB 外接逻辑分析仪，连接 P2_2 和 Pin，观察 GPIO 输出的波形图。

6.3.2 软件流程

1. 配置 PAD、PINMUX，设置 P2_2 为 GPIO 输出。
2. 初始化 GPIO 外设：GPIO_ControlMode 设置为 GPIO_HARDWARE_MODE，即硬件模式，用于 GDMA 搬数据到 GPIO。

```

1.  void driver_gpio_init(void)
2.  {
3.      .....
4.      GPIO_InitStruct.GPIO_ControlMode = GPIO_HARDWARE_MODE;
5.      .....
6.  }
```

3. 配置 PAD、PINMUX，设置 P2_3 为 timer_pwm5，初始化 TIMER 外设。
 - 1) 设置 PWM 模式。
 - 2) 配置高、低电平周期为 5us，即输出周期为 10us、占空比为 50%的 PWM。
 - 3) 用户定义模式。

```

1.  void driver_pwm_init(void)
2.  {
3.      .....
4.      TIM_InitStruct.TIM_PWM_En = PWM_ENABLE;
5.      TIM_InitStruct.TIM_PWM_High_Count = PWM_HIGH_COUNT;
6.      TIM_InitStruct.TIM_PWM_Low_Count = PWM_LOW_COUNT;
```

```

7.     TIM_InitStruct.TIM_Mode = TIM_Mode_UserDefine;
8.     .....
9.     }

```

4. 初始化 GDMA 外设。

- 1) 使用 GDMA 通道 1。
- 2) GDMA 的传输方向为内存到外设传输；源端地址为 GPIO_Ctl_AutoReload，数组中定义 32 位数据，对应 GPIO0~GPIO31；目的端地址为 0x40011200。（固定使用）
- 3) 设置目的端 GDMA_Handshake_TIM5。
- 4) 使能 Multi-block 传输，根据需求设置 GDMA_Multi_Block_Mode，如果需要 GPIO 反复输出，则自动加载源端地址（GPIO_Ctl_AutoReload）；否则自动加载 LLI 结构体中源端地址（&(GPIO_Ctl_LLI[i]）。

5. 使能 GDMA 通道 1 GDMA 总传输完成中断(GDMA_INT_Transfer)，使能 GDMA 传输，使能 TIM5。

```

1.  void driver_gdma_multiblock_init(void)
2.  {
3.      .....
4.      GDMA_InitStruct.GDMA_ChannelNum    = GDMA_CHANNEL_NUM;
5.      GDMA_InitStruct.GDMA_DIR            = GDMA_DIR_MemoryToPeripheral;
6.      .....
7.      GDMA_InitStruct.GDMA_SourceAddr     = (uint32_t)GPIO_Ctl_AutoReload;
8.      GDMA_InitStruct.GDMA_DestinationAddr = (uint32_t)(0x40011200);
          //vendor gpio reg address
9.      GDMA_InitStruct.GDMA_DestHandshake  = GDMA_Handshake_TIM5;
10.     GDMA_InitStruct.GDMA_Multi_Block_En  = 1;
11.     GDMA_InitStruct.GDMA_Multi_Block_Struct = (uint32_t)GDMA_LLIStruct;
12.     #if (GPIO_OUT_REPEAT == 1)
13.         GDMA_InitStruct.GDMA_Multi_Block_Mode = LLI_WITH_AUTO_RELOAD_SAR;
14.     #else
15.         GDMA_InitStruct.GDMA_Multi_Block_Mode = LLI_TRANSFER;
16.     #endif
17.     for (int i = 0; i < GDMA_LLI_SIZE; i++)
18.     {
19.         GDMA_LLIStruct[i].SAR = (uint32_t)(amp;GPIO_Ctl_LLI[i]));
20.         GDMA_LLIStruct[i].DAR = (uint32_t)(0x40011200);
21.         .....
22.     }
23.     GDMA_INTConfig(GDMA_CHANNEL_NUM, GDMA_INT_Transfer, ENABLE);
24.     GDMA_Cmd(GDMA_CHANNEL_NUM, ENABLE);
25.     TIM_Cmd(PWM_TIMER_NUM, ENABLE);
26. }

```

6. TIM5 输出 PWM，每个周期 GDMA 搬运数据到 0x40011200，当 GDMA 总传输完成时，触发 GDMA_INT_Transfer 中断，进入中断处理函数 GDMA_Channel_Handler。清除 GDMA 通道 1 GDMA_INT_Transfer 中断挂起位。

```
1. void GDMA_Channel_Handler(void)
2. {
3.     GDMA_ClearINTPendingBit(GDMA_CHANNEL_NUM, GDMA_INT_Transfer);
4. }
```

6.3.3 实验现象

搬运 GPIO_Ctl_LLI[24]数据时，使用逻辑分析仪检测 GPIO 输出，得到预期占空比 PWM 波形。

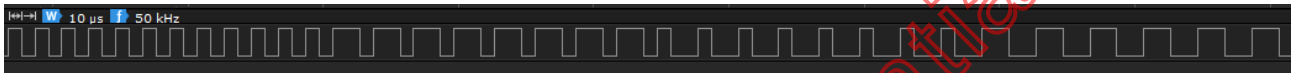


图 6-1 GPIO 输出 PWM 波形

搬运 GPIO_Ctl_AutoReload[24]数据时，使用逻辑分析仪检测 GPIO 输出，得到预期占空比 PWM 波形。

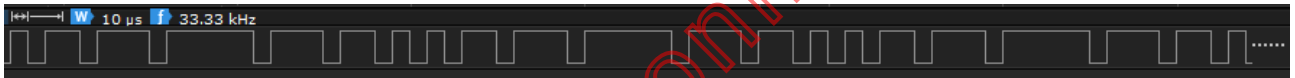


图 6-2 GPIO 输出 PWM 波形

7 增强定时器（ENHTIM）

7.1 GPIO 触发锁存

使用 ENH_TIM0，实现 GPIO 触发锁存器状态，并记录计数器计数的功能。P2_4 设定为 GPIO 输出引脚，设定 P2_4 引脚电平随时间变化不断翻转；P2_2 设定为 GPIO 输入引脚，同时将该引脚设定为 ENHTIM 的计数触发引脚，将 P2_2 与 P2_4 引脚连接，P2_2 电压变化会触发计数，进入 ENH_TIM0 中断，并打印触发时计数器数值。

工程目录：\HoneyComb\sdk\board\evb\io_sample\TIM_ENHANCE\latch_gpio

7.1.1 硬件设计

硬件连接：P2_4→P2_2。

在 EVB 上，连接 P2_4 和 P2_2。

7.1.2 软件流程

1. 配置 PAD、PINMUX，设置 P2_4 为 GPIO 输出功能，设置 P2_2 为 GPIO 输入功能。
（详情参考：GPIO→Output_led）
2. 初始化 ENHTIM 外设。
 - 1) ENHTIM_ClockDiv 设置为 ENHTIM_CLOCK_DIVIDER_1，即一分频模式。
 - 2) ENHTIM_Mode 设置为 ENHTIM_MODE_FreeRun，即自由运行模式。
 - 3) ENHTIM_LatchCountEn[2] 设置为 ENABLE，即开启 GPIO 触发锁存功能。
 - 4) ENHTIM_LatchCountTrigger[2] 设置为 TRIGGER_RISING_EDGE，即上升沿触发。
 - 5) ENHTIM_LatchCount2Thd 设置为 3，即触发中断 FIFO 阈值为 3。
 - 6) ENHTIM_LatchTriggerPad 设置为 GPIO_INPUT_PIN_0。
 - 7) ENHTIM_TimerGPIOTriggerEn 设置为 ENABLE，即使能 GPIO 触发计时功能。
3. 配置 ENHTIM 中断；使能 ENHTIM 计时中断以及达到 FIFO 阈值中断；使能 ENHTIM 外设。

```

1. void driver_enhance_timer_init (void)
2. {
3.     .....
4.     ENHTIM_InitStruct.ENHTIM_ClockDiv      = ENHTIM_CLOCK_DIVIDER_1;
5.     ENHTIM_InitStruct.ENHTIM_Mode          = ENHTIM_MODE_FreeRun;
6.     /* Only enhtim_ ENHTIM_LatchCountEn[2] latch triggered by GPIO. */

```

```

7.   ENHTIM_InitStruct.ENHTIM_LatchCountEn[2]    = ENHTIM_LATCH_COUNT_ENABLE;
8.   ENHTIM_InitStruct.ENHTIM_LatchCountTrigger[2] =
      ENHTIM_LATCH_TRIGGER_RISING_EDGE;
9.   ENHTIM_InitStruct.ENHTIM_LatchCount2Thd      = 3;
10.  ENHTIM_InitStruct.ENHTIM_LatchTriggerPad     = GPIO_INPUT_PIN_0;
11.  ENHTIM_InitStruct.ENHTIM_TimerGPIOTriggerEn  = ENABLE;
12.
13.  ENHTIM_Init(ENHTIMER_NUM, &ENHTIM_InitStruct);
14.  .....
15.  ENHTIM_ClearINTPendingBit(ENHTIMER_NUM, ENHTIM_INT_TIM);
16.  ENHTIM_INTConfig(ENHTIMER_NUM, ENHTIM_INT_TIM, ENABLE);
17.
18.  ENHTIM_ClearINTPendingBit(ENHTIMER_NUM, ENHTIM_INT_LATCH_CNT2_FIFO_FULL);
19.  ENHTIM_INTConfig(ENHTIMER_NUM, ENHTIM_INT_LATCH_CNT2_FIFO_FULL, ENABLE);
20.
21.  ENHTIM_ClearINTPendingBit(ENHTIMER_NUM, ENHTIM_INT_LATCH_CNT2_FIFO_THD);
22.  ENHTIM_INTConfig(ENHTIMER_NUM, ENHTIM_INT_LATCH_CNT2_FIFO_THD, ENABLE);
23.
24.  ENHTIM_Cmd(ENHTIMER_NUM, ENABLE);
25. }

```

4. ENHTIM 定时时间到或者 GPIO 输入引脚上升沿触发次数达到 FIFO 设定阈值，触发中断，进入中断处理函数 Enhanced_Timer0_Handler。

- 1) 判断中断状态位。
- 2) 若为 ENHTIM_INT_TIM，即 ENHTIM 定时中断，打印中断状态，清除中断标志位。
- 3) 若为 ENHTIM_INT_LATCH_CNT2_FIFO_THD，即 GPIO 上升沿触发次数达到设定 FIFO 阈值，产生中断，打印中断信息，打印及 FIFO 中数据数量和计数器当前数值。

```

1.  void Enhanced_Timer0_Handler (void)
2.  {
3.      if (ENHTIM_GetINTStatus(ENH_TIM0, ENHTIM_INT_TIM))
4.      {
5.          APP_PRINT_INFO0("ENH_TIM0 ENHTIM_INT_TIM\r\n");
6.          //add user code here
7.          ENHTIM_ClearINTPendingBit(ENH_TIM0, ENHTIM_INT_TIM);
8.      }
9.      if (ENHTIM_GetINTStatus(ENH_TIM0, ENHTIM_INT_LATCH_CNT2_FIFO_THD))
10.     {
11.         APP_PRINT_INFO0("ENH_TIM0 ENHTIM_INT_LATCH_CNT2_FIFO_THD\r\n");
12.         ENHTIM_INTConfig(ENHTIMER_NUM, ENHTIM_INT_LATCH_CNT2_FIFO_THD, DISABLE);
13.         uint8_t length = ENHTIM_GetLatchCount2FIFOLength(ENH_TIM0);
14.         uint32_t data[4] = {0};
15.         ENHTIM_ReadLatchCount2FIFO(ENH_TIM0, data, length);

```

```
16.      /* Only for debugging, removed in actual application. */
17.      APP_PRINT_INFO1("ENH_TIM0 fifo length = %d\r\n", length);
18.      for (uint8_t i = 0; i < length; i++)
19.      {
20.          /* Only for debugging, removed in actual application. */
21.          APP_PRINT_INFO2("ENH_TIM0 data[%d] = 0x%x\r\n", i, data[i]);
22.      }
23.      //add user code here
24.      ENHTIM_ClearINTPendingBit(ENH_TIM0, ENHTIM_INT_LATCH_CNT2_FIFO_THD);
25.      ENHTIM_INTConfig(ENH_TIM0, ENHTIM_INT_LATCH_CNT2_FIFO_THD, ENABLE);
26.  }
27. }
```

7.1.3 实验现象

在 evb 上将 P2_2 和 P2_4 引脚连接，在 DebugAnalyser 工具上，不断打印 GPIO 触发锁存时的计数器数值。

8 直接内存存取控制器（GDMA）

8.1 单 Block 传输（Single Block）

使用 GDMA Single Block 功能（Block 最大传输数据为 65535），实现 memory 到 memory 搬运数据。在 DebugAnalyser 工具上，打印 GDMA 打印传输数据完成信息。

工程目录：\HoneyComb\sdk\board\evb\io_sample\GDMA\Mem2Mem_single_block

8.1.1 软件流程

1. app_task 初始化。

```
1. os_task_create(&app_task_handle, "app", app_main_task, 0,
    APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);
```

2. 在 app_main_task 中执行 driver_init 函数, 执行 driver_gdma_init 函数, 对 GDMA 外设进行初始化。

- 1) GDMA_ChannelNum 设置为 GDMA_CHANNEL_NUM, 使用 GDMA 通道 2。
- 2) GDMA_DIR 设置为 GDMA_DIR_MemoryToMemory, 即 GDMA 传输方向为内存到内存传输。
- 3) GDMA_BufferSize 设置为 GDMA_TRANSFER_SIZE, 即完成一次 GDMA 传输的数据个数为 200。（数据大小为: GDMA_TRANSFER_SIZE*GDMA_DataSize_Word/4 Bytes）
- 4) 设置 GDMA_SourceInc, 源端地址递增, 设置 GDMA_DestinationInc, 目的端地址递增。
- 5) 设置 GDMA_SourceDataSize, 源端的数据宽度为 32 位, 设置 GDMA_DestinationDataSize, 目的端的数据宽度为 32 位。
- 6) 设置 GDMA_SourceMsize, 源端单次 burst 传输的数据个数为 8, 设置 GDMA_DestinationMsize, 目的端单次 burst 传输的数据个数为 8。
- 7) GDMA_SourceAddr 设置为 GDMA_Send_Buf。
- 8) GDMA_DestinationAddr 设置为 GDMA_Recv_Buf。
- 9) 使能 GDMA 通道 2 总传输完成中断（GDMA_INT_Transfer）。

```
1. void driver_gdma_init(void)
2. {
3.     .....
4.     GDMA_InitTypeDef GDMA_InitStruct;
5.     GDMA_StructInit(&GDMA_InitStruct);
6.     GDMA_InitStruct.GDMA_ChannelNum      = GDMA_CHANNEL_NUM;
7.     GDMA_InitStruct.GDMA_DIR              = GDMA_DIR_MemoryToMemory;
8.     GDMA_InitStruct.GDMA_BufferSize      = GDMA_TRANSFER_SIZE;
```

```

9.     GDMA_InitStruct.GDMA_SourceInc      = DMA_SourceInc_Inc;
10.    GDMA_InitStruct.GDMA_DestinationInc  = DMA_DestinationInc_Inc;
11.    GDMA_InitStruct.GDMA_SourceDataSize  = GDMA_DataSize_Word;
12.    GDMA_InitStruct.GDMA_DestinationDataSize = GDMA_DataSize_Word;
13.    GDMA_InitStruct.GDMA_SourceMsize     = GDMA_Msize_8;
14.    GDMA_InitStruct.GDMA_DestinationMsize = GDMA_Msize_8;
15.    GDMA_InitStruct.GDMA_SourceAddr      = (uint32_t)GDMA_Send_Buf;
16.    GDMA_InitStruct.GDMA_DestinationAddr = (uint32_t)GDMA_Recv_Buf;
17.    GDMA_Init(GDMA_Channel, &GDMA_InitStruct);
18.    .....
19.    GDMA_INTConfig(GDMA_CHANNEL_NUM, GDMA_INT_Transfer, ENABLE);
20.    }

```

3. 开始任务调度。

```

1.    os_sched_start();

```

4. stack 准备好时, 执行 app_handle_dev_state_evt 函数, 执行 GDMA_Cmd 函数, 使能 GDMA 传输。

```

1.    GDMA_Cmd(GDMA_CHANNEL_NUM, ENABLE);

```

5. 数据从 GDMA_Send_Buf 搬到 GDMA_Recv_Buf 完成时, 触发 GDMA_INT_Transfer 中断, 进入中断处理函数 GDMA_Channel_Handler。

- 1) 失能 GDMA 通道 2 GDMA_INT_Transfer 中断。
- 2) 定义消息类型 IO_MSG_TYPE_GDMA, 发送 msg 给 task。
- 3) 清除 GDMA 通道 2 GDMA_INT_Transfer 中断挂起位。

```

1.    void GDMA_Channel_Handler(void)
2.    {
3.        GDMA_INTConfig(GDMA_CHANNEL_NUM, GDMA_INT_Transfer, DISABLE);
4.
5.        T_IO_MSG int_gdma_msg;
6.        int_gdma_msg.type = IO_MSG_TYPE_GDMA;
7.        int_gdma_msg.subtype = 0;
8.        if (false == app_send_msg_to_apptask(&int_gdma_msg))
9.            .....
10.        GDMA_ClearINTPendingBit(GDMA_CHANNEL_NUM, GDMA_INT_Transfer);
11.    }

```

6. app_main_task 循环检测 msg queue, 当有 msg 时, 执行 app_handle_io_msg(io_msg)函数。

7. 在 app_handle_io_msg 函数中, 判断消息类型为 IO_MSG_TYPE_GDMA, 执行 io_handle_gdma_msg。

- 1) 打印传输数据完成信息。
- 2) 判断 GDMA_Recv_Buf 与 GDMA_Send_Buf 是否相等，不等则打印错误数据。（搬运出错）

```

1. void io_handle_gdma_msg(T_IO_MSG *io_gdma_msg)
2. {
3.     APP_PRINT_INFO0("[io_gdma] io_handle_gdma_msg:
4.                     GDMA transfer data completion!");
5.     for (uint32_t i = 0; i < GDMA_TRANSFER_SIZE; i++)
6.     {
7.         if (GDMA_Send_Buf[i] != GDMA_Recv_Buf[i])
8.         {
9.             APP_PRINT_INFO2("[io_gdma] io_handle_gdma_msg:
10.                            Data transmission error!
11.                            GDMA_Send_Buf = %d,
12.                            GDMA_Recv_Buf = %d",
13.                            GDMA_Send_Buf[i],
14.                            GDMA_Recv_Buf[i]);
15.         }
16.     }
17. }

```

8.1.2 实验现象

在 DebugAnalyser 工具上，打印 GDMA 打印传输数据完成信息。

8.2 多 Block 传输（Multi Block）

当 GDMA 连续传输数据大于 65535 时，需使用 Multi Block 功能。使用 GDMA Multi Block 功能（仅 GDMA0、GDMA1 支持），实现 memory 到 memory 搬运数据。在 DebugAnalyser 工具上，打印 GDMA 搬运数据信息。

工程目录：\HoneyComb\sdk\board\evb\io_sample\GDMA\Mem2Mem_multi_block

8.2.1 软件流程

1. 根据需求，开启 GDMA block 传输完成中断（GDMA_INT_Block）或 GDMA 总传输完成中断（GDMA_INT_Transfer）。

```
1.  #define INT_TRANSFER          0
2.  #define INT_BLOCK            1
3.  #define GDMA_INTERRUPT_MODE  INT_BLOCK //INT_TRANSFER
```

2. app_task 初始化。

```
1.  os_task_create(&app_task_handle,"app",app_main_task,0,
                APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);
```

3. 在 app_main_task 中执行 driver_init 函数，执行 driver_gdma_init 函数，对 GDMA 外设初始化。

- 1) 使用 GDMA 通道 1。
- 2) 传输方向为内存到内存传输，源端地址为 GDMA_Send_Buffer，目的端地址为 GDMA_Recv_Buffer。
- 3) 完成一次 GDMA 传输的数据个数为 200，源端地址递增，目的端地址递增，源端数据 8 位，目的端数据 8 位，源端单次 burst 传输数据个数为 1，目的端单次 burst 传输的数据个数为 1。
- 4) 设置每次 block 传输后自动加载 LLI 结构体中源地址与目的地址值。
- 5) 使能 Multi-block 传输。
- 6) 设置传输 LLI 类型结构体首地址。

4. 配置每次 block 传输后 LLI 结构体中源地址、目的地址、链表指针、控制寄存器。

5. 根据需求，使能 GDMA 通道 1 GDMA_INT_Block 中断或 GDMA_INT_Transfer 中断。

```
1.  void driver_gdma_init(void)
2.  {
3.      .....
4.      GDMA_InitStruct.GDMA_Multi_Block_Mode = GDMA_MULTIBLOCK_MODE;
```

```

5.     GDMA_InitStruct.GDMA_Multi_Block_En    = 1;
6.     GDMA_InitStruct.GDMA_Multi_Block_Struct = (uint32_t)GDMA_LLIStruct;
7.     for (uint32_t i = 0; i < GDMA_MULTIBLOCK_SIZE; i++)
8.     {
9.         if (i == GDMA_MULTIBLOCK_SIZE - 1)
10.        {
11.            GDMA_LLIStruct[i].SAR = (uint32_t)GDMA_Send_Buffer[i];
12.            GDMA_LLIStruct[i].DAR = (uint32_t)GDMA_Recv_Buffer[i];
13.            GDMA_LLIStruct[i].LLP = 0;
14.            GDMA_LLIStruct[i].CTL_LOW = BIT(0) | .....;
15.            GDMA_LLIStruct[i].CTL_HIGH = GDMA_InitStruct.GDMA_BufferSize;
16.        }
17.        else
18.        {
19.            GDMA_LLIStruct[i].SAR = (uint32_t)GDMA_Send_Buffer[i];
20.            GDMA_LLIStruct[i].DAR = (uint32_t)GDMA_Recv_Buffer[i];
21.            GDMA_LLIStruct[i].LLP = (uint32_t)&GDMA_LLIStruct[i + 1];
22.            GDMA_LLIStruct[i].CTL_LOW = BIT(0) | .....;
23.            GDMA_LLIStruct[i].CTL_HIGH = GDMA_InitStruct.GDMA_BufferSize;
24.        }
25.    }
26.    .....
27.    #if (GDMA_INTERRUPT_MODE == INT_TRANSFER)
28.        GDMA_INTConfig(GDMA_CHANNEL_NUM, GDMA_INT_Transfer, ENABLE);
29.    #elif (GDMA_INTERRUPT_MODE == INT_BLOCK)
30.        GDMA_INTConfig(GDMA_CHANNEL_NUM, GDMA_INT_Block, ENABLE);
31.    #endif
32. }

```

6. 开始任务调度。

```

1.     os_sched_start();

```

7. stack 准备好时, 执行 app_handle_dev_state_evt 函数, 执行 GDMA_Cmd 函数, 使能 GDMA 传输。

```

1.     GDMA_Cmd(GDMA_CHANNEL_NUM, ENABLE);

```

➤ 如果开启 GDMA_INT_Block 中断

1. 当一次 block 传输完成时, 触发 GDMA_INT_Block 中断, 进入中断处理函数 GDMA_Channel_Handler。

1) 失能 GDMA 通道 1 GDMA_INT_Block 中断。

2) 定义消息类型 IO_MSG_TYPE_GDMA，保存数据，发送 msg 给 task。

```

1. void GDMA_Channel_Handler(void)
2. {
3.     GDMA_INTConfig(GDMA_CHANNEL_NUM, GDMA_INT_Block, DISABLE);
4.
5.     T_IO_MSG int_gdma_msg;
6.     int_gdma_msg.type = IO_MSG_TYPE_GDMA;
7.     int_gdma_msg.subtype = 0;
8.     int_gdma_msg.u.buf = (void *)&GDMA_INT_Block_Counter;
9.     if (false == app_send_msg_to_apptask(&int_gdma_msg))
10.        .....
11. }
```

2. app_main_task 循环检测 msg queue，当有 msg 时，执行 app_handle_io_msg(io_msg)函数。

3. 在 app_handle_io_msg 函数中，判断消息类型为 IO_MSG_TYPE_GDMA，执行 io_handle_gdma_msg。

- 1) 打印 block 传输次数和传输数据完成信息。
- 2) 判断 GDMA_Recv_Buf 与 GDMA_Send_Buf 是否相等，不等则打印错误数据。（搬运出错）
- 3) 使能 GDMA 通道 1 GDMA_INT_Block 中断，直到 GDMA_MULTIBLOCK_SIZE 次 block 传输完成。

```

1. void io_handle_gdma_msg(T_IO_MSG *io_gdma_msg)
2. {
3.     uint8_t *p_buf = io_gdma_msg->u.buf;
4.     APP_PRINT_INFO1("[io_gdma] io_handle_msg:
5.                     GDMA block%d transfer data completion!", *p_buf);
6.     for (uint32_t j = 0; j < GDMA_TRANSFER_SIZE; j++)
7.     {
8.         if (GDMA_Send_Buffer[*p_buf][j] != GDMA_Recv_Buffer[*p_buf][j])
9.         {
10.             APP_PRINT_INFO2("[io_gdma]io_handle_msg:
11.                             Data transmission error!
12.                             GDMA_Send_Buffer = %d,
13.                             GDMA_Recv_Buffer = %d",
14.                             GDMA_Send_Buffer[*p_buf][j],
15.                             GDMA_Recv_Buffer[*p_buf][j]);
16.         }
17.         GDMA_Recv_Buffer[*p_buf][j] = 0;
18.     }
19.     GDMA_INT_Block_Counter++;
20.     if (GDMA_INT_Block_Counter < GDMA_MULTIBLOCK_SIZE)
```

```

15.     {
16.         GDMA_INTConfig(GDMA_CHANNEL_NUM, GDMA_INT_Block, ENABLE);
17.     }
18.     else
19.     {
20.         GDMA_INT_Block_Counter = 0;
21.     }
22. }

```

➤ 如果开启 GDMA 通道 1 GDMA_INT_Transfer 中断

1. 当总传输完成时，触发 GDMA_INT_Transfer 中断，进入中断处理函数 GDMA_Channel_Handler。

- 1) 失能 GDMA 通道 1 GDMA_INT_Transfer 中断。
- 2) 定义消息类型 IO_MSG_TYPE_GDMA，发送 msg 给 task。
- 3) 清除 GDMA 通道 1 GDMA_INT_Transfer 中断挂起位。

```

1.     void GDMA_Channel_Handler(void)
2.     {
3.         GDMA_INTConfig(GDMA_CHANNEL_NUM, GDMA_INT_Transfer, DISABLE);
4.
5.         T_IO_MSG int_gdma_msg;
6.         int_gdma_msg.type = IO_MSG_TYPE_GDMA;
7.         int_gdma_msg.subtype = 0;
8.         if (false == app_send_msg_to_apptask(&int_gdma_msg))
9.             .....
10.        GDMA_ClearINTPendingBit(GDMA_CHANNEL_NUM, GDMA_INT_Transfer);
11.    }

```

2. app_main_task 循环检测 msg queue，当有 msg 时，执行 app_handle_io_msg(io_msg)函数。

3. 在 app_handle_io_msg 函数中，判断消息类型为 IO_MSG_TYPE_GDMA，执行 io_handle_gdma_msg。

- 1) 打印传输数据完成信息。
- 2) 判断 GDMA_Recv_Buf 与 GDMA_Send_Buf 是否相等，不等则打印错误数据。（搬运出错）

```

1.     void io_handle_gdma_msg(T_IO_MSG *io_gdma_msg)
2.     {
3.         APP_PRINT_INFO0("[io_gdma] io_handle_gdma_msg:
4.                         GDMA transfer data completion!");
5.         for (uint32_t i = 0; i < GDMA_MULTIBLOCK_SIZE; i++)
6.         {
7.             for (uint32_t j = 0; j < GDMA_TRANSFER_SIZE; j++)
8.             {

```

```
8.      if (GDMA_Send_Buffer[i][j] != GDMA_Recv_Buffer[i][j])
9.      {
10.         APP_PRINT_INFO2("[io_gdma]io_handle_gdma_msg:
                                Data transmission error!
                                GDMA_Send_Buffer = %d,
                                GDMA_Recv_Buffer = %d",
                                GDMA_Send_Buffer[i][j],
                                GDMA_Recv_Buffer[i][j]);
11.     }
12.     GDMA_Recv_Buffer[i][j] = 0;
13. }
14. }
15. }
```

8.2.2 实验现象

在 DebugAnalyser 工具上，打印 GDMA 搬运数据信息。

Realtek Confidential

9 模数转换器（ADC）

专用硬件设备：直流稳压源

ADC 的详细说明请参考 *RTL8762E ADC 应用实例*^[5]。

9.1 单次采样模式——轮询

使用 ADC 外设的 one shot 模式进行电压检测，以轮询方式检测 P2_2 输入电压，并在 DebugAnalyser 工具上，打印 16 个 ADC 通道的采样数据和采样电压。

工程目录：\HoneyComb\sdk\board\evb\io_sample\ADC\OneShotMode_Polling

9.1.1 硬件设计

硬件连接：P2_2→外部电压输入。

EVB 外接直流稳压源，连接 P2_2 和直流稳压源。P2_2 配置为模电电压输入引脚，若 ADC 采样模式配置为 divide 模式，输入电压范围为 0~3.3V；若配置为 bypass 模式，输入电压范围为 0~0.9V。

9.1.2 软件流程

1. 通过宏定义，配置 ADC 采样模式为 divide mode 或 bypass mode。

```
1.  #define ADC_DIVIDE_MODE           0
2.  #define ADC_BYPASS_MODE           1
3.  #define ADC_MODE_DIVIDE_OR_BYPASS ADC_DIVIDE_MODE
```

2. 引脚定义。

```
1.  #define ADC_SAMPLE_PIN_0          P2_2
```

3. 执行 adc_demo 函数，初始化电压换算公式函数。

```
1.  void adc_demo(void)
2.  {
3.      adc_k_status = ADC_CalibrationInit();
4.      .....
5.  }
```

4. 配置 PAD: 设置引脚、SW 模式、PowerOn、无内部上拉、输出失能、输出低等。

```

1. void board_adc_init(void)
2. {
3.     Pad_Config(ADC_SAMPLE_PIN_0, PAD_SW_MODE, PAD_IS_PWRON,
4.               PAD_PULL_NONE, PAD_OUT_DISABLE, PAD_OUT_LOW);
5. }

```

5. 使能 ADC 时钟。

6. 初始化 ADC 外设。

- 1) adcSamplePeriod 设为 255, 默认值。
- 2) schIndex[0]~schIndex[15]设为 EXT_SINGLE_ENDED(ADC_SAMPLE_CHANNEL_0), 即 16 个 ADC 通道均选择 ADC 单端模式的引脚 P2_2; schIndex[0]~schIndex[15]对应的 bitmap 为 0xFFFF。
- 3) adcPowerAlwaysOnCmd 设为 ADC_POWER_ALWAYS_ON_ENABLE, 即模拟 power 常开。

7. 若配置为 bypass 模式, 使能相应通道的高阻态模式。

```

1. void driver_adc_init(void)
2. {
3.     RCC_PeriphClockCmd(APBPeriph_ADC, APBPeriph_ADC_CLOCK, ENABLE);
4.     ADC_InitTypeDef ADC_InitStruct;
5.     ADC_StructInit(&ADC_InitStruct);
6.     ADC_InitStruct.adcSamplePeriod = 255;
7.     for (uint8_t i = 0; i < 16; i++)
8.     {
9.         ADC_InitStruct.schIndex[i] = EXT_SINGLE_ENDED(ADC_SAMPLE_CHANNEL_0);
10.    }
11.    ADC_InitStruct.bitmap = 0xFFFF;
12.    ADC_InitStruct.adcPowerAlwaysOnCmd = ADC_POWER_ALWAYS_ON_ENABLE;
13.    ADC_Init(ADC, &ADC_InitStruct);
14.    #if (ADC_MODE_DIVIDE_OR_BYPASS == ADC_BYPASS_MODE)
15.        ADC_BypassCmd(ADC_SAMPLE_CHANNEL_0, ENABLE);
16.    #endif
17. }

```

8. 使能 ADC 单次转换模式。

```

1. void adc_demo(void)
2. {
3.     .....
4.     ADC_Cmd(ADC, ADC_One_Shot_Mode, ENABLE);
5. }

```

9. 循环执行 `adc_sample_demo` 函数，每次转换完成，重新使能 ADC 单次转换模式。

```

1.  while (1)
2.  {
3.      DBG_DIRECT("[ADC] Polling reads adc sample data !");
4.      adc_sample_demo();
5.      for (uint32_t i = 500000; i > 0; i--);
6.      ADC_Cmd(ADC, ADC_One_Shot_Mode, ENABLE);
7.  }
```

10. `adc_sample_demo` 实现 ADC 数据采样，计算电压，打印信息。

- 1) 循环判断 ADC 单次转换完成中断状态，RESET 时继续判断，SET 时 ADC 单次转换完成。
- 2) 清除 ADC 单次转换完成中断挂起位。
- 3) 读 `schIndex[0]~schIndex[15]` 采样数据。
- 4) 根据采样数据计算出采样电压。（电压计算方法随着配置不同的 ADC 模式而不同）
- 5) 打印 16 个 ADC 通道的采样数据和采样电压。

```

1.  adc_sample_demo(void)
2.  {
3.      .....
4.      while (ADC_GetIntFlagStatus(ADC, ADC_INT_ONE_SHOT_DONE) == RESET);
5.      ADC_ClearINTPendingBit(ADC, ADC_INT_ONE_SHOT_DONE);
6.      for (uint8_t i = 0; i < 16; i++)
7.      {
8.          sample_data[i] = ADC_ReadByScheduleIndex(ADC, ADC_Schedule_Index_0 + i);
9.      }
10.     for (uint8_t i = 0; i < 16; i++)
11.     {
12.         #if (ADC_MODE_DIVIDE_OR_BYPASS == ADC_BYPASS_MODE)
13.             sample_voltage[i] = ADC_GetVoltage(
14.                 BYPASS_SINGLE_MODE, (int32_t)sample_data[i], &error_status);
15.         #else
16.             sample_voltage[i] = ADC_GetVoltage(
17.                 DIVIDE_SINGLE_MODE, (int32_t)sample_data[i], &error_status);
18.         #endif
19.         .....
20.         DBG_DIRECT("[ADC]adc_sample_demo:
21.             ADC one shot mode sample data_%-4d = %d,
22.             voltage_%-4d = %fmV ",
23.             i, sample_data[i], i, sample_voltage[i]);
24.     }
```

9.1.3 实验现象

在 DebugAnalyser 工具上，打印 16 个 ADC 通道的采样数据和采样电压。

Realtek Confidential

9.2 单次采样模式——中断

使用 ADC 外设的 one shot 模式进行电压检测，开启 ADC 单次采样完成中断设置，检测 P2_2 输入电压，检测结束后进入中断，并在 DebugAnalyser 工具上，打印 16 个通道的采样数据和采样电压。

工程目录：\HoneyComb\sdk\board\evb\io_sample\ADC\OneShotMode

9.2.1 硬件设计

硬件连接：P2_2→外部电压输入。

EVB 外接直流稳压源，连接 P2_2 和直流稳压源。P2_2 配置为模电电压输入引脚，若 ADC 采样模式配置为 divide 模式，输入电压范围为 0~3.3V；若配置为 bypass 模式，输入电压范围为 0~0.9V。

9.2.2 软件流程

1. 初始化电压换算公式函数，初始化 ADC 全局数据。

```
1. void global_data_adc_init(void)
2. {
3.     adc_k_status = ADC_CalibrationInit();
4.     .....
5.     memset(&ADC_Global_Data, 0, sizeof(ADC_Global_Data));
6. }
```

2. 配置 PAD，设置 P2_2 为 ADC 功能。

3. app_task 初始化。

```
1. os_task_create(&app_task_handle,"app",app_main_task,0,
                APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);
```

4. 在 app_main_task 中执行 driver_init 函数，执行 driver_adc_init 函数，对 ADC 外设进行初始化。

- 1) ADC 通道 schIndex[0]选择 ADC 单端模式的引脚 P2_2，schIndex[0]对应的 bitmap 为 0x0001。
- 2) 使能 ADC 单次转换完成中断（ADC_INT_ONE_SHOT_DONE）。

```
1. void driver_adc_init(void)
2. {
3.     .....
4.     ADC_InitStruct.schIndex[0] = EXT_SINGLE_ENDED(ADC_SAMPLE_CHANNEL_0);
5.     ADC_InitStruct.bitmap      = 0x01;
```



```

6.      .....
7.      ADC_INTConfig(ADC, ADC_INT_ONE_SHOT_DONE, ENABLE);
8.      }

```

5. 开始任务调度。

```

1.      os_sched_start();

```

6. 在 stack 准备好时，执行 app_handle_dev_state_evt 函数，执行 adc_sample_start。

- 1) 执行 ADC_Cmd 函数。
- 2) 使能 ADC 单次采样模式。

```

1.      void adc_sample_start(void)
2.      {
3.          ADC_Cmd(ADC, ADC_ONE_SHOT_MODE, ENABLE);
4.      }

```

7. 当 ADC 单次转换完成时，触发 ADC_INT_ONE_SHOT_DONE 中断，进入中断处理函数 ADC_Handler。

8. 判断 ADC 单次转换完成中断状态为 SET 时。

- 1) 清除 ADC_INT_ONE_SHOT_DONE 中断挂起位。
- 2) 读 schIndex[0]采样数据。
- 3) 定义消息类型 IO_MSG_TYPE_ADC，保存数据，发送 msg 给 task。

```

1.      void ADC_Handler(void)
2.      {
3.          if (ADC_GetIntFlagStatus(ADC, ADC_INT_ONE_SHOT_DONE) == SET)
4.          {
5.              ADC_ClearINTPendingBit(ADC, ADC_INT_ONE_SHOT_DONE);
6.              sample_data = ADC_ReadRawData(ADC, ADC_Schedule_Index_0);
7.
8.              T_IO_MSG int_adc_msg;
9.              int_adc_msg.type = IO_MSG_TYPE_ADC;
10.             int_adc_msg.subtype = 0;
11.             ADC_Global_Data.RawDataLen = 1;
12.             ADC_Global_Data.RawData[0] = sample_data;
13.             int_adc_msg.u.buf = (void*)(ADC_Global_Data.RawData);
14.             if (false == app_send_msg_to_apptask(&int_adc_msg))
15.                 .....
16.         }
17.     }

```

9. app_main_task 循环检测 msg queue，当有 msg 时，利用 app_handle_io_msg 函数处理。
10. 在 app_handle_io_msg 函数中，判断消息类型为 IO_MSG_TYPE_ADC，执行 io_handle_adc_msg 函数，执行 io_adc_voltage_calculate。
 - 1) 从 msg 中取出采样数据。
 - 2) 根据采样数据计算出采样电压。（电压计算方法随着配置不同的 ADC 模式而不同）
 - 3) 打印采样数据和采样电压。

```

1.  static void io_adc_voltage_calculate(T_IO_MSG *io_adc_msg)
2.  {
3.      .....
4.      ADC_Data_TypeDef *p_buf = io_adc_msg->u.buf;
5.      sample_data_len = p_buf->RawDataLen;
6.      for (uint8_t i = 0; i < sample_data_len; i++)
7.      {
8.          sample_data = p_buf->RawData[i];
9.          DBG_DIRECT("io_adc_voltage_calculate: raw_data = 0x%X", sample_data);
10.         #if (ADC_DATA_HW_AVERAGE )
11.             sample_data = (p_buf->RawData[i] & 0x3FFC) >> 2;
12.             DBG_DIRECT("io_adc_voltage_calculate: sample_data = %d", sample_data);
13.         #if endif
14.     }

```

9.2.3 实验现象

在 DebugAnalyser 工具上，打印采样数据和采样电压。

9.3 单次、差分采样模式

使用 ADC 外设的 one shot 模式进行差分电压检测，以中断的方式检测 P2_2 和 P2_3 输入差分电压，并在 DebugAnalyser 工具上，打印采样数据和采样电压。

工程目录：\HoneyComb\sdk\board\evb\io_sample\ADC\OneShotMode_DifferentialMode

9.3.1 硬件设计

硬件连接：P2_2→外部电压输入 P 端，P2_3→外部电压输入 N 端。

EVB 外接直流稳压源，连接 P2_2、P2_3 和直流稳压源。P2_2 和 P2_3 配置为电压输入引脚，若 ADC 采样模式配置为 divide 模式，输入电压范围为 0~3.3V；若配置为 bypass 模式，输入电压范围为 0~0.9V。

9.3.2 软件流程

1. 初始化电压换算公式函数，初始化 ADC 全局数据。

```
1. void global_data_adc_init(void)
2. {
3.     adc_k_status = ADC_CalibrationInit();
4.     .....
5.     memset(&ADC_Global_Data, 0, sizeof(ADC_Global_Data));
6. }
```

2. 配置 PAD，设置 P2_2、P2_3 为 ADC 功能。

3. app_task 初始化。

```
1. os_task_create(&app_task_handle, "app", app_main_task, 0,
                APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);
```

4. 在 app_main_task 中执行 driver_init 函数，执行 driver_adc_init 函数，对 ADC 外设进行初始化。

- 1) 使能定时器触发采样功能。
- 2) ADC 通道 schIndex[0]、schIndex[1]选择 ADC 差分模式的引脚 P2_2、P2_3，schIndex[0]和 schIndex[1]对应的 bitmap 为 0x0003。
- 3) 若配置 bypass 模式，需将差分引脚 P2_2、P2_3 设置为 bypass 模式。
- 4) 使能 ADC 单次转换完成中断（ADC_INT_ONE_SHOT_DONE）。
- 5) 使能 ADC 单次转换模式。

```

1.  void driver_adc_init(void)
2.  {
3.      .....
4.      ADC_InitStruct.timerTriggerEn = ENABLE;
5.      ADC_InitStruct.schIndex[0]   = EXT_DIFFERENTIAL(ADC_SAMPLE_CHANNEL_0);
6.      ADC_InitStruct.schIndex[1]   = EXT_DIFFERENTIAL(ADC_SAMPLE_CHANNEL_1);
7.      ADC_InitStruct.bitmap        = 0x03;
8.      .....
9.      #if (ADC_MODE_DIVIDE_OR_BYPASS == ADC_BYPASS_MODE)
10.         uint8_t temp = ADC_SAMPLE_CHANNEL_0 & 0x06;
11.         switch (temp)
12.         {
13.             case 0x0:
14.                 ADC_BypassCmd(0, ENABLE);
15.                 ADC_BypassCmd(1, ENABLE);
16.                 break;
17.         }
18.     #endif
19.     .....
20.     ADC_INTConfig(ADC, ADC_INT_ONE_SHOT_DONE, ENABLE);
21.     ADC_Cmd(ADC, ADC_One_Shot_Mode, ENABLE);
22. }

```

5. 开始任务调度。

```

1.  os_sched_start();

```

6. 定时时间到，触发 ADC 采样，当 ADC 单次转换完成时，触发 ADC_INT_ONE_SHOT_DONE 中断，进入中断处理函数 ADC_Handler：判断 ADC 单次转换完成中断状态为 SET 时。

- 1) 清除 ADC_INT_ONE_SHOT_DONE 中断挂起位。
- 2) 读 schIndex[0]、schIndex[1] 采样数据。
- 3) 定义消息类型 IO_MSG_TYPE_ADC，保存数据，发送 msg 给 task。

（详情参考：ADC→One short mode）

```

1.  void ADC_Handler(void)
2.  {
3.      .....
4.  }

```

7. app_main_task 循环检测 msg queue，当有 msg 时，利用 app_handle_io_msg 函数处理。

8. 在 app_handle_io_msg 函数中，判断消息类型为 IO_MSG_TYPE_ADC，执行 io_handle_adc_msg 函数，执行 io_adc_voltage_calculate。

- 1) 从 msg 中取采样数据。
 - 2) 根据采样数据计算出采样电压。
 - 3) 打印采样数据和采样电压。
- (详情参考: ADC→One short mode)

```
1.  static void io_adc_voltage_calculate(T_IO_MSG *io_adc_msg)
2.  {
3.      .....
4.  }
```

9.3.3 实验现象

在 DebugAnalyser 工具上, 打印采样数据和采样电压。

Realtek Confidential

9.4 单次采样模式——唤醒深度低功耗状态（DLPS）

使用 ADC 外设的 one shot 模式进行内部 VBAT 电压的检测。

同时，系统处于 IDLE 状态时，会自动进入 DLPS；当引脚 P4_0 输入电平为低电平时，唤醒系统。

工程目录：\HoneyComb\sdk\board\evb\io_sample\ADC\DLPS

9.4.1 硬件设计

硬件连接：P4_0→外部电平输入，低电平唤醒系统。

系统处于 DLPS 状态下，使用杜邦线短接 P4_0 和 GND，唤醒系统。

ADC 通道选择 ADC 内部 VBAT 模式，对内部 VBAT 电压进行采样。

9.4.2 软件流程

1. 开启 DLPS 功能。

```
1.  #define DLPS_EN 1
2.  #define USE_UART_DLPS 1
3.  #define USE_USER_DEFINE_DLPS_EXIT_CB 1
4.  #define USE_USER_DEFINE_DLPS_ENTER_CB 1
```

2. 初始化 ADC 全局数据。

```
1.  void global_data_adc_init(void)
2.  {
3.      IO_ADC_DLPS_Enter_Allowed = true;
4.      ADC_DATA_Length = 0;
5.      memset(ADC_DATA_Buffer, 0, sizeof(ADC_DATA_Buffer));
6.  }
```

3. 配置 PAD、PINMUX：设置 P4_0 为 GPIO 功能。

4. 执行 pwr_mgr_init 函数，初始化电源管理。

- 1) 注册用户进入 DLPS 回调函数 app_enter_dlps_config，注册用户退出 DLPS 回调函数 app_exit_dlps_config。
- 2) 注册硬件控制回调函数 DLPS_IO_EnterDlpsCb 和 DLPS_IO_ExitDlpsCb，进入 DLPS 会保存 CPU、PINMUX、Peripheral 等，退出 DLPS 会恢复 CPU、PINMUX、Peripheral 等。
- 3) 设置 DLPS 模式。
- 4) 设置唤醒 DLPS 方式为 ADC_DLPS_WAKEUP_PIN 低电平唤醒。

```

1.  void pwr_mgr_init(void)
2.  {
3.      .....
4.      DLPS_IORegUserDlpsEnterCb(app_enter_dlps_config);
5.      DLPS_IORegUserDlpsExitCb(app_exit_dlps_config);
6.      DLPS_IORegister();
7.      lps_mode_set(LPM_DLPS_MODE);
8.      System_WakeUpPinEnable(ADC_DLPS_WAKEUP_PIN, PAD_WAKEUP_POL_LOW, 0);
9.      .....
10. }
```

5. 在 app_enter_dlps_config 中，执行 io_adc_dlps_enter 函数，设置 ADC_DLPS_WAKEUP_PIN 为 SW 模式，设置唤醒 DLPS 方式为 ADC_DLPS_WAKEUP_PIN 低电平唤醒。

```

1.  void io_adc_dlps_enter(void)
2.  {
3.      .....
4.      Pad_ControlSelectValue(ADC_DLPS_WAKEUP_PIN, PAD_SW_MODE);
5.      System_WakeUpPinEnable(ADC_DLPS_WAKEUP_PIN, PAD_WAKEUP_POL_LOW, 0);
6.  }
```

6. 在 app_exit_dlps_config 中，执行 io_adc_dlps_exit 函数，设置 ADC_DLPS_WAKEUP_PIN 为 PINMUX 模式。

```

1.  void io_adc_dlps_exit(void)
2.  {
3.      .....
4.      Pad_ControlSelectValue(ADC_DLPS_WAKEUP_PIN, PAD_PINMUX_MODE);
5.  }
```

7. app_task 初始化。

```

1.  os_task_create(&app_task_handle,"app",app_main_task,0,
                APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);
```

8. 在 app_main_task 中执行 driver_init 函数，执行 driver_adc_init 函数，对 ADC 外设进行初始化。

- 1) ADC_SampleTime 设为 255，默认值。
- 2) ADC_SchIndex[0]设为 INTERNAL_VBAT_MODE，即 ADC 通道选择 ADC 内部 VBAT 模式。
- 3) ADC_Bitmap 为 0x01。
- 4) ADC_DataAvgEn 设为 ADC_DATA_AVERAGE_ENABLE，使能单次采样模式平均值的计算。
- 5) ADC_DataAvgSel 设为 ADC_DATA_AVERAGE_OF_4，计算平均值的个数为 4 个。
- 6) ADC_PowerAlwaysOnEn 设为 ADC_POWER_ALWAYS_ON_ENABLE，即模拟 power 常开。

7) 使能 ADC 单次转换完成中断（ADC_INT_ONE_SHOT_DONE）。

9. 初始化中断：设置中断优先级，并使能 ADC_IRQn 通道。

```

1.  void driver_adc_init(void)
2.  {
3.      .....
4.      ADC_InitStruct.ADC_SampleTime    = 255;
        /* (n + 1) cycle of 10MHz, n = 0~255 or n = 2048~14591 */
5.      ADC_InitStruct.ADC_SchIndex[0]    = INTERNAL_VBAT_MODE;
6.      ADC_InitStruct.ADC_Bitmap        = 0x01;
7.      ADC_InitStruct.ADC_DataAvgEn      = ADC_DATA_AVERAGE_ENABLE;
8.      ADC_InitStruct.ADC_DataAvgSel     = ADC_DATA_AVERAGE_OF_4;
9.      ADC_InitStruct.ADC_PowerAlwaysOnEn = ADC_POWER_ALWAYS_ON_ENABLE;
10.     ADC_Init(ADC, &ADC_InitStruct);
11.     ADC_INTConfig(ADC, ADC_INT_ONE_SHOT_DONE, ENABLE);
12.     NVIC_InitTypeDef NVIC_InitStruct;
13.     NVIC_InitStruct.NVIC_IRQChannel = ADC_IRQn;
14.     NVIC_InitStruct.NVIC_IRQChannelPriority = 3;
15.     NVIC_InitStruct.NVIC_IRQChannelCmd = ENABLE;
16.     NVIC_Init(&NVIC_InitStruct);
17. }

```

10. 初始化 GPIO 外设。

- 1) GPIO_Pin 设置为 GPIO_PIN_INPUT。
- 2) GPIO_Mode 设置为 GPIO_Mode_IN。
- 3) GPIO_ITTrigger 设置为 GPIO_INT_Trigger_EDGE，即边沿触发中断。
- 4) GPIO_ITPolarity 设置为 GPIO_INT_POLARITY_ACTIVE_LOW，即下降沿触发。
- 5) ITDebounce 设置为 GPIO_INT_DEBOUNCE_ENABLE，即使能 GPIO 中断去抖动。
- 6) GPIO_DebounceTime 设置为 64，即去抖时间设置为 64ms。

11. 初始化中断：设置中断优先级，并使能 GPIO_PIN_INPUT_IRQN 通道。

12. 取消屏蔽 GPIO 外部中断；使能 GPIO 中断。

```

1.  void driver_gpio_init(void)
2.  {
3.      .....
4.      GPIO_InitTypeDef GPIO_InitStruct;
5.      GPIO_StructInit(&GPIO_InitStruct);
6.      GPIO_InitStruct.GPIO_Pin    = GPIO_PIN_INPUT;
7.      GPIO_InitStruct.GPIO_Mode    = GPIO_Mode_IN;
8.      GPIO_InitStruct.GPIO_ITTrigger = GPIO_INT_Trigger_EDGE;

```



```

9.     GPIO_InitStruct.GPIO_ITPolarity = GPIO_INT_POLARITY_ACTIVE_LOW;
10.    GPIO_InitStruct.GPIO_ITDebounce = GPIO_INT_DEBOUNCE_ENABLE;
11.    GPIO_InitStruct.GPIO_DebounceTime = 10; /* unit:ms , can be 1~64 ms */
12.    GPIO_Init(&GPIO_InitStruct);
13.
14.    NVIC_InitTypeDef NVIC_InitStruct;
15.    NVIC_InitStruct.NVIC_IRQChannel = GPIO_PIN_INPUT_IRQN;
16.    NVIC_InitStruct.NVIC_IRQChannelPriority = 3;
17.    NVIC_InitStruct.NVIC_IRQChannelCmd = ENABLE;
18.    NVIC_Init(&NVIC_InitStruct);
19.
20.    GPIO_MaskINTConfig(GPIO_PIN_INPUT, DISABLE);
21.    GPIO_INTConfig(GPIO_PIN_INPUT, ENABLE);
22. }

```

13. 开始任务调度。

```

1.     os_sched_start();

```

14. 当 ADC_DLPS_WAKEUP_PIN 接入低电平后，系统退出 DLPS 状态，触发 GPIO_PIN_INPUT 中断，进入中断处理函数 GPIO_Input_Handler。

- 1) 关 GPIO 中断，屏蔽 GPIO 中断。
- 2) 定义消息类型 IO_MSG_TYPE_GPIO，发送 msg 给 task。
- 3) 清除 GPIO 中断挂起位，取消屏蔽 GPIO 中断，使能 GPIO 中断。

```

1.     void GPIO_Input_Handler(void)
2.     {
3.         GPIO_INTConfig(GPIO_PIN_INPUT, DISABLE);
4.         GPIO_MaskINTConfig(GPIO_PIN_INPUT, ENABLE);
5.         T_IO_MSG int_gpio_msg;
6.
7.         int_gpio_msg.type = IO_MSG_TYPE_GPIO;
8.         int_gpio_msg.subtype = 0;
9.         if (false == app_send_msg_to_apptask(&int_gpio_msg))
10.            .....
11.        GPIO_ClearINTPendingBit(GPIO_PIN_INPUT);
12.        GPIO_MaskINTConfig(GPIO_PIN_INPUT, DISABLE);
13.        GPIO_INTConfig(GPIO_PIN_INPUT, ENABLE);
14.    }

```

15. 当 ADC 单次转换完成时，触发 ADC_INT_ONE_SHOT_DONE 中断，进入中断处理函数 ADC_Handler。

16. 判断 ADC 单次转换完成中断状态为 SET 时。

- 1) 清除 ADC_INT_ONE_SHOT_DONE 中断挂起位。
- 2) 读 ADC_SchIndex[0]采样数据。
- 3) 定义消息类型 IO_MSG_TYPE_ADC，保存数据，发送 msg 给 task。

（详情参考：ADC→One short mode）

```
1. void ADC_Handler(void)
2. {
3.     .....
4. }
```

17. app_main_task 循环检测 msg queue，当有 msg 时，执行 app_handle_io_msg(io_msg)函数。

18. 在 app_handle_io_msg 函数中，判断消息类型为 IO_MSG_TYPE_GPIO，执行 io_adc_sample_start 函数，使能 ADC 单次采样模式。

```
1. void io_adc_sample_start(void)
2. {
3.     ADC_Cmd(ADC, ADC_ONE_SHOT_MODE, ENABLE);
4. }
```

19. 在 app_handle_io_msg 函数中，判断消息类型为 IO_MSG_TYPE_ADC，执行 io_handle_adc_msg 函数，执行 io_adc_voltage_calculate。

- 1) 从 msg 中取采样数据。
 - 2) 根据采样数据计算出采样电压。
 - 3) 打印采样数据和采样电压。
- （详情参考：ADC→One short mode）

```
1. void io_adc_handle_msg(T_IO_MSG *io_adc_msg)
2. {
3.     .....
4. }
```

20. 系统处于 IDLE 状态时，会自动进入 DLPS。

21. ADC_DLPS_WAKEUP_PIN 接入低电平时，唤醒 DLPS，执行 System_Handler。

- 1) 清除 ADC_DLPS_WAKEUP_PIN 的唤醒中断挂起位。
- 2) 失能 DC_DLPS_WAKEUP_PIN 的唤醒功能。
- 3) IO_ADC_DLPS_Enter_Allowed 值设置为 false。

```
1. void System_Handler(void)
```

```
2.  {
3.    if (System_WakeUpInterruptValue(ADC_DLPS_WAKEUP_PIN) == SET)
4.    {
5.        APP_PRINT_INFO0("System_Handler");
6.        Pad_ClearWakeUpINTPendingBit(ADC_DLPS_WAKEUP_PIN);
7.        System_WakeUpPinDisable(ADC_DLPS_WAKEUP_PIN);
8.        IO_ADC_DLPS_Enter_Allowed = false;
9.    }
10. }
```

9.4.3 实验现象

系统默认进入 DLPS 状态，在 DLPS 状态下，P4_0 引脚接低电平唤醒系统，开启 ADC 检测内部 VBAT 电压模式，在 DebugAnalyser 工具上，打印采样数据。

Realtek Confidential

9.5 连续采样模式

使用 ADC 外设的 continuous 模式进行双通道电压检测，以中断的方式同时检测 P2_2 和 P2_3 输入电压，并在 DebugAnalyser 工具上，打印采样数据和采样电压。

工程目录：\HoneyComb\sdk\board\evb\io_sample\ADC\ContinuousMode

9.5.1 硬件设计

硬件连接：P2_2→外部电压输入，P2_3→外部电压输入。

EVB 外接直流稳压源，连接 P2_2、P2_3 和直流稳压源。P2_2、P2_3 配置为模电电压输入引脚，若 ADC 采样模式配置为 divide 模式，输入电压范围为 0~3.3V；若配置为 bypass 模式，输入电压范围为 0~0.9V。

9.5.2 软件流程

1. 初始化电压换算公式函数，初始化 ADC 全局数据。

```
1. void global_data_adc_init(void)
2. {
3.     adc_k_status = ADC_CalibrationInit();
4.     .....
5.     memset(&ADC_Global_Data, 0, sizeof(ADC_Global_Data));
6. }
```

2. 配置 PAD，设置 P2_2、P2_3 为 ADC 功能。

3. app_task 初始化。

```
1. os_task_create(&app_task_handle, "app", app_main_task, 0,
                APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);
```

4. 在 app_main_task 中执行 driver_init 函数，执行 driver_adc_init 函数，对 ADC 外设进行初始化。

- 1) ADC 通道 schIndex[0]、schIndex[1]选择 ADC 差分模式的引脚 P2_2、P2_3，schIndex[0]和 schIndex[1]对应的 bitmap 为 0x0003。
- 2) 设置连续采样率为 199，实际采样时间为 $(199+1)/10K = 20ms$ 。
- 3) 设置 FIFO 阈值为 16。
- 4) 使能 ADC FIFO 超过给定值中断（ADC_INT_FIFO_TH）。
- 5) 使能 ADC 连续转换模式。

```

1.  void driver_adc_init(void)
2.  {
3.      .....
4.      ADC_InitStruct.schIndex[0] = EXT_SINGLE_ENDED(ADC_SAMPLE_CHANNEL_0);
5.      ADC_InitStruct.schIndex[1] = EXT_SINGLE_ENDED(ADC_SAMPLE_CHANNEL_1);
6.      ADC_InitStruct.bitmap      = 0x03;
7.      ADC_InitStruct.adcSamplePeriod = ADC_CONTINUOUS_SAMPLE_PERIOD;
8.      ADC_InitStruct.adcFifoThd     = 16;
9.      .....
10.     ADC_INTConfig(ADC, ADC_INT_FIFO_TH, ENABLE);
11.     ADC_Cmd(ADC, ADC_Continuous_Mode, ENABLE);
12. }

```

5. 开始任务调度。

```

1.  os_sched_start();

```

6. 当 ADC FIFO 数目超过阈值时,触发 ADC_INT_FIFO_TH 中断,进入中断处理函数 ADC_Handler。

7. 判断 ADC FIFO 数目超过给定值的中断状态为 SET 时。

- 1) 失能 ADC 连续转换模式。
- 2) 获取 FIFO 长度及 FIFO 中的采样数据。
- 3) 清除 FIFO, 清除 ADC_INT_FIFO_TH 中断挂起位。
- 4) 定义消息类型 IO_MSG_TYPE_ADC, 保存数据, 发送 msg 给 task。

```

1.  void ADC_Handler(void)
2.  {
3.      if (ADC_GetIntFlagStatus(ADC, ADC_INT_FIFO_TH) == SET)
4.      {
5.          ADC_Cmd(ADC, ADC_Continuous_Mode, DISABLE);
6.          data_len = ADC_GetFifoLen(ADC);
7.          ADC_GetFifoData(ADC, sample_data, data_len);
8.          ADC_ClearFifo(ADC);
9.          ADC_ClearINTPendingBit(ADC, ADC_INT_FIFO_TH);
10.         T_IO_MSG int_adc_msg;
11.         int_adc_msg.type = IO_MSG_TYPE_ADC;
12.         int_adc_msg.subtype = 0;
13.         ADC_Global_Data.RawData[0] = data_len;
14.         for (uint8_t i = 0; i < data_len; i++)
15.         {
16.             ADC_Global_Data.RawData[i + 1] = sample_data[i];
17.         }
18.         int_adc_msg.u.buf = (void*)(ADC_Global_Data.RawData);

```

```

19.         if (false == app_send_msg_to_apptask(&int_adc_msg))
20.             .....
21.     }
22. }

```

8. app_main_task 循环检测 msg queue，当有 msg 时，利用 app_handle_io_msg 函数处理。
9. 在 app_handle_io_msg 函数中，判断消息类型为 IO_MSG_TYPE_ADC，执行 io_handle_adc_msg 函数，执行 io_adc_voltage_calculate。
 - 1) 从 msg 中取出采样数据。
 - 2) 根据采样数据计算出采样电压。（电压计算方法随着配置不同的 ADC 模式而不同）
 - 3) 打印采样数据和采样电压。
 - 4) 重新使能 ADC 连续转换模式。

```

1.  static void io_adc_voltage_calculate(T_IO_MSG *io_adc_msg)
2.  {
3.      .....
4.      uint16_t *p_buf = io_adc_msg->u.buf;
5.      sample_data_len = p_buf[0];
6.      for (uint8_t i = 0; i < sample_data_len; i++)
7.      {
8.          sample_data[i] = p_buf[i + 1];
9.      }
10.     for (uint8_t i = 0; i < sample_data_len; i++)
11.     {
12.         #if (ADC_MODE_DIVIDE_OR_BYPASS == ADC_BYPASS_MODE)
13.             sample_voltage[i] = ADC_GetVoltage(
14.                 BYPASS_SINGLE_MODE, (int32_t)sample_data[i], &error_status);
15.         #else
16.             sample_voltage[i] = ADC_GetVoltage(
17.                 DIVIDE_SINGLE_MODE, (int32_t)sample_data[i], &error_status);
18.         #endif
19.         APP_PRINT_INFO4("[io_adc] io_adc_voltage_calculate:
20.             ADC rawdata_%-4d = %d, voltage_%-4d = %dmV ",
21.                 i, sample_data[i], i, (uint32_t)sample_voltage[i]);
22.         ADC_Cmd(ADC, ADC_Continuous_Mode, ENABLE);
23.     }

```

9.5.3 实验现象

在 DebugAnalyser 工具上，打印采样数据和采样电压。

9.6 连续采样模式——GDMA 搬运采样数据

使用 ADC 外设的 continuous 模式进行双通道电压检测，以 GDMA 的方式同时检测 P2_2 和 P2_3 输入电压，并在 DebugAnalyser 工具上，打印采样数据和采样电压。

工程目录：\HoneyComb\sdk\board\evb\io_sample\ADC\ContinuousMode+GDMA

9.6.1 硬件设计

硬件连接：P2_2→外部电压输入，P2_3→外部电压输入。

EVB 外接直流稳压源，连接 P2_2、P2_3 和直流稳压源。P2_2、P2_3 配置为模电电压输入引脚，若 ADC 采样模式配置为 divide 模式，输入电压范围为 0~3.3V；若配置为 bypass 模式，输入电压范围为 0~0.9V。

9.6.2 软件流程

1. 初始化电压换算公式函数，初始化 ADC 全局数据。

```
1. void global_data_adc_init(void)
2. {
3.     adc_k_status = ADC_CalibrationInit();
4.     .....
5.     memset(&ADC_Global_Data, 0, sizeof(ADC_Global_Data));
6. }
```

2. 配置 PAD，设置 P2_2、P2_3 为 ADC 功能。

3. app_task 初始化。

```
1. os_task_create(&app_task_handle, "app", app_main_task, 0,
                APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);
```

4. 在 app_main_task 中执行 driver_init 函数，执行 driver_adc_init 函数，对 ADC 外设进行初始化。

- 1) ADC 通道 schIndex[0]、schIndex[1]选择 ADC 差分模式的引脚 P2_2、P2_3，schIndex[0]和 schIndex[1]对应的 bitmap 为 0x0003。
- 2) 采样周期为 20ms。
- 3) 设置 FIFO 阈值为 16。
- 4) 使能 ADC 误读中断（ADC_INT_FIFO_RD_ERR）。

```
1. void driver_adc_init(void)
2. {
```

```

3.      .....
4.      ADC_INTConfig(ADC, ADC_INT_FIFO_RD_ERR, ENABLE);
5.      .....
6.  }
```

5. 执行 driver_gdma_adc_init 函数，对 GDMA 外设进行初始化。

- 1) 使用 GDMA 通道 2。
- 2) GDMA 的传输方向为外设到内存传输。
- 3) GDMA_SourceAddr 设置为 ADC 的 FIFO。
- 4) GDMA_DestinationAddr 设置为 ADC_Recv_Buffer。
- 5) 使能 GDMA 通道 2 总传输完成中断（GDMA_INT_Transfer）。

```

1.  void driver_gdma_adc_init(void)
2.  {
3.      GDMA_InitStruct.GDMA_ChannelNum      = ADC_GDMA_CHANNEL_NUM;
4.      GDMA_InitStruct.GDMA_DIR              = GDMA_DIR_PeripheralToMemory;
5.      .....
6.      GDMA_InitStruct.GDMA_SourceAddr       = (uint32_t)&(ADC->FIFO);
7.      GDMA_InitStruct.GDMA_DestinationAddr  = (uint32_t)ADC_Recv_Buffer;
8.      .....
9.      GDMA_INTConfig(ADC_GDMA_CHANNEL_NUM, GDMA_INT_Transfer, ENABLE);
10.     .....
11. }
```

6. 开始任务调度。

```

1.  os_sched_start();
```

7. 在 stack 准备好时，执行 app_handle_dev_state_evt 函数。

- 1) 执行 GDMA_Cmd 函数：使能 GDMA 传输。
- 2) 执行 ADC_Cmd 函数：使能 ADC 连续转换模式。

```

1.  GDMA_Cmd(ADC_GDMA_CHANNEL_NUM, ENABLE);
2.  ADC_Cmd(ADC, ADC_Continuous_Mode, ENABLE);
```

8. 当 ADC FIFO 数据读取错误时，触发 ADC_INT_FIFO_RD_ERR 中断，进入中断处理函数 ADC_Handler，定义消息类型 IO_MSG_TYPE_ADC，消息子类型为 IO_MSG_ADC_FIFO_READ_ERR，发送 msg 给 task。

```

1.  void ADC_Handler(void)
2.  {
3.      if (ADC_GetIntFlagStatus(ADC, ADC_INT_FIFO_RD_ERR) == SET)
```



```

4.      {
5.          T_IO_MSG int_adc_msg;
6.          int_adc_msg.type = IO_MSG_TYPE_ADC;
7.          int_adc_msg.subtype = IO_MSG_ADC_FIFO_READ_ERR;
8.          if (false == app_send_msg_to_apptask(&int_adc_msg))
9.              .....
10.     }
11. }

```

9. 当 GDMA 搬运数据完成时，触发 GDMA_INT_Transfer 中断，进入 GDMA 中断处理函数 ADC_GDMA_Channel_Handler。

- 1) 失能 ADC 连续采样模式；清除 GDMA 通道 2 GDMA_INT_Transfer 中断挂起位。
- 2) 定义消息类型 IO_MSG_TYPE_GDMA，保存 ADC_Recv_Buffer 数据，发送 msg 给 task。

```

1.  void ADC_GDMA_Channel_Handler(void)
2.  {
3.      ADC_Cmd(ADC, ADC_Continuous_Mode, DISABLE);
4.      GDMA_ClearINTPendingBit(ADC_GDMA_CHANNEL_NUM, GDMA_INT_Transfer);
5.
6.      T_IO_MSG int_gdma_msg;
7.      int_gdma_msg.type = IO_MSG_TYPE_GDMA;
8.      int_gdma_msg.subtype = 0;
9.      int_gdma_msg.u.buf = (void*)(ADC_Recv_Buffer);
10.     if (false == app_send_msg_to_apptask(&int_gdma_msg))
11.         .....
12. }

```

10. app_main_task 循环检测 msg queue，当有 msg 时，利用 app_handle_io_msg 函数处理。

11. 在 app_handle_io_msg 函数中，判断消息类型为 IO_MSG_TYPE_ADC，执行 io_handle_adc_msg 函数，执行 io_handle_adc_error：打印 ADC 错误信息。

```

1.  static void io_handle_adc_error(T_IO_MSG *io_adc_msg)
2.  {
3.      .....
4.      case IO_MSG_ADC_FIFO_READ_ERR:
5.          {
6.              APP_PRINT_ERROR0("[io_adc]io_handle_adc_error:
                                IO_MSG_ADC_FIFO_READ_ERR!");
7.          }
8.      .....
9.  }

```

12. 在 `app_handle_io_msg` 函数中,判断消息类型为 `IO_MSG_TYPE_GDMA`,执行 `io_handle_gdma_msg` 函数,执行 `io_adc_voltage_calculate`。

- 1) 从 `msg` 中取出采样数据。
- 2) 根据采样数据计算出采样电压。(电压计算方法随着配置不同的 ADC 模式而不同)
- 3) 打印采样数据和采样电压。
- 4) 重新使能 GDMA 传输,使能 ADC 连续采样模式。

```

1.  static void io_adc_voltage_calculate(T_IO_MSG *io_adc_msg)
2.  {
3.      .....
4.      uint16_t *p_buf = io_adc_msg->u.buf;
5.      for (uint32_t i = 0; i < GDMA_TRANSFER_SIZE; i++)
6.      {
7.          sample_data[i] = p_buf[i];
8.      }
9.      for (uint32_t i = 0; i < GDMA_TRANSFER_SIZE; i++)
10.     {
11.         #if (ADC_MODE_DIVIDE_OR_BYPASS == ADC_BYPASS_MODE)
12.             sample_voltage[i] = ADC_GetVoltage(
13.                 BYPASS_SINGLE_MODE, (int32_t)sample_data[i], &error_status);
14.         #else
15.             sample_voltage[i] = ADC_GetVoltage(
16.                 DIVIDE_SINGLE_MODE, (int32_t)sample_data[i], &error_status);
17.         #endif
18.         .....
19.         APP_PRINT_INFO4("[io_adc]io_adc_voltage_calculate:
20.             ADC rawdata_%-4d = %d,
21.             voltage_%-4d = %dmV ",
22.                 i, sample_data[i],
23.                 i, (uint32_t)sample_voltage[i]);
24.     }
25.     for (uint32_t i = 0; i < 1000000; i++);
26.     GDMA_Cmd(ADC_GDMA_CHANNEL_NUM, ENABLE);
27.     ADC_Cmd(ADC, ADC_Continuous_Mode, ENABLE);
28. }

```

9.6.3 实验现象

在 DebugAnalyser 工具上,打印采样数据和采样电压。

10 串型外设接口（SPI）

专用硬件设备：GD25Q128 模块、APS6404L 模块

10.1 EEPROM 模式读取 ID

使用 SPI0 与 GD25Q128 以 EEPROM 模式进行数据传输。通过轮询方式获取芯片 ID，在 DebugAnalyser 工具上，打印 Flash ID 信息。

工程目录：\HoneyComb\sdk\board\evb\io_sample\SPI\Polling_eeprom

10.1.1 硬件设计

硬件连接：P4_0→CLK，P4_1→DO，P4_2→DI，P4_3→CS#。

EVB 外接 FM25Q16 模块，连接 P4_0 和 CLK，P4_1 和 DO，P4_2 和 DI，P4_3 和 CS#，同时连接 VDD 和模块的 VDD、WP#、HOLD#，连接 GND 和模块的 VSS。

10.1.2 软件流程

1. 引脚定义。

```
1.  #define SPI0_SCK_PIN    P4_0
2.  #define SPI0_MISO_PIN   P4_1
3.  #define SPI0_MOSI_PIN   P4_2
4.  #define SPI0_CS_PIN     P4_3
```

2. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、内部上拉、输出使能、输出高。

3. 配置 PINMUX：分配引脚分别为 SPI0_CLK_MASTER、SPI0_MO_MASTER、SPI0_MI_MASTER、SPI0_SS_N_0_MASTER 功能。

```
1.  void board_spi_init(void)
2.  {
3.      Pad_Config(SPI0_SCK_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
4.                  PAD_PULL_UP, PAD_OUT_ENABLE, PAD_OUT_HIGH);
5.      Pad_Config(SPI0_MOSI_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
6.                  PAD_PULL_UP, PAD_OUT_ENABLE, PAD_OUT_HIGH);
7.      Pad_Config(SPI0_MISO_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
8.                  PAD_PULL_UP, PAD_OUT_ENABLE, PAD_OUT_HIGH);
9.      Pad_Config(SPI0_CS_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
10.                 PAD_PULL_UP, PAD_OUT_ENABLE, PAD_OUT_HIGH);
```

```

7.     Pinmux_Config(SPI0_SCK_PIN, SPI0_CLK_MASTER);
8.     Pinmux_Config(SPI0_MOSI_PIN, SPI0_MO_MASTER);
9.     Pinmux_Config(SPI0_MISO_PIN, SPI0_MI_MASTER);
10.    Pinmux_Config(SPI0_CS_PIN, SPI0_SS_N_0_MASTER);
11.    }

```

4. app_task 初始化。

```

1.     os_task_create(&app_task_handle,"app",app_main_task,0,
                    APP_TASK_STACK_SIZE,APP_TASK_PRIORITY);

```

5. 在 app_main_task 中执行 driver_init 函数，执行 driver_spi_init 函数，对 SPI0 外设进行初始化。

- 1) SPI_Direction 设置为 SPI_Direction_EEPROM，即 SPI 的数据传输模式为 EEPROM 模式。
- 2) SPI_Mode 设置为 SPI_Mode_Master，即 SPI 的工作模式为主设备模式。
- 3) SPI_DataSize 设置为 SPI_DataSize_8b，即 SPI 的数据大小为 8 位帧结构。
- 4) SPI_CPOL 设置为 SPI_CPOL_High，即串行时钟的稳态为时钟悬空高。
- 5) SPI_CPHA 设置为 SPI_CPHA_2Edge，即位捕获的时钟活动沿为数据捕获于第二个时钟沿。
- 6) SPI_BaudRatePrescaler 设置为 100，即 SPI 外设的时钟分频系数为 100。
- 7) SPI_RxThresholdLevel 设置为 1，即发送 FIFO 的阈值为 2。
- 8) SPI_NDF 设置为 0，即读取的数据长度阈值为 1。
- 9) SPI_FrameFormat 设置为 SPI_Frame_Motorola，即 SPI 的数据传输格式为 Motorola 传输格式。

6. 使能 SPI0 外设。

```

1.     void driver_spi_init(void)
2.     {
3.         .....
4.         SPI_InitTypeDef SPI_InitStruct;
5.         SPI_StructInit(&SPI_InitStruct);
6.         SPI_InitStruct.SPI_Direction = SPI_Direction_EEPROM;
7.         SPI_InitStruct.SPI_Mode      = SPI_Mode_Master;
8.         SPI_InitStruct.SPI_DataSize  = SPI_DataSize_8b;
9.         SPI_InitStruct.SPI_CPOL      = SPI_CPOL_High;
10.        SPI_InitStruct.SPI_CPHA       = SPI_CPHA_2Edge;
11.        SPI_InitStruct.SPI_BaudRatePrescaler = 100;
12.        SPI_InitStruct.SPI_RxThresholdLevel = 2 - 1;
13.        SPI_InitStruct.SPI_NDF         = 1 - 1;
14.        SPI_InitStruct.SPI_FrameFormat = SPI_Frame_Motorola;
15.        SPI_Init(SPI0, &SPI_InitStruct);
16.        SPI_Cmd(SPI0, ENABLE);
17.    }

```

7. 开始任务调度。

```
1. os_sched_start();
```

8. 在 stack 准备好时，执行 app_handle_dev_state_evt 函数，执行 spi_demo。

- 1) 依次以 EEPROM 模式读取 DEVICE_ID、MF_DEVICE_ID 以及 JEDEC_ID 的数据信息。
- 2) 打印 Flash_ID 的类型、长度和 ID 数据。
- 3) 以 SPI_FLASH_READ_DATA 方式读取 Flash 中的数据信息并打印。
- 4) 以 SPI_FLASH_FAST_READ 方式读取 Flash 中的数据信息并打印。

```
1. void spi_demo(void)
2. {
3.     uint8_t flash_id[10] = {0};
4.     while (flash_id_type < 3)
5.     {
6.         spi_flash_read_id((Flash_ID_Type)flash_id_type, flash_id);
7.         APP_PRINT_INFO3("[io_spi] spi_demo:
8.                             flash_id_type = %d,
9.                             data_lenth = %d,
10.                            flash_id = %b ",
11.                            flash_id_type, flash_id[0],
12.                            TRACE_BINARY(flash_id[0],
13.                            &flash_id[1]));
14.         flash_id_type++;
15.         memset(flash_id, 0, sizeof(flash_id));
16.     }
17.     flash_id_type = 0;
18.     uint8_t read_data[100] = {0};
19.     spi_flash_read(SPI_FLASH_READ_DATA, 0x000101, read_data, 100);
20.     APP_PRINT_INFO1("[io_spi] spi_demo: read_data = %b,",
21.                     TRACE_BINARY(100, read_data));
22.     spi_flash_read(SPI_FLASH_FAST_READ, 0x000210, read_data, 100);
23.     APP_PRINT_INFO1("[io_spi] spi_demo: read_data = %b,",
24.                     TRACE_BINARY(100, read_data));
25. }
```

10.1.3 实验现象

在 DebugAnalyser 工具上，依此打印 Flash ID 以及 Flash 内部数据等信息。

若芯片型号为 GD25Q128，则读取的 DEVICE ID 为 18；MANU ID 为 C2-18；JEDEC ID 为 C2-20-19。

10.2 FullDuplex 模式读取 ID

使用 SPI0 与 GD25Q128 以 FullDuplex 模式进行数据传输。通过轮询方式获取芯片 ID，在 DebugAnalyser 工具上，打印 Flash ID 信息。

工程目录：\HoneyComb\sdh\board\evb_stack_img\io_sample\SPI\Polling_fullduplex

10.2.1 硬件设计

硬件连接：P4_0→CLK，P4_1→DO，P4_2→DI，P4_3→CS#。

EVb 外接 FM25Q16 模块，连接 P4_0 和 CLK，P4_1 和 DO，P4_2 和 DI，P4_3 和 CS#，同时连接 VDD 和模块的 VDD、WP#、HOLD#，连接 GND 和模块的 VSS。

10.2.2 软件流程

1. 引脚定义。

```
1.  #define SPI0_SCK_PIN      P4_0
2.  #define SPI0_MISO_PIN    P4_1
3.  #define SPI0_MOSI_PIN    P4_2
4.  #define SPI0_CS_PIN      P4_3
```

2. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、内部上拉、输出使能、输出高。
3. 配置 PINMUX：分配引脚分别为 SPI0_CLK_MASTER、SPI0_MO_MASTER、SPI0_MI_MASTER、SPI0_SS_N_0_MASTER 功能。（详情参考：SPI→EEPROM）
4. app_task 初始化。

```
1.  os_task_create(&app_task_handle,"app",app_main_task,0,
                APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);
```

5. 在 app_main_task 中执行 driver_init 函数，执行 driver_spi_init 函数，对 SPI0 外设进行初始化。
 - 1) SPI_Direction 设置为 SPI_Direction_Fullduplex，即 SPI 的数据传输模式为 FullDuplex 模式。
 - 2) SPI_Mode 设置工作模式为主设备模式；数据大小 8 位帧结构，串行时钟的稳态为时钟悬空高。
 - 3) 位捕获的时钟活动沿为数据捕获于第二个时钟沿，时钟分频系数为 100。
 - 4) SPI_RxThresholdLevel 设置为 2，即发送 FIFO 的阈值为 3。
 - 5) SPI_NDF 设置为 2，即读取的数据长度阈值为 3。
 - 6) SPI_FrameFormat 设置为 SPI_Frame_Motorola，即 SPI 的数据传输格式为 Motorola 传输格式。

6. 使能 SPI0 外设。

```

1.  void driver_spi_init(void)
2.  {
3.      .....
4.      SPI_InitTypeDef SPI_InitStruct;
5.      SPI_StructInit(&SPI_InitStruct);
6.      SPI_InitStruct.SPI_Direction = SPI_Direction_FullDuplex;
7.      SPI_InitStruct.SPI_Mode      = SPI_Mode_Master;
8.      SPI_InitStruct.SPI_DataSize  = SPI_DataSize_8b;
9.      SPI_InitStruct.SPI_CPOL      = SPI_CPOL_High;
10.     SPI_InitStruct.SPI_CPHA       = SPI_CPHA_2Edge;
11.     SPI_InitStruct.SPI_BaudRatePrescaler = 100;
12.     SPI_InitStruct.SPI_RxThresholdLevel = 3 - 1;
13.     SPI_InitStruct.SPI_NDF         = 3 - 1;
14.     SPI_InitStruct.SPI_FrameFormat = SPI_Frame_Motorola;
15.     SPI_Init(SPI0, &SPI_InitStruct);
16.     SPI_Cmd(SPI0, ENABLE);
17. }

```

7. 开始任务调度。

```

1.  os_sched_start();

```

8. 在 stack 准备好时，执行 app_handle_dev_state_evt 函数，执行 spi_demo。

- 1) 依次以 FullDuplex 模式读取 DEVICE_ID、MF_DEVICE_ID 以及 JEDEC_ID 的长度以及数据。
- 2) 打印 Flash_ID 的类型、长度和 ID 数据。
- 3) 以 SPI_FLASH_READ_DATA 方式读取 Flash 中的数据信息并打印。
- 4) 以 SPI_FLASH_FAST_READ 方式读取 Flash 中的数据信息并打印。

```

1.  void spi_demo(void)
2.  {
3.      uint8_t flash_id[10] = {0};
4.      while (flash_id_type < 3)
5.      {
6.          spi_flash_read_id((Flash_ID_Type)flash_id_type, flash_id);
7.          APP_PRINT_INFO3("[io_spi] spi_demo:
                                flash_id_type = %d,
                                data_lenth = %d,
                                flash_id = %b ",
                                flash_id_type, flash_id[0],
                                TRACE_BINARY(flash_id[0],
                                &flash_id[1]));

```

```
8.         flash_id_type++;
9.         memset(flash_id, 0, sizeof(flash_id));
10.    }
11.    flash_id_type = 0;
12.
13.    uint8_t read_data[100] = {0};
14.    spi_flash_read(SPI_FLASH_READ_DATA, 0x000101, read_data, 100);
15.    APP_PRINT_INFO1("[io_spi] spi_demo: read_data = %b",
                     TRACE_BINARY(100, read_data));
16.    spi_flash_read(SPI_FLASH_FAST_READ, 0x000210, read_data, 100);
17.    APP_PRINT_INFO1("[io_spi] spi_demo: read_data = %b",
                     TRACE_BINARY(100, read_data));
18. }
```

10.2.3 实验现象

在 DebugAnalyser 工具上，打印依此打印 Flash ID 以及读取数据信息。

若芯片型号为 GD25Q128，则读取的 DEVICE ID 为 18；MANU ID 为 C2-18；JEDEC ID 为 C2-20-19。

10.3 中断读取 ID

使用 SPI0 与 GD25Q128 进行数据传输。获取芯片 ID，通过 SPI0 中断向 app_task 发送消息事件，app_task 检测到消息事件，在 app 层解析消息，并执行用户程序：在 DebugAnalyser 工具上，打印 Flash ID 信息。

工程目录：\HoneyComb\sdk\board\evb\io_sample\SPI\Interrupt

10.3.1 硬件设计

硬件连接：P4_0→CLK，P4_1→DO，P4_2→DI，P4_3→CS#。

EVB 外接 FM25Q16 模块，连接 P4_0 和 CLK，P4_1 和 DO，P4_2 和 DI，P4_3 和 CS#，同时连接 VDD 和模块的 VDD、WP#、HOLD#，连接 GND 和模块的 VSS。

10.3.2 软件流程

1. 引脚定义。

```
1.  #define SPI0_SCK_PIN    P4_0
2.  #define SPI0_MISO_PIN   P4_1
3.  #define SPI0_MOSI_PIN   P4_2
4.  #define SPI0_CS_PIN     P4_3
```

2. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、内部上拉、输出使能、输出高。

3. 配置 PINMUX：分配引脚分别为 SPI0_CLK_MASTER、SPI0_MO_MASTER、SPI0_MI_MASTER、SPI0_SS_N_0_MASTER 功能。（详情参考：SPI→EEPROM）

4. app_task 初始化。

```
1.  os_task_create(&app_task_handle,"app",app_main_task,0,
                APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);
```

5. 在 app_main_task 中执行 driver_init 函数，执行 driver_spi_init 函数，对 SPI0 外设进行初始化。

- 1) SPI 的数据传输模式为 EEPROM 模式。
- 2) 工作模式为主设备模式。
- 3) 串行时钟的稳态为时钟悬空高。
- 4) 捕获的时钟活动沿为数据捕获于第二个时钟沿。
- 5) 时钟分频系数为 100。

- 6) 送 FIFO 的阈值为 1。
- 7) 读取的数据长度阈值为 1。
- 8) 数据传输格式为 Motorola 传输格式。
(详情参考: SPI→Polling_eeprom)

6. 使能 SPI0 外设; 使能 SPI0 接收缓冲区已满中断 (SPI_INT_RXF)。

```

1.  void driver_spi_init(void)
2.  {
3.      .....
4.      SPI_Cmd(SPI0, ENABLE);
5.      SPI_INTConfig(SPI0, SPI_INT_RXF, ENABLE);
6.      .....
7.  }
```

7. 开始任务调度。

```

1.  os_sched_start();
```

8. 在 stack 准备好时, 执行 app_handle_dev_state_evt 函数, 执行 spi_demo, 获取 DEVICE_ID 信息。

```

1.  void spi_demo(void)
2.  {
3.      .....
4.      spi_flash_read_id(DEVICE_ID, id);
5.  }
```

9. 当 SPI0 接收缓冲区已满时 (即收到 1bytes 数据), 触发 SPI_INT_RXF 中断, 进入中断处理函数 SPI0_Handler。

- 1) 清除 SPI_INT_RXF 中断挂起位。
- 2) 判断直到满足 FIFO 中数据长度等于 Flash_ID 长度。
- 3) 获取 FIFO 中 Flash_ID 数据长度。
- 4) 定义消息类型 IO_MSG_TYPE_SPI, 保存数据, 发送 msg 给 task。

```

1.  void SPI0_Handler(void)
2.  {
3.      .....
4.      SPI_ClearINTPendingBit(SPI0, SPI_INT_RXF);
5.
6.      while (SPI_GetRxFIFOLen(SPI0) < Flash_ID_Length);
7.      data_len = SPI_GetRxFIFOLen(SPI0);
8.      Flash_Data[0] = data_len;
```

```

9.     for (uint8_t i = 0; i < data_len; i++)
10.    {
11.        Flash_Data[1 + i] = SPI_ReceiveData(SPI0);
12.    }
13.
14.    T_IO_MSG int_spi_msg;
15.    int_spi_msg.type = IO_MSG_TYPE_SPI;
16.    int_spi_msg.subtype = 0;
17.    int_spi_msg.u.buf = (void*)(Flash_Data);
18.    if (false == app_send_msg_to_apptask(&int_spi_msg))
19.        .....
20.    }

```

10. app_main_task 循环检测 msg queue，当有 msg 时，执行 app_handle_io_msg(io_msg)函数。

11. 在 app_handle_io_msg 函数中，判断消息类型为 IO_MSG_TYPE_SPI，执行 io_spi_handle_msg。

- 1) 打印 Flash ID 信息。
- 2) 读取 Flash ID 的其他类型 ID 信息，通过中断方式依此打印获取 ID 信息。

```

1.     void io_spi_handle_msg(T_IO_MSG *io_spi_msg)
2.    {
3.        uint8_t *p_buf = io_spi_msg->u.buf;
4.        uint8_t data_lenth = p_buf[0];
5.        APP_PRINT_INFO2("[io_spi]io_spi_handle_msg: data_lenth = %d, data = %b ",
                           data_lenth, TRACE_BINARY(data_lenth, &p_buf[1]));
6.        for (uint8_t i = 0; i < data_lenth; i++)
7.        {
8.            APP_PRINT_INFO1("[io_spi]io_spi_handle_msg: data = 0x%x ", p_buf[i + 1]);
9.        }
10.       .....
11.       if (flash_id_type < 3)
12.       {
13.           spi_flash_read_id((Flash_ID_Type)flash_id_type, id);
14.       }
15.    }

```

10.3.3 实验现象

在 DebugAnalyser 工具上，依此打印 Flash ID 信息。

若芯片型号为 GD25Q128，则读取的 DEVICE ID 为 18；MANU ID 为 C2-18；JEDEC ID 为 C2-20-19。

10.4 SPI 与闪存（Flash）通信

使用 SPI0 与 GD25Q128 进行数据传输。获取芯片 ID，通过 SPI0 擦除 Flash 扇区数据，进行数据的读出和写入操作，并执行用户程序：在 DebugAnalyser 工具上，打印 Flash ID 以及 FLASH 内部数据信息的读取和写入信息。

Sample 中提供了软件控制 CS 线和硬件控制 CS 线的两种方式，当传输速率较快时，建议使用软件控制 CS 线。

硬件控制 CS 线工程目录：\HoneyComb\sdk\board\evb\io_sample\SPI\SPI_Flash

软件控制 CS 线工程目录：\HoneyComb\sdk\board\evb\io_sample\SPI\SPI_Software_CS

10.4.1 硬件设计

硬件连接：P4_0→CLK，P4_1→DO，P4_2→DI，P4_3→CS#。

EVB 外接 FM25Q16 模块，连接 P4_0 和 CLK，P4_1 和 DI，P4_2 和 DO，P4_3 和 CS#，同时连接 VDD 和模块的 VDD、WP#、HOLD#，连接 GND 和模块的 VSS。

10.4.2 软件流程

1. 引脚定义,配置 PAD,配置 PINMUX:分配引脚分别为 SPI0_CLK_MASTER、SPI0_MO_MASTER、SPI0_MI_MASTER、SPI0_SS_N_0_MASTER 功能。（详情参考：SPI→Polling_full duplex）

2. app_task 初始化。

```
1. os_task_create(&app_task_handle,"app",app_main_task,0,
APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);
```

3. 在 app_main_task 中执行 driver_init 函数，执行 driver_spi_init 函数，对 SPI0 外设进行初始化。

- 1) SPI 的数据传输模式为全双工模式。
- 2) 工作模式为主设备模式。
- 3) 数据大小为 8 位帧结构。
- 4) 串行时钟的稳态为时钟悬空高。
- 5) 位捕获的时钟活动沿为数据捕获于第二个时钟沿。
- 6) 时钟分频系数为 100。
- 7) 发送 FIFO 的阈值为 2。
- 8) 读取的数据长度阈值为 1。
- 9) 数据传输格式为 Motorola 传输格式。（详情参考：SPI→Polling_full duplex）

4. 使能 SPI0 外设。
5. 开始任务调度。

```
1. os_sched_start();
```

6. 在 stack 准备好时，执行 app_handle_dev_state_evt 函数，执行 spi_demo。

- 1) 通过 SPI0 读取 Flash_ID 信息并打印 ID 信息。
- 2) 通过 SPI0 擦除 Flash 扇区。
- 3) 通过 SPI0 读取 Flash 内部扇区内数据信息并打印数据信息。
- 4) 通过 SPI0 将数据写入 Flash。
- 5) 写入完成后，读取 Flash 写入信息并打印数据信息。

```
1. void spi_demo(void)
2. {
3.     .....
4.     for (uint16_t i = 0; i < 100; i++)
5.     {
6.         write_data[i] = i & 0xFF;
7.     }
8.
9.     spi_flash_read_id(flash_id);
10.    APP_PRINT_INFO1("[io_spi] spi_demo: flash_id = %b ", TRACE_BINARY(3, flash_id));
11.
12.    spi_flash_sector_erase(0x001000);
13.    APP_PRINT_INFO0("[io_spi] spi_demo: spi_flash_sector_erase done");
14.
15.    spi_flash_read(SPI_FLASH_FAST_READ, 0x001000, read_data, 100);
16.    APP_PRINT_INFO1("[io_spi] spi_demo: read_data = %b,",
17.                    TRACE_BINARY(100, read_data));
18.
19.    spi_flash_page_write(0x001000, write_data, 100);
20.
21.    spi_flash_read(SPI_FLASH_FAST_READ, 0x001000, read_data, 100);
22.    APP_PRINT_INFO1("[io_spi] spi_demo: read_data = %b,",
23.                    TRACE_BINARY(100, read_data));
24. }
```

10.4.3 实验现象

在 DebugAnalyser 工具上，打印 Flash ID 以及 Flash 内部数据的读取以及写入信息。

如果通信的芯片型号为 GD25Q128E，则读取 ID 为 C2-20-19。

10.5 GDMA 搬运 SPI 数据

使用 SPI0 与 GD25Q128 以 GDMA 搬运数据方式进行数据传输。通过 GDMA 搬运 Flash 内部数据信息，再通过 UART 发送数据到 PC 端，并执行用户程序：在 DebugAnalyser 工具上，打印读取 FLASH 内部数据信息。

Sample 中提供了 GDMA 单 Block 传输和多 Block 传输两种方式，本节提供了 GDMA_Multiblock 传输的详细说明。

工程目录：\HoneyComb\sdk\board\evb\io_sample\SPI\GDMA_Multiblock

10.5.1 硬件设计

硬件连接：P3_0→RX，P3_1→TX；P4_0→CLK，P4_1→DO，P4_2→DI，P4_3→CS#。

EVB 外接 FT232 模块，连接 P3_0 和 FT232 的 RX，P3_1 和 FT232 的 TX。

EVB 外接 GD25Q128 模块，连接 P4_0 和 CLK，P4_1 和 DO，P4_2 和 DI，P4_3 和 CS#，同时连接 VDD 和模块的 VDD、WP#、HOLD#，连接 GND 和模块的 VSS。

10.5.2 软件流程

1. 引脚定义。

```
1.  #define UART_TX_PIN      P3_0
2.  #define UART_RX_PIN      P3_1
```

2. 开启 UART_GDMA 功能。

```
1.  #define UART_GDMA_TX_ENABLE    EVB_ENABLE
2.  #define UART_GDMA_RX_ENABLE    EVB_ENABLE
```

3. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、内部上拉、输出失能、输出高。

4. 配置 PINMUX：分配引脚分别为 UART0_TX、UART0_RX 功能。

（详情参考：UART→Polling）

5. 引脚定义，配置 PAD，配置 PINMUX：分配引脚分别为 SPI0_CLK_MASTER、SPI0_MO_MASTER、SPI0_MI_MASTER、SPI0_SS_N_0_MASTER 功能。

（详情参考：SPI→EEPROM）

6. app_task 初始化。

```
1. os_task_create(&app_task_handle,"app",app_main_task,0,
                APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);
```

7. 在 app_main_task 中执行 driver_init 函数，执行 driver_UART_init 函数，对 UART0 外设进行初始化，默认波特率 115200，无奇偶校验，停止位 1 位，数据长度 8bits，FIFO 阈值为 16，空闲时间为 2byte 等。（详情参考：UART→Polling）

```
1. void driver_uart_init(void)
2. {
3.     .....
4. }
```

8. 开始任务调度。

```
1. os_sched_start();
```

9. 在 stack 准备好时，执行 app_handle_dev_state_evt 函数，通过 UART0 发送字符串"IO-SPI+GDMA read flash data demo!"到 PC 端，并执行 spi_demo 函数，执行 driver_spi_gdma_init，初始化 SPI0 以及 GDMA 通道 2 外设。

10. 初始化 SPI0 外设。

- 1) SPI 的数据传输模式为 EEPROM 模式。
- 2) 工作模式为主设备模式。
- 3) 数据大小为 8 位帧结构。
- 4) 串行时钟的稳态为时钟悬空高。
- 5) 位捕获的时钟活动沿为数据捕获于第二个时钟沿。
- 6) 时钟分频系数为 100。
- 7) 数据传输格式为 Motorola 传输格式。
- 8) 读取的数据长度阈值为 GDMA_READ_SIZE_MAX。
- 9) 开启接收数据 GDMA 通道。
- 10) SPI_RxWaterlevel 设置为 1。

（详情参考：SPI→Polling_eeprom）

11. 初始化 GDMA 外设。

- 1) 使用 GDMA 通道 2。
- 2) 传输方向为外设到内存传输，源端地址为 FLASH_SPI->DR，目的端地址为 GDMA_Recv_Buffer。
- 3) 使能 Multi-block 传输。
- 4) 设置每次 block 传输后自动加载 LLI 结构体中源地址与目的地址值；使能 Multi-block 传输；设置传输 LLI 类型结构体首地址。

- 5) 配置每次 block 传输后 LLI 结构体中源地址、目的地址、链表指针、控制寄存器。
(详情参考: GDMA→Mem2Mem_multi_block)

12. 使能 GDMA 通道 2 GDMA_INT_Block 中断。

- 1) 使能 GDMA 传输。
- 2) 使能 SPI0 外设。
- 3) 通过 SPI0 外设发送读取数据命令以及地址。

```

1.  void driver_spi_init(void)
2.  {
3.      .....
4.      /*-----SPI init-----*/
5.      SPI_InitTypeDef SPI_InitStruct;
6.      SPI_StructInit(&SPI_InitStruct);
7.      .....
8.      SPI_InitStruct.SPI_NDF          = GDMA_READ_SIZE_MAX - 1;
9.      SPI_InitStruct.SPI_RxDmaEn      = ENABLE;
10.     SPI_InitStruct.SPI_RxWaterlevel = 1;
11.     .....
12.     /*-----SPI init-----*/
13.     GDMA_StructInit(&GDMA_InitStruct);
14.     GDMA_InitStruct.GDMA_ChannelNum  = GDMA_CHANNEL_NUM;
15.     GDMA_InitStruct.GDMA_DIR         = GDMA_DIR_PeripheralToMemory;
16.     .....
17.     GDMA_InitStruct.GDMA_SourceAddr  = (uint32_t) FLASH_SPI->DR;
18.     GDMA_InitStruct.GDMA_DestinationAddr = (uint32_t)GDMA_Recv_Buffer;
19.     GDMA_InitStruct.GDMA_Multi_Block_Mode = LLI_TRANSFER;
20.     GDMA_InitStruct.GDMA_Multi_Block_En = 1;
21.     GDMA_InitStruct.GDMA_Multi_Block_Struct = (uint32_t)GDMA_LLIStruct;
22.     for (int i = 0; i < GDMA_MULTIBLOCK_SIZE; i++)
23.     {
24.         .....
25.     }
26.     .....
27.     /* Enable transfer interrupt */
28.     GDMA_INTConfig(GDMA_CHANNEL_NUM, GDMA_INT_Block, ENABLE);
29.     GDMA_Cmd(GDMA_CHANNEL_NUM, ENABLE);
30.     SPI_Cmd(FLASH_SPI, ENABLE);
31.     SPI_SendBuffer(FLASH_SPI, GDMA_WriteCmdBuffer, 5);
32. }

```

13. 当一次 block 传输完成时, 触发 GDMA_INT_Block 中断, 进入中断处理函数 GDMA_Channel_Handler。

- 1) 失能 GDMA 通道 2 GDMA_INT_Block 中断。
- 2) 定义消息类型 IO_MSG_TYPE_GDMA，保存数据，发送 msg 给 task。
- 3) 清除 GDMA 中断悬挂位，使能 GDMA_INT_Block 中断。

```

1. void GDMA_Channel_Handler(void)
2. {
3.     GDMA_INTConfig(GDMA_CHANNEL_NUM, GDMA_INT_Block, DISABLE);
4.
5.     T_IO_MSG int_gdma_msg;
6.     int_gdma_msg.type = IO_MSG_TYPE_GDMA;
7.     int_gdma_msg.subtype = 0;
8.     int_gdma_msg.u.buf = (void *)GDMA_Recv_Buffer;
9.     if (false == app_send_msg_to_apptask(&int_gdma_msg))
10.        .....
11.     GDMA_ClearINTPendingBit(GDMA_CHANNEL_NUM, GDMA_INT_Block);
12.     GDMA_INTConfig(GDMA_CHANNEL_NUM, GDMA_INT_Block, ENABLE);
13. }

```

14. app_main_task 循环检测 msg queue，当有 msg 时，执行 app_handle_io_msg(io_msg)函数。

15. 在 app_handle_io_msg 函数中，判断消息类型为 IO_MSG_TYPE_GDMA，执行 io_handle_gdma_msg。

- 1) 打印 block 传输次数，数据长度和传输数据完成信息。
- 2) 通过 UART0 发送 GDMA 搬运的数据信息到 PC 端。

```

1. void io_handle_gdma_msg(T_IO_MSG *io_gdma_msg)
2. {
3.     .....
4.     APP_PRINT_INFO2("[io_gdma] io_handle_gdma_msg:
5.         read data complete,
6.         num = %d,
7.         data_len = %d",
8.         gdma_transfer_num,
9.         data_len);
10.    uart_senddata_continuous(UART0, &p_buf[gdma_transfer_num * data_len], data_len);
11.    gdma_transfer_num++;
12. }

```

10.5.3 实验现象

在 DebugAnalyser 工具上，打印 GDMA 搬运数据信息，并在串口调试助手收到芯片搬运的数据。

10.6 伪静态随机存储（PSRAM）

使用 SPI0 与 APS6404L 模块以 GDMA 方式进行数据传输。

通过 GDMA 搬运内存数据信息到 PARAM 芯片中，通过 GDMA 传输完成中断向 app_task 发送消息事件，app_task 检测到消息事件，在 app 层解析消息，并执行用户程序：在 DebugAnalyser 工具上，打印发送数据完成信息。

通过 GDMA 搬运 PARAM 芯片内部数据信息到内存中，通过 GDMA 传输完成中断向 app_task 发送消息事件，app_task 检测到消息事件，在 app 层解析消息，并执行用户程序：在 DebugAnalyser 工具上，打印接收数据完成信息，并通过 UART 外设发送接收数据到 PC 端。

工程目录：\HoneyComb\sdk\board\evb\io_sample\SPI\PSRAM

10.6.1 硬件设计

- 硬件连接：P3_0→RX，P3_1→TX。

EVB 外接 FT232 模块，连接 P3_0 和 FT232 的 RX，P3_1 和 FT232 的 TX。

- 硬件连接：P4_0→CLK，P4_1→DO，P4_2→DI，P4_3→CS#。

EVB 外接 FM25Q16 模块，连接 P4_0 和 CLK，P4_1 和 DO，P4_2 和 DI，P4_3 和 CS#，同时连接 VDD 和模块的 VDD、WP#、HOLD#，连接 GND 和模块的 VSS。

10.6.2 软件流程

1. 引脚定义。

```
1.  #define UART_TX_PIN      P3_0
2.  #define UART_RX_PIN      P3_1
```

2. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、内部上拉、输出失能、输出高。

3. 配置 PINMUX：分配引脚分别为 UART0_TX、UART0_RX 功能。（详情参考：UART→Polling）

4. 引脚定义。配置 PAD，配置 PINMUX：分配引脚分别为 SPI0_CLK_MASTER、SPI0_MO_MASTER、SPI0_MI_MASTER、SPI0_SS_N_0_MASTER 功能。（详情参考：SPI→EEPROM）

5. app_task 初始化。

```
1.  os_task_create(&app_task_handle,"app",app_main_task,0,
                APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);
```

6. 在 app_main_task 中执行 driver_init 函数，执行 driver_UART_init 函数，对 UART0 外设进行初始化。默认波特率 115200，无奇偶校验，停止位 1 位，数据长度 8bits，FIFO 阈值为 16，空闲时间为 2byte 等。（详情参考：UART→Polling）

```

1.  void driver_uart_init(void)
2.  {
3.      .....
4.  }
```

7. 开始任务调度。

```

1.  os_sched_start();
```

8. 在 stack 准备好时，执行 app_handle_dev_state_evt 函数，执行 spi_demo 函数，执行 driver_spi_gdma_tx_init，初始化 SPI0 以及 GDMA 通道 0 外设。

9. 初始化 SPI0 外设。

- 1) SPI 的数据传输模式为发送数据模式。
- 2) 工作模式为主设备模式。
- 3) 数据大小为 8 位帧结构。
- 4) 串行时钟的稳态为时钟悬空高。
- 5) 位捕获的时钟活动沿为数据捕获于第二个时钟沿。
- 6) 时钟分频系数为 100。
- 7) 数据传输格式为 Motorola 传输格式。
- 8) 读取数据长度阈值为 1。
- 9) SPI_TxDmaEn 设置为 ENABLE。

10. 初始化 GDMA 外设。

- 1) 使用 GDMA 通道 0。
- 2) 传输方向为内存到外设传输，源端地址为 GDMA_Send_Buffer，目的端地址为 SPI0→DR。
- 3) 使能 GDMA 通道 0 总传输完成中断（GDMA_INT_Transfer）。
- 4) 使能 SPI0 外设传输。
- 5) 使能 GDMA 通道 0 传输。

```

1.  void driver_spi_gdma_tx_init (void)
2.  {
3.      .....
4.      /*-----SPI init-----*/
5.      SPI_InitTypeDef SPI_InitStruct;
6.      SPI_StructInit(&SPI_InitStruct);
```

```

7.     SPI_InitStructure.SPI_Direction  = SPI_Direction_TxOnly;
8.     .....
9.     SPI_InitStructure.SPI_NDF       = 0;
10.    SPI_InitStructure.SPI_TxDmaEn   = ENABLE;
11.    SPI_InitStructure.SPI_TxWaterlevel = 15;
12.
13.    /*-----GDMA initial-----*/
14.    GDMA_StructInit(&GDMA_InitStruct);
15.    .....
16.    GDMA_InitStruct.GDMA_ChannelNum   = GDMA_SPI_TX_NUM;
17.    GDMA_InitStruct.GDMA_DIR          = GDMA_DIR_MemoryToPeripheral;
18.    .....
19.    GDMA_InitStruct.GDMA_SourceAddr   = (uint32_t)GDMA_Send_Buffer;
20.    GDMA_InitStruct.GDMA_DestinationAddr = (uint32_t)SPI0->DR;
21.    .....
22.    /* Enable transfer interrupt */
23.    GDMA_INTConfig(GDMA_SPI_TX_NUM, GDMA_INT_Transfer, ENABLE);
24.    SPI_Cmd(SPI0, ENABLE);
25.    GDMA_Cmd(GDMA_SPI_TX_NUM, ENABLE);
26.    }

```

11. 当 GDMA 通道 0 搬运数据完成时，触发 GDMA_INT_Transfer 中断，进入 GDMA 中断处理函数 GDMA_SPI_TX_Handler。

- 1) 失能 GDMA 通道 0 GDMA_INT_Transfer 中断。
- 2) 定义消息类型 IO_MSG_TYPE_GDMA，子类型为 0，保存 GDMA_Send_Buffer 数据，发送 msg 给 task。
- 3) 清除 GDMA 通道 0 GDMA_INT_Transfer 中断挂起位。
- 4) 失能 GDMA 通道 0 传输。

```

1.     GDMA_SPI_TX_Handler ()
2.     {
3.         GDMA_INTConfig(GDMA_SPI_TX_NUM, GDMA_INT_Transfer, DISABLE);
4.         T_IO_MSG int_gdma_msg;
5.         int_gdma_msg.type = IO_MSG_TYPE_GDMA;
6.         int_gdma_msg.subtype = 0;
7.         int_gdma_msg.u.buf = (void *)GDMA_Send_Buffer;
8.         if (false == app_send_msg_to_apptask(&int_gdma_msg))
9.             .....
10.        GDMA_ClearINTPendingBit(GDMA_SPI_TX_NUM, GDMA_INT_Transfer);
11.        GDMA_Cmd(GDMA_SPI_TX_NUM, DISABLE);
12.    }

```

12. app_main_task 循环检测 msg queue，当有 msg 时，执行 app_handle_io_msg(io_msg)函数。

13. 在 `app_handle_io_msg` 函数中。

- 1) 判断消息类型为 `IO_MSG_TYPE_GDMA`，执行 `io_handle_gdma_msg` 函数。
- 2) 判断子类型若为 0，打印发送数据完成信息；执行函数 `driver_spi_gdma_rx_init()`。

```

1.  void io_handle_gdma_msg(T_IO_MSG *io_gdma_msg)
2.  {
3.      .....
4.      if (sub_type == 0)
5.      {
6.          APP_PRINT_INFO0("[io_gdma] io_handle_gdma_msg: send data complete");
7.          driver_spi_gdma_rx_init();
8.      }
9.      .....
10. }
```

14. 在 `driver_spi_gdma_rx_init` 中，初始化 SPI0 以及 GDMA 通道 2 外设。

15. 初始化 SPI0 外设。

- 1) SPI 的数据传输模式为 EEPROM。
- 2) 工作模式为主设备模式。
- 3) 数据大小为 8 位帧结构。
- 4) 串行时钟的稳态为时钟悬空高。
- 5) 位捕获的时钟活动沿为数据捕获于第二个时钟沿。
- 6) 时钟分频系数为 100。
- 7) 数据传输格式为 Motorola 传输格式。
- 8) 读取数据长度阈值为 1000。
- 9) `SPI_RxDmaEn` 设置为 ENABLE。

16. 初始化 GDMA 外设。

- 1) 使用 GDMA 通道 2。
- 2) 传输方向为内存到外设传输，源端地址为 `SPI0->DR`，目的端地址为 `GDMA_Recv_Buffer`。
- 3) 使能 GDMA 通道 2 总传输完成中断（`GDMA_INT_Transfer`）。
- 4) 使能 SPI0 外设传输。
- 5) 使能 GDMA 通道 2 传输。

```

1.  void driver_spi_gdma_tx_init (void)
2.  {
3.      .....
4.      /*-----SPI init-----*/
5.      SPI_InitTypeDef SPI_InitStruct;
6.      SPI_StructInit(&SPI_InitStruct);
```

```

7.     SPI_InitStructure.SPI_Direction  = SPI_Direction_EEPROM;
8.     .....
9.     SPI_InitStructure.SPI_NDF       = 0;
10.    SPI_InitStructure.SPI_TxDmaEn   = ENABLE;
11.    SPI_InitStructure.SPI_TxWaterlevel = 15;
12.
13.    /*-----GDMA initial-----*/
14.    GDMA_StructInit(&GDMA_InitStruct);
15.    .....
16.    GDMA_InitStruct.GDMA_ChannelNum   = GDMA_SPI_RX_NUM;
17.    GDMA_InitStruct.GDMA_DIR          = GDMA_DIR_PeripheralToMemory;
18.    .....
19.    GDMA_InitStruct.GDMA_SourceAddr   = (uint32_t)SPI0->DR;
20.    GDMA_InitStruct.GDMA_DestinationAddr = (uint32_t)GDMA_Recv_Buffer;
21.    .....
22.    /* Enable transfer interrupt */
23.    GDMA_INTConfig(GDMA_SPI_RX_NUM, GDMA_INT_Transfer, ENABLE);
24.    SPI_Cmd(SPI0, ENABLE);
25.    GDMA_Cmd(GDMA_SPI_RX_NUM, ENABLE);
26.    }

```

17. 当 GDMA 通道 2 搬运数据完成时，触发 GDMA_INT_Transfer 中断，进入 GDMA 中断处理函数 GDMA_SPI_RX_Handler。

- 1) 失能 GDMA 通道 2 GDMA_INT_Transfer 中断。
- 2) 定义消息类型 IO_MSG_TYPE_GDMA，子类型为 1，保存 GDMA_Recv_Buffer 数据，发送 msg 给 task。
- 3) 清除 GDMA 通道 2GDMA_INT_Transfer 中断挂起位。

```

1.     GDMA_SPI_RX_Handler ()
2.     {
3.         GDMA_INTConfig(GDMA_SPI_RX_NUM, GDMA_INT_Transfer, DISABLE);
4.
5.         T_IO_MSG int_gdma_msg;
6.         int_gdma_msg.type = IO_MSG_TYPE_GDMA;
7.         int_gdma_msg.subtype = 1;
8.         int_gdma_msg.u.buf = (void *)GDMA_Recv_Buffer;
9.         if (false == app_send_msg_to_apptask(&int_gdma_msg))
10.            .....
11.         GDMA_ClearINTPendingBit(GDMA_SPI_RX_NUM, GDMA_INT_Transfer);
12.     }

```

18. app_main_task 循环检测 msg queue，当有 msg 时，执行 app_handle_io_msg(io_msg)函数。

19. 在 `app_handle_io_msg` 函数中。

- 1) 判断消息类型为 `IO_MSG_TYPE_GDMA`，执行 `io_handle_gdma_msg` 函数。
- 2) 判断子类型若为 1，打印读取数据完成信息。
- 3) 执行函数 `uart_senddata_continuous(UART0, p_buf, GDMA_TRANSFER_SIZE)`，将读取的数据通过 UART0 发送到 PC 端。

```

1.  void io_handle_gdma_msg(T_IO_MSG *io_gdma_msg)
2.  {
3.      .....
4.      if (sub_type == 1)
5.      {
6.          APP_PRINT_INFO1("[io_gdma] io_handle_gdma_msg:
                                read data complete data_len = %d",
                                GDMA_TRANSFER_SIZE);
7.          uart_senddata_continuous(UART0, p_buf, GDMA_TRANSFER_SIZE);
8.      }
9.      .....
10. }
```

10.6.3 实验现象

当 GDMA 搬运发送数据完成时，在 DebugAnalyser 工具上，打印发送数据完成信息。

当 GDMA 搬运接收数据完成时，在 DebugAnalyser 工具上，打印接收数据完成信息，并在串口调试助手收到读取的 FLASH 中的数据。

11 内部集成电路（I2C）

专用硬件设备：STK8321 模块

11.1 I2C 自发自收

使用 I2C0 作为主机，I2C1 作为从机，实现数据的传输。

工程目录：\HoneyComb\sdk\board\evb\io_sample\I2C\I2C_SelfTest

11.1.1 硬件设计

硬件连接：P4_0→P4_2，P4_1→P4_3。

EVB 上，连接 P4_0 和 P4_2，P4_1 和 P4_3。

11.1.2 软件流程

1. 引脚定义。

```
1.  #define I2C_MASTER_SCL_PIN    P4_0
2.  #define I2C_MASTER_SDA_PIN    P4_1
3.  #define I2C_SLAVES_SCL_PIN    P4_2
4.  #define I2C_SLAVE_SDA_PIN     P4_3
```

2. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、内部上拉、输出使能、输出高。

3. 配置 PINMUX：分配引脚分别为 I2C0_CLK、I2C0_DAT 和 I2C1_CLK、I2C1_DAT 功能。

4. 执行 driver_i2c_master_init 函数，对 I2C0 外设进行初始化。

- 1) I2C_ClockSpeed 设置为 100000。
- 2) I2C_DeiveMode 设置为 I2C_DeiveMode_Master，即主设备。
- 3) I2C_AddressMode 设置为 I2C_AddressMode_7BIT，即 7 位地址模式。
- 4) I2C_SlaveAddress 设置为（0x50），即从机地址为 0x50。
- 5) I2C_Ack 设置为 I2C_Ack_Enable，即使能 Ack 功能。

5. 使能 I2C0。

```
1.  void driver_i2c_init(void)
```



```

2.  {
3.      .....
4.      I2C_InitTypeDef I2C_InitStruct;
5.      I2C_StructInit(&I2C_InitStruct);
6.      I2C_InitStruct.I2C_ClockSpeed      = 100000;
7.      I2C_InitStruct.I2C_DeiveMode       = I2C_DeiveMode_Master;
8.      I2C_InitStruct.I2C_AddressMode     = I2C_AddressMode_7BIT;
9.      I2C_InitStruct.I2C_SlaveAddress     = 0x50;
10.     I2C_InitStruct.I2C_RxThresholdLevel = 8;
11.     I2C_InitStruct.I2C_Ack              = I2C_Ack_Enable;
12.     .....
13.     I2C_Cmd(I2C0, ENABLE);
14. }

```

6. 执行 driver_i2c_slave_init 函数，对 I2C1 外设进行初始化。

- 1) I2C_ClockSpeed 设置为 100000。
- 2) I2C_DeiveMode 设置为 I2C_DeiveMode_Slave，即从设备。
- 3) I2C_AddressMode 设置为 I2C_AddressMode_7BIT，即 7 位地址模式。
- 4) I2C_SlaveAddress 设置为（0x50），即从机地址为 0x50。
- 5) I2C_Ack 设置为 I2C_Ack_Enable，即使能 Ack 功能。

7. 使能 I2C1。

```

1.  void driver_i2c_init(void)
2.  {
3.      .....
4.      I2C_InitTypeDef I2C_InitStruct;
5.      I2C_StructInit(&I2C_InitStruct);
6.      I2C_InitStruct.I2C_ClockSpeed      = 100000;
7.      I2C_InitStruct.I2C_DeiveMode       = I2C_DeiveMode_Slave;
8.      I2C_InitStruct.I2C_AddressMode     = I2C_AddressMode_7BIT;
9.      I2C_InitStruct.I2C_SlaveAddress     = 0x50;
10.     I2C_InitStruct.I2C_RxThresholdLevel = 8;
11.     I2C_InitStruct.I2C_Ack              = I2C_Ack_Enable;
12.     .....
13.     I2C_Cmd(I2C1, ENABLE);
14. }

```

8. 执行 nvic_i2c_config 函数，对 I2C0 和 I2C1 外设的中断进行初始化配置。

- 1) 清除中断挂起位。
- 2) 根据程序中具体发送或接收设定使能 I2C0 和 I2C1 相关中断。
- 3) I2C0 和 I2C1 中断 NVIC 初始化配置。

```

1.  void nvic_i2c_config (void)
2.  {
3.      /* Detect stop signal */
4.      I2C_ClearINTPendingBit(I2C0, I2C_INT_STOP_DET);
5.      I2C_ClearINTPendingBit(I2C1, I2C_INT_STOP_DET);
6.      /* Detect read require signal */
7.      I2C_ClearINTPendingBit(I2C1, I2C_INT_RD_REQ);
8.      /* Detect rx fifo full signal */
9.      I2C_ClearINTPendingBit(I2C1, I2C_INT_RX_FULL);
10.     /* Detect rx transmission finished signal */
11.     I2C_ClearINTPendingBit(I2C1, I2C_INT_RX_DONE);
12.
13.     #if I2C_MASTER_SEND_SLAVE_RECEIVE
14.         I2C_INTConfig(I2C0, I2C_INT_STOP_DET, ENABLE);
15.         I2C_INTConfig(I2C1, I2C_INT_RX_FULL, ENABLE);
16.         I2C_INTConfig(I2C1, I2C_INT_STOP_DET, ENABLE);
17.         I2C_INTConfig(I2C1, I2C_INT_RX_DONE, ENABLE);
18.     #endif
19.     .....
20. }

```

➤ 如果开启 I2C 主机发送从机接收，即开启 **I2C_MASTER_SEND_SLAVE_RECEIVE**

1. 主机发送 write_data 数组中的数据给从机。

```

1.  void i2c_demo(void)
2.  {
3.      .....
4.      #if I2C_MASTER_SEND_SLAVE_RECEIVE
5.          I2C_MasterWrite(I2C0, write_data, TransferLength);
6.      #endif
7.  }

```

2. 从机接收主机发送的数据，当从机接收 FIFO 满时触发 I2C_INT_RX_FULL，或者检测到停止信号时触发 I2C_INT_STOP_DET 中断，进入中断处理函数 I2C1_Handler。
3. 判断从机检测到停止信号状态为 SET 时，则
 - 1) 获取 RXFIFO 中接收数据的长度以及保存接收数据。
 - 2) 在 DebugAnalyser 工具上打印接收数据总长度以及接收数据信息。
 - 3) 清除 I2C_INT_STOP_DET 中断挂起位。

4. 判断检测到接收 FIFO 满状态为 SET 时，则
 - 1) 获取 RXFIFO 中接收数据的长度以及保存接收数据。
 - 2) 清除 I2C_INT_RX_FULL 中断挂起位。

```

1.  void I2C1_Handler (void)
2.  {
3.      if (I2C_GetINTStatus(I2C1, I2C_INT_STOP_DET) == SET)
4.      {
5.          .....
6.          lenth = I2C_GetRxFIFOLen(I2C1);
7.          for (uint32_t i = 0; i < lenth; i++)
8.          {
9.              I2C_Rev_Data[I2C_Rev_Index++] = I2C_ReceiveData(I2C1);
10.         }
11.         I2C_Rev_Data_Lenth += lenth;
12.         I2C_ClearINTPendingBit(I2C1, I2C_INT_STOP_DET);
13.     }
14. }

```

5. 在 DebugAnalyser 工具上，打印中断信息及 I2C1 作为从机接收到 I2C1 传递的数据长度和信息。

➤ 如果开启 I2C 主机接收从机发送，即开启 I2C_MASTER_RECEIVE_SLAVE_SEND

1. 主机发送 MasterRead 命令给从机。

```

1.  void i2c_demo(void)
2.  {
3.      .....
4.      /* I2C master read */
5.      uint8_t data[TransferLength] = {0};
6.      I2C_MasterRead(I2C0, data, TransferLength);
7.  }

```

2. 从机检测到 RD_REQ 申请时触发 I2C_INT_RD_REQT 中断，进入中断处理函数 I2C1_Handler。
3. 判断 RD_REQ 中断状态为 SET 时，则
 - 1) 执行 I2C_SendCmd 命令发送数据给主机。
 - 2) 发送数据完成后，清除 I2C_INT_RD_REQ 中断挂起位。

```

1.  void I2C1_Handler (void)
2.  {

```

```

3.     if (I2C_GetINTStatus(I2C1, I2C_INT_RD_REQ) == SET)
4.     {
5.         /* Notes: DBG_DIRECT is only used for debug demo, do not use in app!
6.            In app project, the APP_PRINT_INFO can be used.
7.         */
8.         DBG_DIRECT("Enter I2C1 interrupt I2C1_INT_RD_REQ!");
9.
10.        for (uint32_t i = 0; i < TransferLength; i++)
11.        {
12.            I2C_SendCmd(I2C1, I2C_WRITE_CMD, send_data_buffer[i],
13.                        I2C_STOP_DISABLE);
14.        }
15.        I2C_ClearINTPendingBit(I2C1, I2C_INT_RD_REQ);
16.    }

```

4. 在 DebugAnalyser 工具上,打印中断信息及 I2C0 作为主机读取 I2C1 传递的数据长度和数据信息。

➤ 如果开启 I2C 主机连续发送和接收数据,即开启 I2C_MASTER_REPEAT_READ

1. 主机发送 I2C_RepeatRead 命令给从机,发送 tx_data 中的数据,接收数据存储在 rx_data 中。

```

1.     void i2c_demo(void)
2.     {
3.         .....
4.         #if I2C_MASTER_REPEAT_READ
5.             /* I2C master repeat read */
6.             uint8_t tx_data[10] = {01, 02, 03, 04};
7.             uint8_t rx_data[TransferLength] = {0};
8.             I2C_RepeatRead(I2C0, tx_data, 4, rx_data, TransferLength);
9.         #endif
10.    }
11.
12.    void i2c_demo(void)
13.    {
14.        .....
15.        #if I2C_MASTER_SEND_SLAVE_RECEIVE
16.            I2C_MasterWrite(I2C0, write_data, TransferLength);
17.        #endif
18.    }

```

2. 从机接收主机发送的数据，当从机接收 FIFO 满时触发 I2C_INT_RX_FULL，或者检测到停止信号时触发 I2C_INT_STOP_DET 中断；从机检测到 RD_REQ 申请时触发 I2C_INT_RD_REQT 中断，进入中断处理函数 I2C1_Handler（此部分详情参考此章节主机发送从机接收和从机发送主机接收）。

11.1.3 实验现象

在 DebugAnalyser 工具上，打印中断信息以及 I2C1 作为从机接收 I2C0 发送数据长度和数据信息；I2C0 作为主机读取 I2C1 传递的数据长度和数据信息。

Realtek Confidential

11.2 I2C 通过 GDMA 实现主机发送从机接收

使用 I2C0 作为主机，I2C1 作为从机，利用 GDMA 外设实现数据的传输。用 GDMA 通道 0 搬运内存数据到 I2C0 发送端；GDMA 通道 1 搬运 I2C1 接收数据到内存。

工程目录：\HoneyComb\sdk\board\evb\io_sample\I2C\GDMA_Master_Tx_Slave_Rx

11.2.1 硬件设计

硬件连接：P4_0→P4_2，P4_1→P4_3。

EVB 上，连接 P4_0 和 P4_2，P4_1 和 P4_3。

11.2.2 软件流程

1. 引脚定义。

```
1.  #define I2C_MASTER_SCL_PIN    P4_0
2.  #define I2C_MASTER_SDA_PIN    P4_1
3.  #define I2C_SLAVER_SCL_PIN    P4_2
4.  #define I2C_SLAVE_SDA_PIN     P4_3
```

2. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、内部上拉、输出使能、输出高。
3. 配置 PINMUX：分配引脚分别为 I2C0_CLK、I2C0_DAT 和 I2C1_CLK、I2C1_DAT 功能。
4. 执行 driver_i2c_master_init 函数，对 I2C0 外设进行初始化。
 - 1) I2C_ClockSpeed 设置为 100000。
 - 2) I2C_DeiveMode 设置为 I2C_DeiveMode_Master，即主设备。
 - 3) I2C_AddressMode 设置为 I2C_AddressMode_7BIT，即 7 位地址模式。
 - 4) I2C_SlaveAddress 设置为（0x50），即从机地址为 0x50。
 - 5) I2C_Ack 设置为 I2C_Ack_Enable，即使能 Ack 功能。
 - 6) I2C_TxDmaEn 设置为 ENABLE，即使能 I2C 发送 DMA 功能。
 - 7) I2C_TxWaterlevel 设置为 22。
5. 使能 I2C0。

```
1.  void driver_i2c_init(void)
2.  {
3.      .....
4.      I2C_InitTypeDef I2C_InitStruct;
```

```

5.      I2C_StructInit(&I2C_InitStruct);
6.
7.      I2C_InitStruct.I2C_ClockSpeed      = 100000;
8.      I2C_InitStruct.I2C_DeiveMode       = I2C_DeiveMode_Master;
9.      I2C_InitStruct.I2C_AddressMode     = I2C_AddressMode_7BIT;
10.     I2C_InitStruct.I2C_SlaveAddress     = 0x50;
11.     I2C_InitStruct.I2C_RxThresholdLevel = 8;
12.     I2C_InitStruct.I2C_Ack              = I2C_Ack_Enable;
13.     I2C_InitStruct.I2C_TxDmaEn          = ENABLE;
14.     I2C_InitStruct.I2C_TxWaterlevel     = 22;
15.     .....
16.     I2C_Cmd(I2C0, ENABLE);
17. }

```

6. 执行 driver_i2c_slave_init 函数，对 I2C1 外设进行初始化。

- 1) I2C_ClockSpeed 设置为 100000。
- 2) I2C_DeiveMode 设置为 I2C_DeiveMode_Slave，即从设备。
- 3) I2C_AddressMode 设置为 I2C_AddressMode_7BIT，即 7 位地址模式。
- 4) I2C_SlaveAddress 设置为（0x50），即从机地址为 0x50。
- 5) I2C_Ack 设置为 I2C_Ack_Enable，即使能 Ack 功能。
- 6) I2C_RxDmaEn 设置为 ENABLE，即开启 I2C 接收 DMA 功能。
- 7) RxWaterlevel 设置为 3。

7. 使能 I2C1。

```

1.  void driver_i2c_init(void)
2.  {
3.      .....
4.      I2C_InitTypeDef I2C_InitStruct;
5.      I2C_StructInit(&I2C_InitStruct);
6.      I2C_InitStruct.I2C_ClockSpeed      = 100000;
7.      I2C_InitStruct.I2C_DeiveMode       = I2C_DeiveMode_Slave;
8.      I2C_InitStruct.I2C_AddressMode     = I2C_AddressMode_7BIT;
9.      I2C_InitStruct.I2C_SlaveAddress     = 0x50;
10.     I2C_InitStruct.I2C_RxThresholdLevel = 8;
11.     I2C_InitStruct.I2C_Ack              = I2C_Ack_Enable;
12.     I2C_InitStruct.I2C_RxDmaEn          = ENABLE;
13.     I2C_InitStruct.I2C_RxWaterlevel     = 3;
14.     .....
15.     I2C_Cmd(I2C1, ENABLE);
16. }

```

8. 执行 `driver_i2c_master_tx_init` 函数，对 GDMA 通道 0 外设进行初始化。

- 1) GDMA_ChannelNum 设置为 I2C0_GDMA_CHANNEL_NUM，使用 GDMA 通道 0。
- 2) GDMA_DIR 设置为 GDMA_DIR_MemoryToPeripheral，即 GDMA 的传输方向为内存到外设传输。
- 3) GDMA_BufferSize 设置为 GDMA_TRANSFER_SIZE，即完成一次 GDMA 传输的数据个数为 200。（数据大小为：GDMA_TRANSFER_SIZE*GDMA_DataSize_Word/4 Bytes）
- 4) 设置 GDMA_SourceInc，源端地址递增，设置 GDMA_DestinationFix，目的端地址固定。
- 5) 设置 GDMA_SourceDataSize，源端的数据宽度为 16 位，设置 GDMA_DestinationDataSize，目的端的数据宽度为 16 位。
- 6) 设置 GDMA_SourceMsize，源端单次 burst 传输的数据个数为 1，设置 GDMA_DestinationMsize，目的端单次 burst 传输的数据个数为 1。
- 7) GDMA_SourceAddr 设置为 GDMA_I2C_Send_Buf，buffer data 的最后一笔需要加 STOP 位。
- 8) GDMA_DestinationAddr 设置为(&(I2C0->IC_DATA_CMD))。
- 9) GDMA_DestHandshake 设置为 GDMA_Handshake_I2C0_TX。
- 10) 使能 GDMA 通道 0 总传输完成中断（GDMA_INT_Transfer）。
- 11) 执行 GDMA_Cmd 函数：使能 GDMA 传输。

```

1.  void gdma_i2c_master_tx_init (void)
2.  {
3.      .....
4.      GDMA_InitStruct.GDMA_ChannelNum    = I2C0_GDMA_CHANNEL_NUM;
5.      GDMA_InitStruct.GDMA_DIR           = GDMA_DIR_MemoryToPeripheral;
6.      GDMA_InitStruct.GDMA_BufferSize    = GDMA_TRANSFER_SIZE;
7.      GDMA_InitStruct.GDMA_SourceInc     = DMA_SourceInc_Inc;
8.      GDMA_InitStruct.GDMA_DestinationInc = DMA_DestinationInc_Fix;
9.      GDMA_InitStruct.GDMA_SourceDataSize = GDMA_DataSize_HalfWord;
10.     GDMA_InitStruct.GDMA_DestinationDataSize = GDMA_DataSize_HalfWord;
11.     GDMA_InitStruct.GDMA_SourceMsize    = GDMA_Msize_1;
12.     GDMA_InitStruct.GDMA_DestinationMsize = GDMA_Msize_1;
13.     GDMA_InitStruct.GDMA_SourceAddr     = (uint32_t)GDMA_I2C_Send_Buf;
14.     GDMA_InitStruct.GDMA_DestinationAddr = (uint32_t)(&(I2C0->IC_DATA_CMD));
15.     GDMA_InitStruct.GDMA_DestHandshake  = GDMA_Handshake_I2C0_TX;
16.     GDMA_Init(I2C0_GDMA_Channel, &GDMA_InitStruct);
17.     .....
18.     GDMA_INTConfig(I2C0_GDMA_CHANNEL_NUM, GDMA_INT_Transfer, ENABLE);
19.     GDMA_Cmd(I2C0_GDMA_CHANNEL_NUM, ENABLE);
20. }

```

9. 执行 `gdma_i2c_slave_rx_init` 函数，对 GDMA 通道 1 外设进行初始化。

- 1) GDMA_ChannelNum 设置为 I2C1_GDMA_CHANNEL_NUM，使用 GDMA 通道 1。

- 2) GDMA_DIR 设置为 GDMA_DIR_PeripheralToMemory, GDMA 的传输方向为外设到内存传输。
- 3) GDMA_BufferSize 设置为 GDMA_TRANSFER_SIZE, 即完成一次 GDMA 传输的数据个数为 200。(数据大小为: GDMA_TRANSFER_SIZE*GDMA_DataSize_Word/4 Bytes)
- 4) 设置 GDMA_SourceFix, 源端地址固定, 设置 GDMA_DestinationInc, 目的端地址递增。
- 5) 设置 GDMA_SourceDataSize, 源端的数据宽度为 8 位, 设置 GDMA_DestinationDataSize, 目的端的数据宽度为 8 位。
- 6) 设置 GDMA_SourceMsize, 源端单次 burst 传输的数据个数为 4, 设置 GDMA_DestinationMsize, 目的端单次 burst 传输的数据个数为 4。
- 7) GDMA_SourceAddr 设置为(&(I2C1->IC_DATA_CMD))。
- 8) GDMA_DestinationAddr 设置为 DMA_I2C_Recv_Buf。
- 9) GDMA_SourceHandshake 设置为 GDMA_Handshake_I2C1_RX。
- 10) 使能 GDMA 通道 1 总传输完成中断 (GDMA_INT_Transfer)。
- 11) 执行 GDMA_Cmd 函数: 使能 GDMA 传输。

```

1.  void gdma_i2c_slave_rx_init (void)
2.  {
3.      .....
4.      GDMA_InitStruct.GDMA_ChannelNum    = I2C1_GDMA_CHANNEL_NUM;
5.      GDMA_InitStruct.GDMA_DIR            = GDMA_DIR_PeripheralToMemory;
6.      GDMA_InitStruct.GDMA_BufferSize     = GDMA_TRANSFER_SIZE;
7.      GDMA_InitStruct.GDMA_SourceInc      = DMA_SourceInc_Fix;
8.      GDMA_InitStruct.GDMA_DestinationInc = DMA_DestinationInc_Inc;
9.      GDMA_InitStruct.GDMA_SourceDataSize = GDMA_DataSize_Byte;
10.     GDMA_InitStruct.GDMA_DestinationDataSize = GDMA_DataSize_Byte;
11.     GDMA_InitStruct.GDMA_SourceMsize     = GDMA_Msize_4;
12.     GDMA_InitStruct.GDMA_DestinationMsize = GDMA_Msize_4;
13.     GDMA_InitStruct.GDMA_SourceAddr      = (uint32_t)(I2C1->IC_DATA_CMD);
14.     GDMA_InitStruct.GDMA_DestinationAddr = (uint32_t)(GDMA_I2C_Recv_Buf);
15.     GDMA_InitStruct.GDMA_SourceHandshake = GDMA_Handshake_I2C1_RX;
16.     GDMA_Init(I2C1_GDMA_Channel, &GDMA_InitStruct);
17.     .....
18.     GDMA_INTConfig(I2C1_GDMA_CHANNEL_NUM, GDMA_INT_Transfer, ENABLE);
19.     GDMA_Cmd(I2C0_GDMA_CHANNEL_NUM, ENABLE);
20. }

```

10. 数据从 GDMA_I2C_Send_Buf 搬到(&(I2C0->IC_DATA_CMD))完成时, 触发 GDMA_INT_Transfer 中断, 进入中断处理函数 I2C0_GDMA_Channel_Handler。
 - 1) 失能 GDMA 通道 0 GDMA_INT_Transfer 中断。
 - 2) 打印 GDMA 通道 0 搬运数据完成信息。
 - 3) 清除 GDMA 通道 0 GDMA_INT_Transfer 中断挂起位。

```

1. void I2C0_GDMA_Channel_Handler (void)
2. {
3.     GDMA_INTConfig(I2C0_GDMA_CHANNEL_NUM, GDMA_INT_Transfer, DISABLE);
4.     DBG_DIRECT("I2C0_GDMA_Channel_Handler!
               I2C master send data by GDMA successfully");
5.     //Add user code here
6.     GDMA_ClearINTPendingBit(I2C0_GDMA_CHANNEL_NUM, GDMA_INT_Transfer);
7. }

```

11. 数据从(&(I2C1->IC_DATA_CMD))搬到 GDMA_I2C_Recv_Buf 完成时，触发 GDMA_INT_Transfer 中断，进入中断处理函数 I2C1_GDMA_Channel_Handler。

- 1) 失能 GDMA 通道 1 GDMA_INT_Transfer 中断。
- 2) 打印 GDMA 通道 1 搬运数据完成信息。
- 3) 对比 GDMA_I2C_Recv_Buf 与 GDMA_I2C_Send_Buf 中数据，查看是否有数据传输错误。
- 4) 清除 GDMA 通道 1 GDMA_INT_Transfer 中断挂起位。

```

1. void I2C0_GDMA_Channel_Handler (void)
2. {
3.     GDMA_INTConfig(I2C0_GDMA_CHANNEL_NUM, GDMA_INT_Transfer, DISABLE);
4.     DBG_DIRECT("I2C0_GDMA_Channel_Handler!
               I2C master send data by GDMA successfully");
5.     //Add user code here
6.     GDMA_ClearINTPendingBit(I2C0_GDMA_CHANNEL_NUM, GDMA_INT_Transfer);
7. }

```

11.2.3 实验现象

在 DebugAnalyser 工具上，打印 GDMA 搬运发送数据完成信息以及 GDMA 搬运接收数据完成信息，若数据出错则打印出错数据。

11.3 I2C 通过 GDMA 实现从机发送主机接收

使用 I2C0 作为主机，I2C1 作为从机，利用 GDMA 外设实现数据的传输。用 GDMA 通道 1 搬运内存数据到 I2C1 发送端；GDMA 通道 0 搬运 I2C0 接收数据到内存。

工程目录：\HoneyComb\sdk\board\evb\io_sample\I2C\GDMA_Master_Rx_Slave_Tx

11.3.1 硬件设计

硬件连接：P4_0→P4_2，P4_1→P4_3。

EVB 上，连接 P4_0 和 P4_2，P4_1 和 P4_3。

11.3.2 软件流程

1. 引脚定义。

```
1.  #define I2C_MASTER_SCL_PIN    P4_0
2.  #define I2C_MASTER_SDA_PIN    P4_1
3.  #define I2C_SLAVR_SCL_PIN     P4_2
4.  #define I2C_SLAVE_SDA_PIN     P4_3
```

2. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、内部上拉、输出使能、输出高。
3. 配置 PINMUX：分配引脚分别为 I2C0_CLK、I2C0_DAT 和 I2C1_CLK、I2C1_DAT 功能。
4. 执行 driver_i2c_master_init 函数，对 I2C0 外设进行初始化。
(详情参考：IIC→GDMA_Master_Tx_Slave_Rx)
5. 执行 driver_i2c_slave_init 函数，对 I2C1 外设进行初始化。
 - 1) I2C_ClockSpeed 设置为 100000。
 - 2) I2C_DeviveMode 设置为 I2C_DeviveMode_Slave，即从设备。
 - 3) I2C_AddressMode 设置为 I2C_AddressMode_7BIT，即 7 位地址模式。
 - 4) I2C_SlaveAddress 设置为 (0x50)，即从机地址为 0x50。
 - 5) I2C_Ack 设置为 I2C_Ack_Enable，即使能 Ack 功能。
 - 6) I2C_RxDmaEn 设置为 ENABLE，即开启 I2C 接收 DMA 功能。
 - 7) TxWaterlevel 设置为 16。
6. 使能 NVIC 中断相关配置，使能 I2C_INT_RD_REQ 中断。

7. 使能 I2C1。

```

1.  void driver_i2c_init(void)
2.  {
3.      .....
4.      I2C_InitTypeDef I2C_InitStruct;
5.      I2C_StructInit(&I2C_InitStruct);
6.      I2C_InitStruct.I2C_ClockSpeed      = 100000;
7.      I2C_InitStruct.I2C_DeviveMode      = I2C_DeviveMode_Slave;
8.      I2C_InitStruct.I2C_AddressMode     = I2C_AddressMode_7BIT;
9.      I2C_InitStruct.I2C_SlaveAddress    = 0x50;
10.     I2C_InitStruct.I2C_RxThresholdLevel = 8;
11.     I2C_InitStruct.I2C_Ack              = I2C_Ack_Enable;
12.     I2C_InitStruct.I2C_TxDmaEn         = ENABLE;
13.     I2C_InitStruct.I2C_TxWaterlevel    = 16;
14.     .....
15.     I2C_INTConfig(I2C1, I2C_INT_RD_REQ, ENABLE);
16.     I2C_Cmd(I2C1, ENABLE);
17. }

```

8. 执行 driver_i2c_slave_tx_init 函数，对 GDMA 通道 1 外设进行初始化。

- 1) GDMA_ChannelNum 设置为 I2C1_GDMA_CHANNEL_NUM，使用 GDMA 通道 1。
- 2) GDMA_DIR 设置为 GDMA_DIR_MemoryToPeripheral，GDMA 的传输方向为内存到外设传输。
- 3) GDMA_BufferSize 设置为 GDMA_TRANSFER_SIZE，即完成一次 GDMA 传输的数据个数为 200。（数据大小为：GDMA_TRANSFER_SIZE*GDMA_DataSize_Word/4 Bytes）
- 4) 设置 GDMA_SourceInc，源端地址递增，设置 GDMA_DestinationFix，目的端地址固定。
- 5) 设置 GDMA_SourceDataSize，源端的数据宽度为 16 位，设置 GDMA_DestinationDataSize，目的端的数据宽度为 16 位。
- 6) 设置 GDMA_SourceMsize，源端单次 burst 传输的数据个数为 4，设置 GDMA_DestinationMsize，目的端单次 burst 传输的数据个数为 4。
- 7) GDMA_SourceAddr 设置为 GDMA_I2C_Send_Buf。
- 8) GDMA_DestinationAddr 设置为(&(I2C1>IC_DATA_CMD))。
- 9) GDMA_DestHandshake 设置为 GDMA_Handshake_I2C1_TX。
- 10) 使能 GDMA 通道 1 总传输完成中断（GDMA_INT_Transfer）。

```

1.  void gdma_i2c_slave_tx_init (void)
2.  {
3.      .....
4.      GDMA_InitStruct.GDMA_ChannelNum    = I2C1_GDMA_CHANNEL_NUM;
5.      GDMA_InitStruct.GDMA_DIR           = GDMA_DIR_MemoryToPeripheral;
6.      GDMA_InitStruct.GDMA_BufferSize    = GDMA_TRANSFER_SIZE;

```

```

7.   GDMA_InitStruct.GDMA_SourceInc      = DMA_SourceInc_Inc;
8.   GDMA_InitStruct.GDMA_DestinationInc  = DMA_DestinationInc_Fix;
9.   GDMA_InitStruct.GDMA_SourceDataSize  = GDMA_DataSize_HalfWord;
10.  GDMA_InitStruct.GDMA_DestinationDataSize = GDMA_DataSize_HalfWord;
11.  GDMA_InitStruct.GDMA_SourceMsize     = GDMA_Msize_4;
12.  GDMA_InitStruct.GDMA_DestinationMsize = GDMA_Msize_4;
13.  GDMA_InitStruct.GDMA_SourceAddr      = (uint32_t)GDMA_I2C_Send_Buf;
14.  GDMA_InitStruct.GDMA_DestinationAddr = (uint32_t)(I2C1->IC_DATA_CMD);
15.  GDMA_InitStruct.GDMA_DestHandshake   = GDMA_Handshake_I2C1_TX;
16.  GDMA_Init(I2C0_GDMA_Channel, &GDMA_InitStruct);
17.  .....
18.  GDMA_INTConfig(I2C0_GDMA_CHANNEL_NUM, GDMA_INT_Transfer, ENABLE);
19.  }

```

9. 执行 `gdma_i2c_master_rx_init` 函数，对 GDMA 通道 0 外设进行初始化。

- 1) `GDMA_ChannelNum` 设置为 `I2C0_GDMA_CHANNEL_NUM`，使用 GDMA 通道 0。
- 2) `GDMA_DIR` 设置为 `GDMA_DIR_PeripheralToMemory`，GDMA 的传输方向为外设到内存传输。
- 3) `GDMA_BufferSize` 设置为 `GDMA_TRANSFER_SIZE`，即完成一次 GDMA 传输的数据个数为 200。（数据大小为：`GDMA_TRANSFER_SIZE*GDMA_DataSize_Word/4 Bytes`）
- 4) 设置 `GDMA_SourceFix`，源端地址固定，设置 `GDMA_DestinationInc`，目的端地址递增。
- 5) 设置 `GDMA_SourceDataSize`，源端的数据宽度为 8 位，设置 `GDMA_DestinationDataSize`，目的端的数据宽度为 8 位。
- 6) 设置 `GDMA_SourceMsize`，源端单次 burst 传输的数据个数为 1，设置 `GDMA_DestinationMsize`，目的端单次 burst 传输的数据个数为 1。
- 7) `GDMA_SourceAddr` 设置为 `(I2C0->IC_DATA_CMD)`。
- 8) `GDMA_DestinationAddr` 设置为 `DMA_I2C_Recv_Buf`。
- 9) `GDMA_SourceHandshake` 设置为 `GDMA_Handshake_I2C0_RX`。
- 10) 使能 GDMA 通道 1 总传输完成中断（`GDMA_INT_Transfer`）。
- 11) 执行 `GDMA_Cmd` 函数：使能 GDMA 传输。

```

1.  void gdma_i2c_slave_rx_init (void)
2.  {
3.      .....
4.      GDMA_InitStruct.GDMA_ChannelNum      = I2C1_GDMA_CHANNEL_NUM;
5.      GDMA_InitStruct.GDMA_DIR              = GDMA_DIR_PeripheralToMemory;
6.      GDMA_InitStruct.GDMA_BufferSize       = GDMA_TRANSFER_SIZE;
7.      GDMA_InitStruct.GDMA_SourceInc        = DMA_SourceInc_Fix;
8.      GDMA_InitStruct.GDMA_DestinationInc   = DMA_DestinationInc_Inc;
9.      GDMA_InitStruct.GDMA_SourceDataSize   = GDMA_DataSize_Byte;
10.     GDMA_InitStruct.GDMA_DestinationDataSize = GDMA_DataSize_Byte;
11.     GDMA_InitStruct.GDMA_SourceMsize       = GDMA_Msize_4;

```

```

12.   GDMA_InitStruct.GDMA_DestinationMsize   = GDMA_Msize_4;
13.   GDMA_InitStruct.GDMA_SourceAddr         = (uint32_t)(I2C1->IC_DATA_CMD));
14.   GDMA_InitStruct.GDMA_DestinationAddr     = (uint32_t)(GDMA_I2C_Recv_Buf);
15.   GDMA_InitStruct.GDMA_SourceHandshake     = GDMA_Handshake_I2C1_RX;
16.   GDMA_Init(I2C1_GDMA_Channel, &GDMA_InitStruct);
17.   .....
18.   GDMA_INTConfig(I2C1_GDMA_CHANNEL_NUM, GDMA_INT_Transfer, ENABLE);
19.   GDMA_Cmd(I2C0_GDMA_CHANNEL_NUM, ENABLE);
20. }

```

10. 执行 i2c_master_send_read_cmd 函数，对 GDMA 通道 2 外设进行初始化。

- 1) GDMA_ChannelNum 设置为 I2C0_READ_CMD_GDMA_CHANNEL_NUM，使用 GDMA 通道 2。
- 2) GDMA_DIR 设置为 MemoryToPeripheral，即 GDMA 的传输方向为内存到外设传输。
- 3) GDMA_BufferSize 设置为 GDMA_TRANSFER_SIZE，即完成一次 GDMA 传输的数据个数为 200。（数据大小为：GDMA_TRANSFER_SIZE*GDMA_DataSize_Word/4 Bytes）
- 4) 设置 GDMA_SourceFix，源端地址固定，设置 GDMA_DestinationInc，目的端地址递增。
- 5) 设置 GDMA_SourceDataSize，源端的数据宽度为 32 位，设置 GDMA_DestinationDataSize，目的端的数据宽度为 32 位。
- 6) 设置 GDMA_SourceMsize，源端单次 burst 传输的数据个数为 1，设置 GDMA_DestinationMsize，目的端单次 burst 传输的数据个数为 1。
- 7) GDMA_SourceAddr 设置为 GDMA_I2C_Tx_Read_Buf，Read_buf 中最后一笔数据加停止位，且所有数据的 BTI8 位置 1，为读命令。
- 8) GDMA_DestinationAddr 设置为(I2C0->IC_DATA_CMD)。
- 9) GDMA_SourceHandshake 设置为 GDMA_Handshake_I2C0_TX。
- 10) 使能 GDMA 通道 2 总传输完成中断（GDMA_INT_Transfer）。
- 11) 执行 GDMA_Cmd 函数：使能 GDMA 传输。

```

1.   void gdma_i2c_slave_rx_init (void)
2.   {
3.       .....
4.       GDMA_InitStruct.GDMA_ChannelNum   = I2C0_READ_CMD_GDMA_CHANNEL_NUM;
5.       GDMA_InitStruct.GDMA_DIR           = GDMA_DIR_MemoryToPeripheral;
6.       GDMA_InitStruct.GDMA_BufferSize    = GDMA_TRANSFER_SIZE;
7.       GDMA_InitStruct.GDMA_SourceInc     = DMA_SourceInc_Inc;
8.       GDMA_InitStruct.GDMA_DestinationInc = DMA_DestinationInc_Fix;
9.       GDMA_InitStruct.GDMA_SourceDataSize = GDMA_DataSize_Word;
10.      GDMA_InitStruct.GDMA_DestinationDataSize = GDMA_DataSize_Word;
11.      GDMA_InitStruct.GDMA_SourceMsize    = GDMA_Msize_1;
12.      GDMA_InitStruct.GDMA_DestinationMsize = GDMA_Msize_1;
13.      GDMA_InitStruct.GDMA_SourceAddr     = (uint32_t)GDMA_I2C_Tx_Read_Buf;

```

```

14.   GDMA_InitStruct.GDMA_DestinationAddr   = (uint32_t)(I2C0->IC_DATA_CMD));
15.   GDMA_InitStruct.GDMA_DestHandshake     = GDMA_Handshake_I2C0_TX;
16.   .....
17.   GDMA_INTConfig(I2C0_READ_CMD_GDMA_CHANNEL_NUM,
                    GDMA_INT_Transfer, ENABLE);
18.   GDMA_Cmd(I2C0_READ_CMD_GDMA_CHANNEL_NUM, ENABLE);
19. }

```

11. GDMA 通道 2 开始搬运读命令数据到 I2C1 时，触发 I2C_INT_RD_REQ 中断，进入中断处理函数 I2C1_Handler。

- 1) 使能 GDMA 通道 1 传输，开始搬运 TX_Buf 中的数据。
- 2) 清除 I2C_INT_RD_REQ 中断挂起位。
- 3) 失能 I2C_INT_RD_REQ 中断。

```

1.   void I2C1_Handler (void)
2.   {
3.       GDMA_Cmd(I2C1_GDMA_CHANNEL_NUM, ENABLE);
4.       I2C_ClearINTPendingBit(I2C1, I2C_INT_RD_REQ);
5.       I2C_INTConfig(I2C1, I2C_INT_RD_REQ, DISABLE);
6.   }

```

12. GDMA 通道 2 搬运读取命令完成时，触发 GDMA_INT_Transfer 中断，进入中断处理函数 I2C0_READ_CMD_GDMA_Channel_Handler。

- 1) 失能 GDMA 通道 2 GDMA_INT_Transfer 中断。
- 2) 清除 GDMA 通道 2 GDMA_INT_Transfer 中断挂起位。

```

1.   void I2C0_READ_CMD_GDMA_Channel_Handler (void)
2.   {
3.       GDMA_INTConfig(I2C0_READ_CMD_GDMA_CHANNEL_NUM,
                    GDMA_INT_Transfer, DISABLE);
4.       GDMA_ClearINTPendingBit(I2C0_READ_CMD_GDMA_CHANNEL_NUM,
                    GDMA_INT_Transfer);
5.   }

```

13. 数据从 GDMA_I2C_Send_Buf 搬到(I2C1->IC_DATA_CMD)完成时，触发 GDMA_INT_Transfer 中断，进入中断处理函数 I2C1_GDMA_Channel_Handler。

- 1) 失能 GDMA 通道 1 GDMA_INT_Transfer 中断。
- 2) 打印 GDMA 通道 1 搬运数据完成信息。
- 3) 清除 GDMA 通道 1 GDMA_INT_Transfer 中断挂起位。

```

1.   void I2C1_GDMA_Channel_Handler (void)

```



```

2.  {
3.      GDMA_INTConfig(I2C1_GDMA_CHANNEL_NUM, GDMA_INT_Transfer, DISABLE);
4.      DBG_DIRECT("I2C1_GDMA_Channel_Handler!
                    I2C master send data by GDMA successfully");
5.      //Add user code here
6.      GDMA_ClearINTPendingBit(I2C1_GDMA_CHANNEL_NUM, GDMA_INT_Transfer);
7.  }

```

14. 数据从(&(I2C0->IC_DATA_CMD))搬到 GDMA_I2C_Recv_Buf 完成时，触发 GDMA_INT_Transfer 中断，进入中断处理函数 I2C0_GDMA_Channel_Handler。

- 1) 失能 GDMA 通道 0 GDMA_INT_Transfer 中断。
- 2) 打印 GDMA 通道 0 搬运数据完成信息。
- 3) 对比 GDMA_I2C_Recv_Buf 与 GDMA_I2C_Send_Buf 中数据，查看是否有数据传输错误。
- 4) 清除 GDMA 通道 0 GDMA_INT_Transfer 中断挂起位。

```

1.  void I2C1_GDMA_Channel_Handler (void)
2.  {
3.      GDMA_INTConfig(I2C1_GDMA_CHANNEL_NUM, GDMA_INT_Transfer, DISABLE);
4.      DBG_DIRECT("I2C1_GDMA_Channel_Handler!
                    I2C master send data by GDMA successfully");
5.      //Add user code here
6.      GDMA_ClearINTPendingBit(I2C1_GDMA_CHANNEL_NUM, GDMA_INT_Transfer);
7.  }

```

11.3.3 实验现象

在 DebugAnalyser 工具上，打印 GDMA 搬运发送数据完成信息以及 GDMA 搬运接收数据完成信息，若数据出错则打印出错数据。

11.4 I2C 与 STK8321 通信

使用 I2C0 与 STK8321 模块进行数据传输。获取 STK8321 ID、三轴输出数据,通过 I2C0 中断向 app_task 发送消息事件, app_task 检测到消息事件,在 app 层解析消息,执行用户程序:在 DebugAnalyser 工具上,打印 GSensor 三轴输出数据。

工程目录: \HoneyComb\sdk\board\evb\io_sample\I2C\STK8321

11.4.1 硬件设计

硬件连接: P4_0→SCL, P4_1→SDA。

EVb 外接 STK8321 模块,连接 P4_0 和 SCL, P4_1 和 SDA,连接 GND、VDD、VCC、CS。

11.4.2 软件流程

1. 引脚定义。

```
1.  #define I2C0_SCL_PIN      P4_0
2.  #define I2C0_SDA_PIN      P4_1
```

2. 配置 PAD: 设置引脚、PINMUX 模式、PowerOn、内部上拉、输出使能、输出高。

3. 配置 PINMUX: 分配引脚分别为 I2C0_CLK、I2C0_DAT 功能。

```
1.  void board_i2c_init(void)
2.  {
3.      Pad_Config(I2C0_SCL_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
4.                  PAD_PULL_UP, PAD_OUT_ENABLE, PAD_OUT_HIGH);
5.      Pad_Config(I2C0_SDA_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
6.                  PAD_PULL_UP, PAD_OUT_ENABLE, PAD_OUT_HIGH);
7.      Pinmux_Config(I2C0_SCL_PIN, I2C0_CLK);
8.      Pinmux_Config(I2C0_SDA_PIN, I2C0_DAT);
9.  }
```

4. app_task 初始化。

```
1.  os_task_create(&app_task_handle,"app",app_main_task,0,
2.                  APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);
```

5. 在 app_main_task 中执行 driver_init 函数,执行 driver_i2c_init 函数,对 I2C0 外设进行初始化。

- 1) I2C_ClockSpeed 设置为 100000。
 - 2) I2C_DeviveMode 设置为 I2C_DeviveMode_Master，即主设备。
 - 3) I2C_AddressMode 设置为 I2C_AddressMode_7BIT，即 7 位地址模式。
 - 4) I2C_SlaveAddress 设置为 STK8321_ADDRESS（0x0F），即从机地址为 0x0F。
 - 5) I2C_Ack 设置为 I2C_Ack_Enable，即使能 Ack 功能。
6. 使能 I2C0。

```

1. void driver_i2c_init(void)
2. {
3.     .....
4.     I2C_InitStruct.I2C_ClockSpeed = 100000;
5.     I2C_InitStruct.I2C_DeviveMode = I2C_DeviveMode_Master;
6.     I2C_InitStruct.I2C_AddressMode = I2C_AddressMode_7BIT;
7.     I2C_InitStruct.I2C_SlaveAddress = STK8321_ADDRESS;
8.     I2C_InitStruct.I2C_Ack = I2C_Ack_Enable;
9.     .....
10.    I2C_Cmd(I2C0, ENABLE);
11. }

```

7. 开始任务调度。

```

1. os_sched_start();

```

8. 在 stack 准备好时，执行 app_handle_dev_state_evt，执行 i2c_demo，判断 ID 是否为 0x23，正确则
- 1) 使能 I2C0 检测到停止信号中断（I2C_INT_STOP_DET），清除 I2C_INT_STOP_DET 中断挂起位。
 - 2) 获取 stk8321 三轴输出数据。

```

1. void i2c_demo(void)
2. {
3.     if (0x23 == stk8321_id_get())
4.     {
5.         APP_PRINT_INFO0("[io_i2c] i2c_demo: Get stk8321 chip id OK. ");
6.         nvic_i2c_config();
7.         stk8321_outdata_get();
8.     }
9. }

```

9. 当检测到停止信号时，触发 I2C_INT_STOP_DET 中断，进入中断处理函数 I2C0_Handler：判断检测到停止信号状态为 SET 时。
- 1) 定义消息类型 IO_MSG_TYPE_I2C，保存 GSensor_Data.OutData 数据，发送 msg 给 task。

2) 清除 I2C_INT_STOP_DET 中断挂起位。

```

1.  void I2C0_Handler(void)
2.  {
3.      if (I2C_GetINTStatus(I2C0, I2C_INT_STOP_DET) == SET)
4.      {
5.          T_IO_MSG int_i2c_msg;
6.          int_i2c_msg.type = IO_MSG_TYPE_I2C;
7.          int_i2c_msg.subtype = 0;
8.          int_i2c_msg.u.buf = (void *)(&GSensor_Data.OutData);
9.          if (false == app_send_msg_to_apptask(&int_i2c_msg))
10.             .....
11.          I2C_ClearINTPendingBit(I2C0, I2C_INT_STOP_DET);
12.      }
13.  }
```

10. app_main_task 循环检测 msg queue。当有 msg 时，执行 app_handle_io_msg(io_msg)函数。

11. 在 app_handle_io_msg 函数中，判断消息类型为 IO_MSG_TYPE_I2C，执行 io_handle_i2c_msg 函数，执行 io_i2c_handle_msg：打印三轴输出数据信息。

```

1.  void io_i2c_handle_msg(T_IO_MSG *io_i2c_msg)
2.  {
3.      uint8_t *p_buf = io_i2c_msg->u.buf;
4.      GSensor_Data.XData = p_buf[0] & 0xFF;
5.      GSensor_Data.XData = (GSensor_Data.XData + ((uint16_t)p_buf[1] << 8)) >> 4;
6.      GSensor_Data.YData = p_buf[2] & 0xFF;
7.      GSensor_Data.YData = (GSensor_Data.YData + ((uint16_t)p_buf[3] << 8)) >> 4;
8.      GSensor_Data.ZData = p_buf[4] & 0xFF;
9.      GSensor_Data.ZData = (GSensor_Data.ZData + ((uint16_t)p_buf[5] << 8)) >> 4;
10.     APP_PRINT_INFO3("[io_i2c]io_i2c_handle_msg:
        GSensor x_axis_data = %d,
        y_axis_data = %d,
        z_axis_data = %d",
        GSensor_Data.XData,
        GSensor_Data.YData,
        GSensor_Data.ZData);
11. }
```

11.4.3 实验现象

在 DebugAnalyser 工具上，打印 GSensor 三轴输出数据。

12 按键扫描 (KEYSCAN)

专用硬件设备：矩阵键盘

12.1 按键扫描

Keyscan 采用手动扫描模式，即扫描一次就结束。实现矩阵键盘扫描，通过按键中断向 app_task 发送按键消息事件，app_task 检测到消息事件，在 app 层解析按键消息，并执行用户程序；在 DebugAnalyser 工具上，打印按键信息。

Key press debounce 及 key release debounce 使用软件定时器实现，本工程中 Key press debounce 根据按键的机械特性设置为 200ms，key release debounce 设置为 10ms。

工程目录：\HoneyComb\sdk\board\evb\io_sample\KEYSCAN\Keyscan

12.1.1 硬件设计

硬件连接：P2_3→ROW0，P2_4→ROW1，P4_0→COLUMN0，P4_1→COLUMN1。

EVB 外接矩阵键盘模块，连接 P2_3 和 ROW0，P2_4 和 ROW1，P4_0 和 COLUMN0，P4_1 和 COLUMN1，外接矩阵键盘如图 12-1 所示。

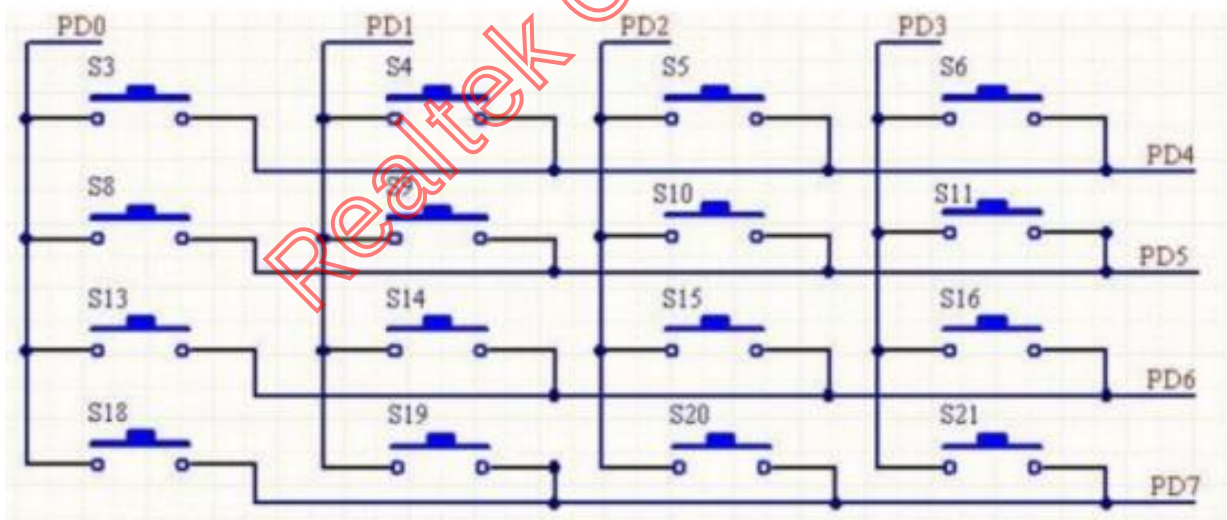


图 12-1 外接矩阵键盘

12.1.2 软件流程

1. 引脚定义：定义 2×2 矩阵键盘。

```

1.  #define KEYBOARD_ROW_SIZE      2
2.  #define KEYBOARD_COLUMN_SIZE   2
3.  #define KEYBOARD_ROW_0         P2_3
4.  #define KEYBOARD_ROW_1         P2_4
5.  #define KEYBOARD_COLUMN_0      P4_0
6.  #define KEYBOARD_COLUMN_1      P4_1

```

2. 初始化 keyscan 全局数据。

```

1.  void global_data_keyscan_init(void)
2.  {
3.      memset(&Current_Key_Data, 0, sizeof(KeyScan_Data_TypeDef));
4.  }

```

3. 配置 PAD: 设置引脚、PINMUX 模式、PowerOn、内部上拉或无上拉、输出使能或失能、输出低。

4. 配置 PINMUX: 分配引脚分别为 KEY_ROW_0、KEY_ROW_1、KEY_COL_0、KEY_COL_1 功能。

```

1.  void board_keyboard_init(void)
2.  {
3.      Pad_Config(KEYBOARD_ROW_0, PAD_PINMUX_MODE, PAD_IS_PWRON,
4.                  PAD_PULL_UP, PAD_OUT_DISABLE, PAD_OUT_LOW);
5.      Pad_Config(KEYBOARD_ROW_1, PAD_PINMUX_MODE, PAD_IS_PWRON,
6.                  PAD_PULL_UP, PAD_OUT_DISABLE, PAD_OUT_LOW);
7.      Pad_Config(KEYBOARD_COLUMN_0, PAD_PINMUX_MODE, PAD_IS_PWRON,
8.                  PAD_PULL_NONE, PAD_OUT_ENABLE, PAD_OUT_LOW);
9.      Pad_Config(KEYBOARD_COLUMN_1, PAD_PINMUX_MODE, PAD_IS_PWRON,
10.                 PAD_PULL_NONE, PAD_OUT_ENABLE, PAD_OUT_LOW);
11.     Pinmux_Config(KEYBOARD_ROW_0, KEY_ROW_0);
12.     Pinmux_Config(KEYBOARD_ROW_1, KEY_ROW_1);
13.     Pinmux_Config(KEYBOARD_COLUMN_0, KEY_COL_0);
14.     Pinmux_Config(KEYBOARD_COLUMN_1, KEY_COL_1);
15. }

```

5. 初始化 Keyscan 软件定时器，定时时间 200ms，回调函数为 timer_keyscan_callback。

```

1.  os_timer_create(&KeyScan_Timer_Handle, "keyscan_timer", 1,
2.                  KEYSCAN_SW_INTERVAL, false, timer_keyscan_callback));

```

6. app_task 初始化。

```

1.  os_task_create(&app_task_handle, "app", app_main_task, 0,
2.                  APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);

```

7. 在 `app_main_task` 中执行 `driver_init` 函数，执行 `driver_keyboard_init` 函数，对 KEYSCAN 外设进行初始化。

- 1) `rowSize` 设置为 `KEYBOARD_ROW_SIZE`，`colSize` 设置为 `KEYBOARD_COLUMN_SIZE`，即 2×2 矩阵键盘。
- 2) `scanmode` 设置为 `KeyScan_Manual_Scan_Mode`，即手动扫描模式。
- 3) `debounceEn` 设置为 `vDebounce_En`，即开启 `keyscan` 的硬件去抖动功能。
- 4) 使能 KEYSCAN。
- 5) 使能 KEYSCAN 单次扫描结束中断（`KEYSCAN_INT_SCAN_END`）。
- 6) 清除 `KEYSCAN_INT_SCAN_END` 中断挂起位。
- 7) 取消屏蔽 `KEYSCAN_INT_SCAN_END` 中断。

```

1.  void driver_keyboard_init(uint32_t vDebounce_En)
2.  {
3.      .....
4.      KEYSCAN_InitStruct.rowSize = KEYBOARD_ROW_SIZE;
5.      KEYSCAN_InitStruct.colSize = KEYBOARD_COLUMN_SIZE;
6.      KEYSCAN_InitStruct.scanmode = KeyScan_Manual_Scan_Mode;
7.      KEYSCAN_InitStruct.debounceEn = vDebounce_En;
8.      .....
9.      KeyScan_Cmd(KEYSCAN, ENABLE);
10.     KeyScan_INTConfig(KEYSCAN, KEYSCAN_INT_SCAN_END, ENABLE);
11.     KeyScan_ClearINTPendingBit(KEYSCAN, KEYSCAN_INT_SCAN_END);
12.     KeyScan_INTMask(KEYSCAN, KEYSCAN_INT_SCAN_END, DISABLE);
13.     .....
14. }
```

8. 开始任务调度。

```

1.  os_sched_start();
```

9. Keyscan 扫描键盘，键盘扫描结束触发中断，进入中断处理函数 `Keyscan_Handler`。

- 1) 屏蔽 `KEYSCAN_INT_SCAN_END` 中断。
- 2) 如果 Keyscan 的 FIFO 不为空（即有按键按下），则
 - (1) 从 `KeyScan FIFO` 中读取数据长度和数据，即按下按键的个数和键值，置位 `Key_Pressed_Flag`。
 - (2) 启动软件定时器，OS 实现定时 200ms（按键消抖时间），时间到进入 `timer_keyscan_callback` 回调函数。
 - (3) 定义消息类型 `IO_MSG_TYPE_KEYSCAN`，子类型 `IO_MSG_KEYSCAN_RX_PKT`，保存数据，发送 `msg` 给 `task`。
- 3) 清除 `KEYSCAN_INT_SCAN_END` 中断挂起位，取消屏蔽 `KEYSCAN_INT_SCAN_END` 中断。

```

1. void KeyScan_Handler(void)
2. {
3.     .....
4.     if (KeyScan_GetFlagState(KEYSCAN, KEYSCAN_INT_FLAG_SCAN_END) == SET)
5.     {
6.         KeyScan_INTMask(KEYSCAN, KEYSCAN_INT_SCAN_END, ENABLE);
7.         memset(&Current_Key_Data, 0, sizeof(KeyScan_Data_TypeDef));
8.         if (KeyScan_GetFlagState(KEYSCAN, KEYSCAN_FLAG_EMPTY) != SET)
9.         {
10.            fifo_length = (uint32_t)KeyScan_GetFifoDataNum(KEYSCAN);
11.            KeyScan_Read(KEYSCAN, (uint16_t *)&Current_Key_Data.key[0], fifo_length);
12.            Current_Key_Data.length = fifo_length;
13.            Key_Pressed_Flag = true;
14.
15.            if (!os_timer_restart(&KeyScan_Timer_Handle, KEYSCAN_SW_INTERVAL))
16.                .....
17.            int_keyscan_msg.type = IO_MSG_TYPE_KEYSCAN;
18.            int_keyscan_msg.subtype = IO_MSG_KEYSCAN_RX_PKT;
19.            int_keyscan_msg.u.buf = (void *)&Current_Key_Data;
20.            if (false == app_send_msg_to_apptask(&int_keyscan_msg))
21.                .....
22.        }
23.        KeyScan_ClearINTPendingBit(KEYSCAN, KEYSCAN_INT_SCAN_END);
24.        KeyScan_INTMask(KEYSCAN, KEYSCAN_INT_SCAN_END, DISABLE);
25.    }
26. }

```

10. 在 timer_keyscan_callback 回调函数中，判断 Key_Pressed_Flag。

1) 标志位为 true 时

- (1) 初始化 KEYSCAN 驱动，使能 KEYSCAN 功能，重新启动键盘扫描。
- (2) 同时启动软件定时器，定时 10ms，Release debounce 时间。

2) 标志位为 false 时

- (1) 定义消息类型 IO_MSG_TYPE_KEYSCAN，子类型 IO_MSG_KEYSCAN_ALLKEYRELEASE，并发送 msg 给 task。
- (2) 重新初始化 keyscan 全局数据，初始化 KEYSCAN 外设驱动。

```

1. void timer_keyscan_callback(void *p_xTimer)
2. {
3.     if (true == Key_Pressed_Flag)
4.     {
5.         Key_Pressed_Flag = false;
6.         driver_keyboard_init(KeyScan_Debounce_Disable);
7.         /* Start timer to check key status */

```

```

8.      os_timer_restart(&p_xTimer, KEYSCAN_SW_RELEASE_TIMEOUT);
9.      }
10.     else
11.     {
12.         T_IO_MSG int_keyscan_msg;
13.         int_keyscan_msg.type = IO_MSG_TYPE_KEYSCAN;
14.         int_keyscan_msg.subtype = IO_MSG_KEYSCAN_ALLKEYRELEASE;
15.         if (false == app_send_msg_to_apptask(&int_keyscan_msg))
16.             .....
17.         global_data_keyscan_init();
18.         driver_keyboard_init(KeyScan_Debounce_Enable);
19.     }
20. }

```

11. app_main_task 循环检测 msg queue，当有 msg 时，执行 app_handle_io_msg(io_msg)函数。

12. 在 app_handle_io_msg 函数中，判断消息类型为 IO_MSG_TYPE_KEYSCAN 时，执行 io_handle_keyscan_msg 函数，执行 io_keyscan_handle_keys，判断子类型。

- 1) 子类型为 IO_MSG_KEYSCAN_RX_PKT 时，获取消息中的按键数据，打印相应键值信息等。
- 2) 子类型为 IO_MSG_KEYSCAN_ALLKEYRELEASE 时，打印 All key release 信息。

```

1.  static void io_keyscan_handle_keys(T_IO_MSG *io_keyscan_msg)
2.  {
3.      uint16_t subtype = io_keyscan_msg->subtype;
4.      if (subtype == IO_MSG_KEYSCAN_RX_PKT)
5.      {
6.          KeyScan_Data_TypeDef *p_key_data =
              (KeyScan_Data_TypeDef *)io_keyscan_msg->u.buf;
7.          .....
8.      }
9.      else if (subtype == IO_MSG_KEYSCAN_ALLKEYRELEASE)
10.     {
11.         APP_PRINT_INFO0("[io_keyscan] io_keyscan_handle_keys: All keys release.");
12.     }
13.     .....
14. }

```

12.1.3 实验现象

按下按键时，在 DebugAnalyser 工具上，打印按键信息。

13 红外（IR）

专用硬件设备：IR 收发模块、红外遥控编码分析仪、红外遥控器

专用测试软件：IRReader

13.1 红外发送

使用 IR 外设进行数据发送，实现 TX 功能。使用 IR 遥控编码分析仪接收 IR 发送的数据，并观察发送数据是否符合预期。

工程目录：\HoneyComb\sdk\board\evb\io_sample\IR\Tx

13.1.1 硬件设计

硬件连接：P2_5→IR 收发模块的发送端。

EVb 外接 IR 收发模块，连接 P2_5 和 IR 收发模块的发送端，连接 VCC 和 IR 收发模块的 VCC，连接 GND 和 IR 收发模块的 GND。

13.1.2 软件流程

1. 定义 IR 发送数据数组，并赋值：有载波数据用载波个数与 0x80000000 进行或运算表示，无载波数据用载波个数与 0x00000000 进行或运算表示，载波个数为：timing/CarrierFreq。

```

1. void ir_demo(void)
2. {
3.     IR_TxData.CarrierFreq = 38000;
4.     IR_TxData.DataLen = 67 + 1; //2+64+1;
5.     IR_TxData.DataBuf[0] = 0x80000000 | 0x156; //342 about 9ms
6.     IR_TxData.DataBuf[1] = 0x00000000 | 0xAB; //171 about 4.5ms
7.     for (uint16_t i = 2; i < IR_TxData.DataLen - 1;)
8.     {
9.         IR_TxData.DataBuf[i] = 0x80000000 | 0x15; //21 about 560us
10.        IR_TxData.DataBuf[i + 1] = 0x00000000 | 0x15; //21 about 560us
11.        i += 2;
12.    }
13.    IR_TxData.DataBuf[30] = 0x80000000 | 0x15; //21 about 560us
14.    IR_TxData.DataBuf[31] = 0x00000000 | 0x40; //64 about 1690us
15.    IR_TxData.DataBuf[62] = 0x80000000 | 0x15; //21 about 560us
16.    IR_TxData.DataBuf[63] = 0x00000000 | 0x40; //64 about 1690us

```

```

17.     IR_TxData.DataBuf[64] = 0x80000000 | 0x15; //21 about 560us
18.     IR_TxData.DataBuf[65] = 0x00000000 | 0x40; //64 about 1690us
19.     IR_TxData.DataBuf[66] = 0x80000000 | 0x15; //21 about 560us
20.     IR_TxData.DataBuf[IR_TxData.DataLen - 1] = 0x00000000 | 0x15;
21.     .....
22. }

```

2. 引脚定义。

```

1.     #define IR_TX_PIN          P2_5

```

3. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、无内部上拉、输出使能、输出低。

4. 配置 PINMUX：分配引脚为 IRDA_TX 功能。

```

1.     void board_ir_init(void)
2.     {
3.         Pad_Config(IR_TX_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
                     PAD_PULL_NONE, PAD_OUT_ENABLE, PAD_OUT_LOW);
4.         Pinmux_Config(IR_TX_PIN, IRDA_TX);
5.     }

```

5. 使能 IR 时钟。

- 1) 初始化 IR 外设。
- 2) IR_Freq 设置为 vFreq，即 IR 发送频率为 38KHz。
- 3) IR_DutyCycle 设置为 2，即 IR 载波占空比为 1/2。
- 4) IR_Mode 设置为 IR_MODE_TX，即 IR 模式为 IR 发送模式。
- 5) IR_TxInverse 设置为 IR_TX_DATA_NORMAL，即不反转 IR 发送数据。
- 6) IR_TxFIFOThrLevel 设置为 IR_TX_FIFO_THR_LEVEL，即 IR 发送 FIFO 阈值为 2。

6. 使能 IR 外设发送功能。

```

1.     void driver_ir_init(uint32_t vFreq)
2.     {
3.         RCC_PeriphClockCmd(APBPeriph_IR, APBPeriph_IR_CLOCK, ENABLE);
4.         IR_InitTypeDef IR_InitStruct;
5.         IR_StructInit(&IR_InitStruct);
6.         IR_InitStruct.IR_Freq      = vFreq;
7.         IR_InitStruct.IR_DutyCycle = 2;
8.         IR_InitStruct.IR_Mode      = IR_MODE_TX;
9.         IR_InitStruct.IR_TxInverse = IR_TX_DATA_NORMAL;
10.        IR_InitStruct.IR_TxFIFOThrLevel = IR_TX_FIFO_THR_LEVEL;
11.        IR_Init(&IR_InitStruct);

```

```

12.      .....
13.      IR_Cmd(IR_MODE_TX, ENABLE);
14.  }

```

7. 执行 IR_SendBuf 函数，开始往 IR 发送 FIFO 中塞发送数据。

8. 使能 IR 发送 FIFO 数据个数小于设置的发送阈值中断（IR_INT_TF_LEVEL）。

```

1.  void ir_demo(void)
2.  {
3.      .....
4.      IR_SendBuf(IR_TxData.DataBuf, IR_TX_FIFO_SIZE, DISABLE);
5.      IR_TX_Count = IR_TX_FIFO_SIZE;
6.      IR_INTConfig(IR_INT_TF_LEVEL, ENABLE);
7.  }

```

9. 当 IR 发送 FIFO 数据个数小于设置的发送阈值（此例中设为 2）时，触发 IR_INT_TF_LEVEL 中断，进入中断处理函数 IR_Handler。

1) 屏蔽 IR_INT_TF_LEVEL 中断。

2) 判断 IR_INT_TF_LEVEL 中断状态位是否为 SET，若为 SET（需往 IR 发送 FIFO 中塞数据）

(1) 如果剩余发送数据个数大于发送 FIFO 大小，往 IR 发送 FIFO 中塞(IR_TX_FIFO_SIZE-IR_TX_FIFO_THR_LEVEL)发送数据，清除 IR_INT_TF_LEVEL 中断挂起位。

(2) 否则如果剩余发送数据个数大于 0，往 IR 发送 FIFO 中塞剩余数据，清除 IR_INT_TF_LEVEL 中断挂起位。

(3) 否则失能 IR_INT_TF_LEVEL 中断，清除 IR_INT_TF_LEVEL 中断挂起位。

3) 取消屏蔽 IR_INT_TF_LEVEL 中断源。

```

1.  void IR_Handler(void)
2.  {
3.      IR_MaskINTConfig(IR_INT_TF_LEVEL, ENABLE);
4.      if (IR_GetINTStatus(IR_INT_TF_LEVEL) == SET)
5.      {
6.          if ((IR_TxData.DataLen - IR_TX_Count) >= IR_TX_FIFO_SIZE)
7.          {
8.              IR_SendBuf(IR_TxData.DataBuf+IR_TX_Count, (IR_TX_FIFO_SIZE-2),DISABLE);
9.              IR_TX_Count += (IR_TX_FIFO_SIZE - IR_TX_FIFO_THR_LEVEL);
10.             IR_ClearINTPendingBit(IR_INT_TF_LEVEL_CLR);
11.         }
12.         else if ((IR_TxData.DataLen - IR_TX_Count) > 0)
13.         {
14.             IR_SetTxThreshold(0);
15.             IR_SendBuf(IR_TxData.DataBuf+IR_TX_Count,

```

```
IR_TxData.DataLen-IR_TX_Count, DISABLE);  
16.     IR_TX_Count += (IR_TxData.DataLen - IR_TX_Count);  
17.     IR_ClearINTPendingBit(IR_INT_TF_LEVEL_CLR);  
18. }  
19. else  
20. {  
21.     IR_INTConfig(IR_INT_TF_LEVEL, DISABLE);  
22.     IR_TX_Count = 0;  
23.     IR_ClearINTPendingBit(IR_INT_TF_LEVEL_CLR);  
24. }  
25. }  
26. IR_MaskINTConfig(IR_INT_TF_LEVEL, DISABLE);  
27. }
```

13.1.3 实验现象

使用 IR 遥控编码分析仪抓取 IR 发送的数据，并观察发送数据是否符合预期。

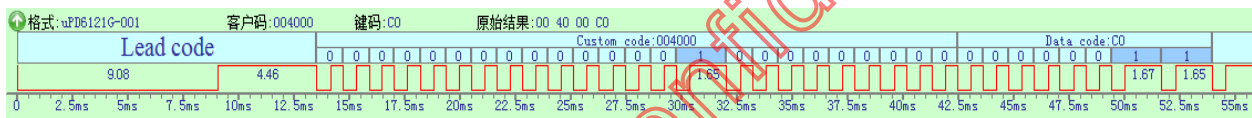


图 13-1 IR 发送数据编码

13.2 红外接收

使用 IR 外设进行数据接收，实现 RX 功能。使用 IR 遥控器向 IR 收发模块发送 IR 数据，通过 IR 中断接收 IR 数据并向 app_task 发送消息事件，app_task 检测到消息事件，在 app 层解析消息，执行用户程序：在 DebugAnalyser 工具上，打印接收到的 IR 数据信息。

工程目录：\HoneyComb\sdk\board\evb\io_sample\IR\Rx

13.2.1 硬件设计

硬件连接：P2_5→IR 收发模块的接收端。

EVb 外接 IR 收发模块，连接 P2_5 和 IR 收发模块的接收端，连接 VCC 和 IR 收发模块的 VCC，连接 GND 和 IR 收发模块的 GND。

13.2.2 软件流程

1. 初始化 IR 全局数据。

```
1. void global_data_ir_init(void)
2. {
3.     APP_PRINT_INFO0("[io_ir]global_data_ir_init");
4.     memset(&IR_Rx_Data, 0, sizeof(IR_Rx_Data));
5.     IR_RX_Count = 0;
6. }
```

2. 配置 PAD、PINMUX，设置 P2_5 为 IRDA_RX 功能。

3. app_task 初始化。

```
1. os_task_create(&app_task_handle,"app",app_main_task,0,
                APP_TASK_STACK_SIZE,APP_TASK_PRIORITY);
```

4. 在 app_main_task 中执行 driver_init 函数，执行 driver_ir_init 函数，对 IR 外设进行初始化。

- 1) 设置 IR 接收频率为 38KHz，IR 模式为 IR 接收模式，启动方式为 IR 自动接收模式，数据接收触发方式为下降沿触发。
- 2) 设置 IR 接收 FIFO 阈值为 30，当 IR 接收 FIFO 满时丢弃最新数据，过滤低于 50ns 的杂波数据。
- 3) 设置触发接收计数器阈值中断的电平类型为高电平，IR_RxCntThr 设置为 0x1F40。

5. 使能 IR 外设接收功能，清除 IR 接收 FIFO。

```

1.  void driver_ir_init(void)
2.  {
3.      .....
4.      IR_InitStruct.IR_Freq          = 38000; /* IR carrier frequency is 38KHz */
5.      IR_InitStruct.IR_Mode          = IR_MODE_RX; /* IR receiveing mode */
6.      IR_InitStruct.IR_RxStartMode    = IR_RX_AUTO_MODE;
7.      IR_InitStruct.IR_RxFIFOThrLevel = IR_RX_FIFO_THR_LEVEL;
8.      IR_InitStruct.IR_RxFIFOFullCtrl = IR_RX_FIFO_FULL_DISCARD_NEWEST;
9.      IR_InitStruct.IR_RxFilterTime   = IR_RX_FILTER_TIME_50ns;
10.     IR_InitStruct.IR_RxTriggerMode   = IR_RX_FALL_EDGE;
11.     IR_InitStruct.IR_RxCntThrType     = IR_RX_Count_High_Level;
12.     IR_InitStruct.IR_RxCntThr        = 0x1F40;
13.     .....
14.     IR_Cmd(IR_MODE_RX, ENABLE);
15.     IR_ClearRxFIFO();
16. }

```

6. 开始任务调度。

```

1.  os_sched_start();

```

7. 在 stack 准备好时，执行 app_handle_dev_state_evt 函数，执行 ir_demo。

- 1) 使能 IR 接收 FIFO 数据个数大于设置的接收阈值中断（IR_INT_RF_LEVEL）和接收计数器达到设定阈值中断（IR_INT_RX_CNT_THR）。
- 2) 取消屏蔽 IR_INT_RF_LEVEL 和 IR_INT_RX_CNT_THR 中断。

```

1.  void ir_demo(void)
2.  {
3.      IR_INTConfig(IR_INT_RF_LEVEL | IR_INT_RX_CNT_THR, ENABLE);
4.      IR_MaskINTConfig(IR_INT_RF_LEVEL | IR_INT_RX_CNT_THR, DISABLE);
5.  }

```

8. 使用 IR 遥控器向 IR 收发模块发送 IR 数据。

9. 当 IR 接收 FIFO 的数据个数达到设置的接收阈值（此例中设为 30）时，触发 IR_INT_RF_LEVEL 中断，或当接收计数器的个数达到设置的阈值（此例中设为 0x1F40）时，触发 IR_INT_RX_CNT_THR 中断，进入中断处理函数 IR_Handler。

- 1) 屏蔽 IR_INT_RF_LEVEL 和 IR_INT_RX_CNT_THR 中断。
- 2) 判断 IR_INT_RF_LEVEL 中断状态位是否为 SET，若为 SET（需从 IR 接收 FIFO 中取数据）

- (1) 从 IR 接收 FIFO 中取数据到 IR_Rx_Data 中。
- (2) 清除 IR_INT_TF_LEVEL 中断挂起位。
- 3) 判断 IR_INT_RX_CNT_THR 中断状态位是否为 SET，若为 SET（取剩余数据）
 - (1) 从 IR 接收 FIFO 中取剩余数据到 IR_Rx_Data 中。
 - (2) 定义消息类型 IO_MSG_TYPE_IR，保存数据，发送 msg 给 task。
 - (3) 清除 IR_INT_RX_CNT_THR 中断挂起位。
- 4) 取消屏蔽 IR_INT_RF_LEVEL 和 IR_INT_RX_CNT_THR 中断。

```

1.  void IR_Handler(void)
2.  {
3.      IR_MaskINTConfig(IR_INT_RF_LEVEL | IR_INT_RX_CNT_THR, ENABLE);
4.      if (IR_GetINTStatus(IR_INT_RF_LEVEL) == SET)
5.      {
6.          len = IR_GetRxDataLen();
7.          IR_ReceiveBuf(IR_Rx_Data.DataBuf + IR_RX_Count, len);
8.          IR_Rx_Data.DataLen += len;
9.          IR_RX_Count += len;
10.         IR_ClearINTPendingBit(IR_INT_RF_LEVEL_CLR);
11.     }
12.     if (IR_GetINTStatus(IR_INT_RX_CNT_THR) == SET)
13.     {
14.         len = IR_GetRxDataLen();
15.         IR_ReceiveBuf(IR_Rx_Data.DataBuf + IR_RX_Count, len);
16.         IR_Rx_Data.DataLen += len;
17.         IR_RX_Count += len;
18.         T_IO_MSG int_ir_msg;
19.         int_ir_msg.type = IO_MSG_TYPE_IR;
20.         int_ir_msg.u.buf = (void *)&IR_Rx_Data;
21.         if (false == app_send_msg_to_apptask(&int_ir_msg))
22.             .....
23.         IR_ClearINTPendingBit(IR_INT_RX_CNT_THR_CLR);
24.     }
25.     IR_MaskINTConfig(IR_INT_RF_LEVEL | IR_INT_RX_CNT_THR, DISABLE);
26. }

```

10. app_main_task 循环检测 msg queue，当有 msg 时，执行 app_handle_io_msg(io_msg)函数。

11. 在 app_handle_io_msg 函数中，当消息类型为 IO_MSG_TYPE_IR 时，执行 io_handle_ir_msg，打印接收到的 IR 数据信息。

```

1.  void io_handle_ir_msg(T_IO_MSG *io_ir_msg)
2.  {
3.      IR_Data_TypeDef *p_buf = io_ir_msg->u.buf;

```

```
4.     for (uint16_t i = 0; i < p_buf->DataLen; i++)
5.     {
6.         APP_PRINT_INFO2("[io_ir]io_handle_ir_msg:
           IR RX data[%d] = 0x%x",
           i, p_buf->DataBuf[i]);
7.     }
8.     .....
9. }
```

13.2.3 实验现象

使用 IR 遥控器向 IR 收发模块发送 IR 数据。在 DebugAnalyser 工具上打印接收到的 IR 数据信息。

Realtek Confidential

13.3 红外编码协议发送

使用 IR 外设的 TX 功能和 IR 编码协议库实现 IR 编码协议发送。发送定义的 RAW 编码协议，发送完成，通过 IR 中断向 app_task 发送消息事件，app_task 检测到消息事件，在 app 层解析消息，执行用户程序：发送 IR 编码协议，并观察发送数据是否符合预期。

工程目录：\HoneyComb\sdk\board\evb\io_sample\IR\SendCode

13.3.1 硬件设计

硬件连接：P2_5→IR 收发模块的发送端，P0_1→LED0。

EVB 外接 IR 收发模块，连接 P2_5 和 IR 收发模块的发送端，连接 VCC 和 IR 收发模块的 VCC，连接 GND 和 IR 收发模块的 GND。

在 EVB 上，使用跳帽短接 J24，连接 P0_1 和 LED0。

13.3.2 软件流程

1. 配置 PAD、PINMUX，设置 P0_1 为 GPIO 输出功能，初始化 GPIO 外设，初始状态：P0_1 置零。
2. app_task 初始化。

```
1. os_task_create(&app_task_handle,"app",app_main_task,0,
    APP_TASK_STACK_SIZE,APP_TASK_PRIORITY);
```

3. 在 app_main_task 中，执行 IR_Send_Init 函数。

- 1) 执行 UIR_TransInit：初始化 IR 外设，包括 PAD、PINMXU、DRIVER；初始化 TIM 外设，用于连续发送编码。
- 2) 调用 UIR_RegisterIRIntrHandlerCB(sendMsgToAppTaskFromISR)函数，注册 IR 中断处理回调函数，实现在 IR 中断处理函数中发送 msg 到 app task。

```
1. void IR_Send_Init(void)
2. {
3.     UIR_TransInit();
4.     UIR_RegisterIRIntrHandlerCB(sendMsgToAppTaskFromISR);
5. }
```

4. 在 UIR_TransInit 中执行 Board_IR_Init 函数，配置 PAD，PINMUX，设置 P2_5 为 IRDA_TX 功能。

5. 在 UIR_TransInit 中执行 Driver_IR_Init 函数，对 IR 外设进行初始化。
 - 1) 设置 IR 发送频率为 38K，IR 载波占空比为 1/2，IR 模式为 IR 发送模式。
 - 2) 设置不反转 IR 发送数据，设置空闲状态输出低电平。
 - 3) 设置 IR 发送 FIFO 的阈值为 2。
6. 使能 IR 外设发送功能，更新 IR 中断处理函数 IR_Handler 到 IR 中断向量表 IR_VECTORn。

```

1.  void Driver_IR_Init(uint32_t freq)
2.  {
3.      .....
4.      IR_InitStruct.IR_Freq      = freq;
5.      IR_InitStruct.IR_DutyCycle  = 2; /* !< 1/2 duty cycle */
6.      IR_InitStruct.IR_Mode      = IR_MODE_TX;
7.      IR_InitStruct.IR_TxInverse  = IR_TX_DATA_NORMAL;
8.      IR_InitStruct.IR_TxFIFOThrLevel = IR_TX_FIFO_THR_LEVEL;
9.      IR_InitStruct.IR_TxIdleLevel = IR_IDLE_OUTPUT_LOW;
10.     .....
11.     IR_Cmd(IR_MODE_TX, ENABLE);
12.     RamVectorTableUpdate(IR_VECTORn, IR_Handler);
13.     .....
14. }

```

7. 开始任务调度。

```

1.  os_sched_start();

```

8. 在 stack 准备好时，执行 app_handle_dev_state_evt 函数，执行 IR_Raw_Packet_Send(ir_raw_data_buf, ...)函数，发送 ir_raw_data_buf 数据。
 - 1) 启动 IR 编码协议发送，并使能 IR 发送 FIFO 数据个数小于设置的发送阈值中断（IR_INT_TF_LEVEL）。
 - 2) 判断状态为 NO_ERROR 时，翻转 P0_1（LED0）。

```

1.  void IR_Raw_Packet_Send(uint32_t *pBuf, uint32_t len)
2.  {
3.      .....
4.      /* Enable IR threshold interrupt: IR_INTConfig(IR_INT_TF_LEVEL, ENABLE) */
5.      status = UIR_OneFrameTransmitting(&uir_param_info, DISABLE);
6.      if (UIR_STATUS_NO_ERROR == status)
7.      {
8.          LED_IR_Send_Swap();
9.      }
10. }

```

9. 当 IR 发送 FIFO 数据个数小于设置的发送阈值（此例中设为 2）时，触发 IR_INT_TF_LEVEL 中断，或当 IR 发送 FIFO 为空时，触发 IR_INT_TF_EMPTY 中断，进入中断处理函数 IR_Handler。
 - 1) 屏蔽 IR_INT_TF_EMPTY 和 IR_INT_TF_LEVEL 中断。
 - 2) 判断 IR_INT_TF_LEVEL 中断状态位是否为 SET，若为 SET（详情参考：IR→TX）
 - (1) 如果剩余发送数据个数大于发送 FIFO 大小，往 IR 发送 FIFO 中塞满数据；否则如果剩余发送数据个数大于 0，往 IR 发送 FIFO 中塞剩余数据；否则失能 IR_INT_TF_LEVEL 中断，并使能 IR_INT_TF_EMPTY 中断。
 - (2) 清除 IR_INT_TF_LEVEL 中断挂起位。
 - 3) 判断 IR_INT_TF_EMPTY 中断状态位是否为 SET，若为 SET（发送 IR 编码协议后，发送 Repeat code，发送 msg 给 task）
 - (1) 发送 Repeat code。
 - (2) 失能 IR_INT_TF_EMPTY 中断，清除 IR_INT_TF_EMPTY 中断挂起位。
 - (3) 调用 pfnIRIntrHandlerCB 函数：定义消息类型 IO_MSG_TYPE_IR，子类型 IO_MSG_TYPE_IR_SEND_COMPLETE，发送 msg 给 task。
 - 4) 取消屏蔽 IR_INT_TF_EMPTY 和 IR_INT_TF_LEVEL 中断。

```

1.  void IR_Handler(void)
2.  {
3.      IR_MaskINTConfig(IR_INT_TF_EMPTY | IR_INT_TF_LEVEL, ENABLE);
4.      if (IR_GetINTStatus(IR_INT_TF_LEVEL) == SET)
5.          .....
6.      if (IR_GetINTStatus(IR_INT_TF_EMPTY) == SET)
7.      {
8.          if (IR_DataStruct.isSendRepeatcode)
9.          {
10.             if ((uir_param_info_base.uir_protocol_index ==
11.                 UIR_CODESET_TYPE_LC7464M) || \
12.                 (uir_param_info_base.uir_protocol_index ==
13.                 UIR_CODESET_TYPE_DVB) || \
14.                 (uir_param_info_base.uir_protocol_index ==
15.                 UIR_CODESET_TYPE_MIT_C8D8) || \
16.                 (uir_param_info_base.uir_protocol_index ==
17.                 UIR_CODESET_TYPE_KONICA))
18.             {
19.                 UIR_BurstSendCmd(ProtocolLib[uir_param_info_base.uir_protocol_index]
20.                                     .repeat_interval, ENABLE);

```

```

21.         pfnIRIntrHandlerCB();
22.     }
23. }
24.     IR_MaskINTConfig(IR_INT_TF_LEVEL | IR_INT_TF_EMPTY, DISABLE);
25. }
```

10. app_main_task 循环检测 msg queue，当有 msg 时，执行 app_handle_io_msg(io_msg)函数。

11. 在 app_handle_io_msg 函数中，判断消息类型为 IO_MSG_TYPE_IR，子类型为 IO_MSG_TYPE_IR_SEND_COMPLETE，执行 ir_send_msg_proc 函数，执行 NEC_SendCode。

- 1) 定义 uir_param_info 并赋值。
- 2) 发送 1 帧 IR 编码协议。
- 3) 发送连续 IR 编码协议。

```

1.  void NEC_SendCode(void)
2.  {
3.      memset(&uir_param_info, 0, sizeof(uir_param_info));
4.      uir_param_info.uir_protocol_index = UIR_CODESET_TYPE_NEC;
5.      uir_param_info.custom_code_value[0] = 0x55;
6.      uir_param_info.custom_code_value[1] = (uint8_t)(~(0x55));
7.      uir_param_info.custom_code_length = 16;
8.      uir_param_info.key_code_value[0] = 0xAA;
9.      uir_param_info.key_code_value[1] = (uint8_t)(~(0xAA));
10.     uir_param_info.key_code_length = 16;
11.     .....
12.     UIR_OneFrameTransmitting(&uir_param_info, DISABLE);
13.     status = UIR_StartContinuouslyTransmitting(&uir_param_info);
14.     status = UIR_StopContinuouslyTransmitting();
15. }
```

13.3.3 实验现象

使用 IR 遥控编码分析仪接收 IR 发送的数据，验证发送数据是否符合预期，如图 13-2 所示。

- ① 为发送 1 帧 ir_raw_data_buf 波形
- ② 为连续发送 uir_param_info 波形
- ③ 为发送 1 帧 uir_param_info 波形

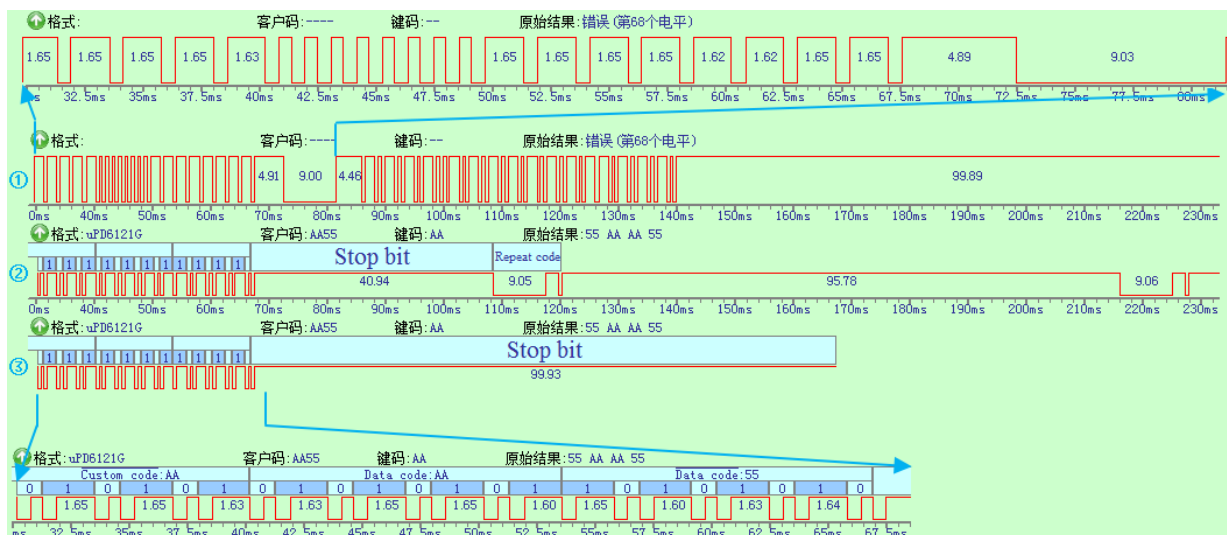


图 13-2 IR 发送数据编码

13.4 红外学习

使用 IR 外设的 Rx 功能实现 IR 学习。使用 IR 遥控器向 IR 收发模块发送 IR 数据，通过 IR 中断接收 IR 学习数据并向 app_task 发送消息事件，app_task 检测到消息事件，在 app 层解析消息，执行用户程序：对 IR 学习解码，在 DebugAnalyser 工具上，打印 IR 学习的解码后数据信息。

工程目录：\HoneyComb\sdk\board\evb\io_sample\IR\Learn

13.4.1 硬件设计

硬件连接：P2_5→IR 收发模块（带学习功能）的接收端，P0_2→LED0。

EVB 外接 IR 收发模块，连接 P2_5 和 IR 收发模块的接收端，连接 VCC 和 IR 收发模块的 VCC，连接 GND 和 IR 收发模块的 GND。

在 EVB 上，使用跳帽短接 J24，连接 P0_2 和 LED0。

13.4.2 软件流程

1. 配置 PAD、PINMUX，设置 P0_2 为 GPIO 输出功能，初始化 GPIO 外设，初始状态：P0_2 置零。
2. 配置 PAD、PINMUX：设置 P2_5 为 IRDA_RX 功能。
3. app_task 初始化。

```
1. os_task_create(&app_task_handle,"app",app_main_task,0,
    APP_TASK_STACK_SIZE,APP_TASK_PRIORITY);
```

4. 在 app_main_task 中执行 driver_init 函数，执行 ir_learn_timer_init 函数，初始化 IR 软件定时器，定时时间 20s，回调函数为 ir_learn_timer_callback。

```
1. os_timer_create(&IR_Learn_Timer, "ir_learn_timer", 1 ,
    IR_LEARN_TIMEOUT, false, ir_learn_timer_callback);
```

5. 在回调函数 ir_learn_timer_callback 中，执行 ir_learn_exit 函数，用于学习超时退出 IR 学习。

```
1. static void ir_learn_timer_callback(TimerHandle_t pxTimer)
2. {
3.     ir_learn_exit();
4. }
```

6. 在 driver_init 中，执行 ir_learn_module_init 函数。

- 1) 启动定时器：IR_Learn_Timer。
- 2) 执行 ir_trans_rx_handler_cb(ir_learn_send_msg_from_isr)函数，注册 IR 中断处理回调函数，实现在 IR 中断处理函数中发送 msg 到 app task。
- 3) 执行 ir_learn_init 函数，执行 ir_trans_rx_init 函数。
 - (1) 初始化 loop queue。
 - (2) 更新 IR 中断处理函数 IR_Handler 到 IR 中断向量表 IR_VECTORn。
 - (3) 初始化 IR 外设。
 - a. 设置 IR 接收频率为 40MHz，占空比为 1/2，接收模式，自动接收，下降沿触发，接收 FIFO 阈值为 20，计数器阈值为 240000 等。
 - b. 使能 IR 外设接收功能，清除 IR 接收 FIFO。
 - c. 使能 IR 接收 FIFO 的数据个数达到设置的接收阈值中断（IR_INT_RF_LEVEL）和接收计数器的个数达到设置的阈值中断（IR_INT_RX_CNT_THR），同时取消屏蔽 IR_INT_RF_LEVEL 和 IR_INT_RX_CNT_THR 中断。

```

1. void ir_learn_module_init(void)
2. {
3.     if (true == os_timer_start(&IR_Learn_Timer))
4.         .....
5.     ir_trans_rx_handler_cb(ir_learn_send_msg_from_isr);
6.     ir_learn_init();
7. }

```

7. 开始任务调度。

```

1. os_sched_start();

```

8. 使用 IR 遥控器向 IR 收发模块发送 IR 数据。

9. 当 IR 接收 FIFO 的数据个数达到设置的接收阈值（此例中设为 20）时，触发 IR_INT_RF_LEVEL 中断，或当接收计数器的个数达到设置的阈值（此例中设为 240000）时，触发 IR_INT_RX_CNT_THR 中断，进入中断处理函数 IR_Handler。

- 1) 屏蔽 IR_INT_RF_LEVEL 和 IR_INT_RX_CNT_THR 中断。
- 2) 判断 IR_INT_RF_LEVEL 中断状态位是否为 SET，若为 SET，则
 - (1) 接收 IR 数据，入列到 IR_RX_Queue 中。
 - (2) 清除 IR_INT_RF_LEVEL 中断挂起位。
- 3) 判断 IR_INT_RX_CNT_THR 中断状态位是否为 SET，若为 SET，则
 - (1) 接收 IR 剩余数据，入列到 IR_RX_Queue 中。
 - (2) 清除 IR_INT_RX_CNT_THR 中断挂起位。

- 4) 调用 pFn_IR_RX_Handler_CB 函数，定义消息类型 IO_MSG_TYPE_IR，子类型 IO_MSG_TYPE_IR_LEARN_STOP 或 IO_MSG_TYPE_IR_LEARN_DATA，发送 msg 给 task。
- 5) 取消屏蔽 IR_INT_RF_LEVEL 和 IR_INT_RX_CNT_THR 中断。

```

1. void IR_RX_Handler(void)
2. {
3.     IR_MaskINTConfig(IR_INT_RF_LEVEL | IR_INT_RX_CNT_THR, ENABLE);
4.     if (IR_GetINTStatus(IR_INT_RF_LEVEL) == SET)
5.     {
6.         data_len = IR_GetRxDataLen();
7.         ir_loop_queue_data_in(&IR_RX_Queue, data_len);
8.         IR_ClearINTPendingBit(IR_INT_RF_LEVEL_CLR);
9.     }
10.    if (IR_GetINTStatus(IR_INT_RX_CNT_THR) == SET)
11.    {
12.        data_len = IR_GetRxDataLen();
13.        ir_loop_queue_data_in(&IR_RX_Queue, data_len);
14.        IR_ClearINTPendingBit(IR_INT_RX_CNT_THR_CLR);
15.        ir_learn_end_flag = true;
16.    }
17.    if (pFn_IR_RX_Handler_CB)
18.    {
19.        pFn_IR_RX_Handler_CB(ir_learn_end_flag);
20.    }
21.    IR_MaskINTConfig(IR_INT_RF_LEVEL | IR_INT_RX_CNT_THR, DISABLE);
22. }

```

10. app_main_task 循环检测 msg queue，当有 msg 时，执行 app_handle_io_msg(io_msg)函数。

11. 在 app_handle_io_msg 中，消息类型为 IO_MSG_TYPE_IR，执行 ir_learn_handle_msg，判断子类型。

- 1) 子类型为 IO_MSG_TYPE_IR_LEARN_DATA
如果 IR 学习解码状态为特定状态，停止 IR 学习。
- 2) 子类型为 IO_MSG_TYPE_IR_LEARN_STOP
 - (1) IR 学习数据解码、获取频率、数据转换等。
 - (2) 打印 IR 学习的频率、数据长度、数据等信息。
 - (3) 退出 IR 学习。
 - (4) 重新执行 ir_learn_module_init 函数：启动定时器，注册回调函数，初始化 IR 外设等。

```

1. bool ir_learn_handle_msg(T_IO_MSG *io_ir_msg)
2. {
3.     if (sub_type == IO_MSG_TYPE_IR_LEARN_DATA)

```



```

4.      {
5.          status = ir_learn_decode(&IR_Learn_Packet);
6.          if (status == IR_LEARN_EXCEED_SIZE)
7.              ir_learn_deinit();
8.          .....
9.      }
10.     else if (sub_type == IO_MSG_TYPE_IR_LEARN_STOP)
11.     {
12.         ir_learn_decode(&IR_Learn_Packet); /* Pick up the last ir data */
13.         ir_learn_freq(&IR_Learn_Packet); /* Decode IR carrier frequency */
14.         ir_learn_data_convert(&IR_Learn_Packet); /* Data reduction */
15.         .....
16.         for (uint32_t i = 0; i < IR_Learn_Packet.buf_index; i++) /* Print decode result */
17.             APP_PRINT_INFO2("learn data%d: 0x%x.", i, IR_Learn_Packet.ir_buf[i]);
18.         ir_learn_exit();
19.         ir_learn_module_init();
20.     }

```

12. 在 app_handle_io_msg 中，消息类型为 IO_MSG_TYPE_IR，执行 led_ir_learn_swap：翻转 P0_2。

```

1.     void app_handle_io_msg(T_IO_MSG io_msg)
2.     {
3.         case IO_MSG_TYPE_IR:
4.         {
5.             if (ir_learn_handle_msg(&io_msg))
6.             {
7.                 extern void led_ir_learn_swap(void);
8.                 led_ir_learn_swap();
9.             }
10.        }
11.        .....
12.    }

```

13.4.3 实验现象

使用 IR 遥控器向 IR 收发模块发送 IR 数据，对 IR 学习解码，在 DebugAnalyser 工具上，打印 IR 学习的解码后数据信息。

13.5 红外 GDMA 发送

使用 IR 外设利用 GDMA 搬运发送数据，实现 TX 功能。使用红外压控编码分析仪接收 IR 发送的数据，并观察发送数据是否符合预期。

工程目录：\HoneyComb\sdk\board\evb\io_sample\IR\Tx+GDMA

13.5.1 硬件设计

硬件连接：P2_5→IR 收发模块的发送端。

EVB 外接 IR 收发模块，连接 P2_5 和 IR 收发模块的发送端，连接 VCC 和 IR 收发模块的 VCC，连接 GND 和 IR 收发模块的 GND。

13.5.2 软件流程

1. 定义 IR 发送数据数组，并赋值：有载波数据用载波个数与 0x80000000 进行或运算表示，无载波数据用载波个数与 0x00000000 进行或运算表示，载波个数为：timing/CarrierFreq。

```

1.  void ir_demo(void)
2.  {
3.      IR_Send_Data.CarrierFreq = 38000;
4.      IR_Send_Data.DataLen = IO_TEST_GDMA_TRANSFER_SIZE;
5.      IR_Send_Data.DataBuf[0] = 0x80000000 | 0x200;
6.      IR_Send_Data.DataBuf[1] = 0x00000000 | 0x100;
7.      for (uint16_t i = 2; i < IR_Send_Data.DataLen - 1;)
8.      {
9.          IR_Send_Data.DataBuf[i] = 0x80000000 | (0x0A + i * 5);
10.         IR_Send_Data.DataBuf[i + 1] = 0x00000000 | (0x14 + i * 5);
11.         i += 2;
12.     }
13.     IR_Send_Data.DataBuf[IR_Send_Data.DataLen - 1] = 0x80000000 | 0x800;
14.     /* Test data buffer */
15.     for (uint32_t i = 0; i < IO_TEST_GDMA_TRANSFER_SIZE; i++)
16.     {
17.         GDMA_Send_Buf[i] = IR_Send_Data.DataBuf[i];
18.     }
19.     .....
20. }
```

2. 引脚定义。

```

1.  #define IR_TX_PIN          P2_5
```

3. 配置 PAD、PINMUX：设置引脚 P2_5 为 IR 发送引脚、PINMUX 模式、PowerOn、无内部上拉、输出使能、输出低，分配引脚 P2_5 为 IRDA_TX 功能。
4. 使能 IR 时钟。
5. 初始化 IR 外设。
 - 1) IR_Freq 设置为 vFreq，即 IR 发送频率为 38KHz。
 - 2) IR_DutyCycle 设置为 3，即 IR 载波占空比为 1/3。
 - 3) IR_Mode 设置为 IR_MODE_TX，即 IR 模式为 IR 发送模式。
 - 4) IR_TxInverse 设置为 IR_TX_DATA_NORMAL，即不反转 IR 发送数据。
 - 5) IR_TxFIFOThrLevel 设置为 IR_TX_FIFO_THR_LEVEL，即 IR 发送 FIFO 阈值为 2。
 - 6) IR_TxDmaEn 设置为 ENABLE，即使能 TX 的 GDMA 传输。
 - 7) IR_TxWaterLevel 设置为 15。
6. 使能 IR 外设发送功能。

```

1.  void driver_ir_init(uint32_t vFreq)
2.  {
3.      RCC_PeriphClockCmd(APBPeriph_IR, APBPeriph_IR_CLOCK, ENABLE);
4.      IR_InitTypeDef IR_InitStruct;
5.      IR_StructInit(&IR_InitStruct);
6.      IR_InitStruct.IR_Freq      = vFreq;
7.      IR_InitStruct.IR_DutyCycle = 2;
8.      IR_InitStruct.IR_Mode      = IR_MODE_TX;
9.      IR_InitStruct.IR_TxInverse = IR_TX_DATA_NORMAL;
10.     IR_InitStruct.IR_TxFIFOThrLevel = IR_TX_FIFO_THR_LEVEL;
11.     IR_InitStruct.IR_TxDmaEn      = ENABLE;
12.     IR_InitStruct.IR_TxWaterLevel = 15;
13.     IR_Init(&IR_InitStruct);
14.     .....
15. }

```

7. 执行 driver_ir_gdma_init 函数，对 GDMA 外设进行初始化。
 - 1) 使用 GDMA 通道 1。
 - 2) GDMA 的传输方向为内存到外设传输。
 - 3) GDMA_SourceAddr 设置为 GDMA_Send_Buf。
 - 4) GDMA_DestinationAddr 设 IR 的 TX_FIFO。
 - 5) 使能 GDMA 通道 1 总传输完成中断（GDMA_INT_Transfer）。
 - 6) 执行 GDMA_Cmd 函数：使能 GDMA 传输。

```

1.  void driver_ir_gdma_init(void)

```

```

2.  {
3.      GDMA_InitStruct.GDMA_ChannelNum    = IO_TEST_GDMA_CHANNEL_MUM;
4.      GDMA_InitStruct.GDMA_DIR           = GDMA_DIR_MemoryToPeripheral;
5.      .....
6.      GDMA_InitStruct.GDMA_SourceAddr     = (uint32_t) ( GDMA_Send_Buf);
7.      GDMA_InitStruct.GDMA_DestinationAddr = (uint32_t) (&IR->TX_FIFO);
8.      .....
9.      GDMA_INTConfig(IO_TEST_GDMA_CHANNEL_MUM, GDMA_INT_Transfer, ENABLE);
10.     .....
11.     GDMA_Cmd(IO_TEST_GDMA_CHANNEL_MUM, ENABLE);
12. }

```

8. 执行 IR_Cmd 函数：使能 IR 外设发送功能。

```

1.  void ir_demo(void)
2.  {
3.      .....
4.      IR_Cmd(IR_MODE_TX, ENABLE);
5.  }

```

9. 当 GDMA 搬运数据完成时，触发 GDMA_INT_Transfer 中断，进入 GDMA 中断处理函数 ADC_GDMA_Channel_Handler。

- 1) 使能 GDMA 通道 1 总传输完成中断（GDMA_INT_Transfer）。
- 2) 失能 GDMA 通道 1 传输。
- 3) 打印 GDMA 传输完成信息。
- 4) 清除 GDMA 通道 1 GDMA_INT_Transfer 中断挂起位。

```

1.  void IO_TEST_GDMA_Channel_Handler (void)
2.  {
3.      GDMA_INTConfig(IO_TEST_GDMA_CHANNEL_MUM, GDMA_INT_Transfer, DISABLE);
4.      GDMA_Cmd(IO_TEST_GDMA_CHANNEL_MUM, DISABLE);
5.      DBG_DIRECT("IO_TEST_GDMA_Channel_Handler\r\n");
6.      GDMA_ClearINTPendingBit(IO_TEST_GDMA_CHANNEL_MUM, GDMA_INT_Transfer);
7.  }

```

13.5.3 实验现象

在 DebugAnalyser 工具上，打印 GDMA 搬运数据完成信息；使用红外遥控编码分析仪接收 IR 发送的数据，并观察发送数据是否符合预期。

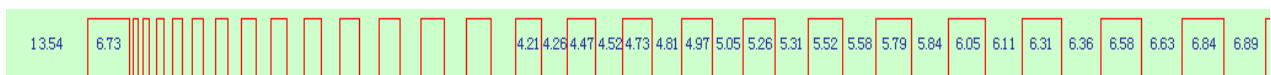


图 13-3 IR-GDMA 发送数据编码

13.6 红外 GDMA 接收

使用 IR 外设用 GDMA 搬运接收数据，实现 RX 功能。使用 IR 遥控器向 IR 收发模块发送 IR 数据，通过 GDMA 中断接收 IR 数据，在 DebugAnalyser 工具上，打印 GDMA 搬运 IR 数据信息。

工程目录：\HoneyComb\sdk\board\evb\io_sample\IR\Rx+GDMA

13.6.1 硬件设计

硬件连接：P2_5→IR 收发模块的接收端。

EVB 外接 IR 收发模块，连接 P2_5 和 IR 收发模块的接收端，连接 VCC 和 IR 收发模块的 VCC，连接 GND 和 IR 收发模块的 GND。

13.6.2 软件流程

1. 配置 PAD、PINMUX：设置 P2_5 为 IRDA_RX 功能。
2. 执行 driver_ir_init 函数，对 IR 外设进行初始化。
 - 1) 设置 IR 接收频率为 40MHz，IR 模式为 IR 接收模式，启动方式为 IR 自动接收模式，数据接收触发方式为下降沿触发。
 - 2) 设置 IR 接收 FIFO 阈值为 30，当 IR 接收 FIFO 满时丢弃最新数据，过滤低于 50ns 的杂波数据。
 - 3) 设置触发接收计数器阈值中断的电平类型为高电平，IR_RxCntThr 设置为 0xEEEEED8。
 - 4) IR_RxDmaEn 设置为 ENABLE，即使能 RX 的 GDMA 传输。
 - 5) IR_TxWaterLevel 设置为 4。
3. 使能 IR 外设接收功能，清除 IR 接收 FIFO。

```

1. void driver_ir_init(void)
2. {
3.     .....
4.     IR_InitStruct.IR_Freq          = 40000000; /* IR carrier frequency is 38KHz */
5.     IR_InitStruct.IR_Mode          = IR_MODE_RX; /* IR receiveing mode */
6.     IR_InitStruct.IR_RxStartMode    = IR_RX_AUTO_MODE;
7.     IR_InitStruct.IR_RxFIFOThrLevel = IR_RX_FIFO_THR_LEVEL;
8.     IR_InitStruct.IR_RxFIFOFullCtrl = IR_RX_FIFO_FULL_DISCARD_NEWEST;
9.     IR_InitStruct.IR_RxFilterTime   = IR_RX_FILTER_TIME_50ns;
10.    IR_InitStruct.IR_RxTriggerMode   = IR_RX_FALL_EDGE;
11.    IR_InitStruct.IR_RxCntThrType    = IR_RX_Count_High_Level;
12.    IR_InitStruct.IR_RxCntThr        = 0xEEEEED8;

```

```

13.     IR_InitStruct.IR_RxDmaEn      = ENABLE;
14.     IR_InitStruct.IR_RxWaterLevel  = 4;
15.     IR_Init(&IR_InitStruct);
16.     IR_Cmd(IR_MODE_RX, ENABLE);
17.     IR_ClearRxFIFO();
18.     }

```

4. 执行 driver_ir_gdma_init 函数，对 GDMA 外设进行初始化。

- 1) 使用 GDMA 通道 1。
- 2) GDMA 的传输方向为外设到内存传输。
- 3) GDMA_SourceAddr 设置为 IR 的 RX_FIFO。
- 4) GDMA_DestinationAddr 设置为 GDMA_Recv_Buf。
- 5) 使能 GDMA 通道 1 总传输完成中断（GDMA_INT_Transfer）。
- 6) 执行 GDMA_Cmd 函数：使能 GDMA 传输。

```

1.  void driver_ir_gdma_init(void)
2.  {
3.      GDMA_InitStruct.GDMA_ChannelNum    = IO_TEST_GDMA_CHANNEL_MUM;
4.      GDMA_InitStruct.GDMA_DIR           = GDMA_DIR_PeripheralToMemory
5.      .....
6.      GDMA_InitStruct.GDMA_SourceAddr    = (uint32_t) (&IR->RX_FIFO);
7.      GDMA_InitStruct.GDMA_DestinationAddr = (uint32_t) (GDMA_Recv_Buf);
8.      .....
9.      GDMA_INTConfig(IO_TEST_GDMA_CHANNEL_MUM, GDMA_INT_Transfer, ENABLE);
10.     .....
11.     GDMA_Cmd(IO_TEST_GDMA_CHANNEL_MUM, ENABLE);
12.     }

```

5. 当 GDMA 搬运数据完成时，触发 GDMA_INT_Transfer 中断，进入 GDMA 中断处理函数 IO_TEST_GDMA_Channel_Handler。

- 1) 失能 GDMA 通道 1 总传输完成中断（GDMA_INT_Transfer）。
- 2) 失能 GDMA 通道 1 传输。
- 3) 保存 GDMA 搬运数据长度。
- 4) 执行 io_handle_gdma_msg 函数，处理 GDMA 搬运数据。
- 5) 清除 GDMA 通道 1 GDMA_INT_Transfer 中断挂起位。

```

1.  void IO_TEST_GDMA_Channel_Handler (void)
2.  {
3.      GDMA_INTConfig(IO_TEST_GDMA_CHANNEL_MUM, GDMA_INT_Transfer, DISABLE);
4.      GDMA_Cmd(IO_TEST_GDMA_CHANNEL_MUM, DISABLE);
5.      IR_GDMA_Rev_Data_Len = IO_TEST_GDMA_TRANSFER_SIZE;

```

```

6.      io_handle_gdma_msg();
7.      GDMA_ClearINTPendingBit(IO_TEST_GDMA_CHANNEL_MUM, GDMA_INT_Transfer);
8.      }

```

6. 在 io_handle_gdma_msg 中，打印传输数据长度以及数据信息。

```

1.      void io_handle_gdma_msg (void)
2.      {
3.          DBG_DIRECT("io_handle_gdma_msg:
                      IR_GDMA_Rev_Data_Len = %d \r\n",
                      IR_GDMA_Rev_Data_Len);
4.          for (uint32_t i = 0; i < IR_GDMA_Rev_Data_Len; i++)
5.          {
6.              DBG_DIRECT("io_handle_gdma_msg:
                          GDMA_Recv_Buf[%d] = 0x%x \r\n",
                          i, GDMA_Recv_Buf[i]);
7.          }
8.      }

```

13.6.3 实验现象

使用 IR 遥控器向 IR 收发模块发送 IR 数据。在 DebugAnalyser 工具上，打印通过 GDMA 接收到的 IR 数据以及其长度信息。

14 集成电路内置音频总线（I2S）

专用硬件设备：逻辑分析仪

专用测试软件：DSview V1.1.2

14.1 音频数据传输

使用 I2S 输出数据，并用逻辑分析仪观察波形。

工程目录：\HoneyComb\sdk\board\evb\io_sample\I2S\Tx_polling

14.1.1 硬件设计

硬件连接：P3_2→Pin_frame，P3_3→Pin_clock，P4_0→Pin_data。

EVB 外接逻辑分析仪，连接 P3_2 和 Pin_frame，P3_3 和 Pin_clock，P4_0 和 Pin_data。

14.1.2 软件流程

1. 引脚定义。

```
1.  #define I2S_LRCK_PIN      P3_2
2.  #define I2S_BCLK_PIN     P3_3
3.  #define I2S_DATA_TX_PIN  P4_0
```

2. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、无内部上拉、输出使能、输出低。

3. 配置 PINMUX：分配引脚分别为 I2S_BCLK_PINMUX、I2S_LRCK_PINMUX、I2S_DATA_TX_PINMUX 功能。

```
1.  void board_i2s_init(void)
2.  {
3.      Pad_Config(I2S_BCLK_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
                  PAD_PULL_NONE, PAD_OUT_ENABLE, PAD_OUT_LOW);
4.      Pad_Config(I2S_LRCK_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
                  PAD_PULL_NONE, PAD_OUT_ENABLE, PAD_OUT_LOW);
5.      Pad_Config(I2S_DATA_TX_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
                  PAD_PULL_NONE, PAD_OUT_ENABLE, PAD_OUT_LOW);
6.      Pinmux_Config(I2S_BCLK_PIN, I2S_BCLK_PINMUX);
7.      Pinmux_Config(I2S_LRCK_PIN, I2S_LRCK_PINMUX);
```



```

8.     Pinmux_Config(I2S_DATA_TX_PIN, I2S_DATA_TX_PINMUX);
9.     }

```

4. 使能 I2S0 时钟。

5. 初始化 I2S0 外设。

- 1) I2S_ClockSource 设置为 I2S_CLK_40M, 即 I2S 时钟源为 40M。
- 2) I2S_BClockMi 设置为 0x271, I2S_BClockNi 设置为 0x10, 即

$$BCLK = I2S_ClockSource * (I2S_BClockNi / I2S_BClockMi) = 1024K, LRCK = BCLK/64 = 16K$$
- 3) I2S_DeviceMode 设置为 I2S_DeviceMode_Master, 即主设备模式。
- 4) I2S_ChannelType 设置为 I2S_Channel_stereo, 即传输通道类型为双声道输出。
- 5) I2S_DataWidth 设置为 I2S_Width_16Bits, 即数据宽度为 16bits 数据宽度。
- 6) I2S_DataFormat 设置为 I2S_Mode, 即数据格式为 I2S 格式。
- 7) I2S_DMAMCmd 设置为 I2S_DMA_DISABLE, 即失能 DMA 数据传输功能。

6. 使能 I2S0 发送模式。

```

1.     void driver_i2s_init(void)
2.     {
3.         RCC_PeriphClockCmd(APB_I2S, APB_I2S_CLOCK, ENABLE);
4.         I2S_InitTypeDef I2S_InitStruct;
5.         I2S_StructInit(&I2S_InitStruct);
6.         I2S_InitStruct.I2S_ClockSource    = I2S_CLK_40M;
7.         I2S_InitStruct.I2S_BClockMi      = 0x271;    /* <!LRCK = 16K */
8.         I2S_InitStruct.I2S_BClockNi      = 0x10;    /* <!BCLK = 1024K */
9.         I2S_InitStruct.I2S_DeviceMode    = I2S_DeviceMode_Master;
10.        I2S_InitStruct.I2S_ChannelType    = I2S_Channel_stereo;
11.        I2S_InitStruct.I2S_DataWidth      = I2S_Width_16Bits;
12.        I2S_InitStruct.I2S_DataFormat     = I2S_Mode;
13.        I2S_InitStruct.I2S_DMAMCmd       = I2S_DMA_DISABLE;
14.        I2S_Init(I2S_NUM, &I2S_InitStruct);
15.        I2S_Cmd(I2S_NUM, I2S_MODE_TX, ENABLE);
16.    }

```

7. 定义发送数据, 循环发送数据。

```

1.     void i2s_senddata(void)
2.     {
3.         uint32_t i = 0x12348800;
4.         while (1)
5.         {
6.             if (I2S_GetTxFIFOFreeLen(I2S_NUM))

```

```

7.      {
8.      /* 16bit format, lower half word send first! */
9.      I2S_SendData(I2S_NUM, i++);
10.     }
11.    }
12.   }

```

14.1.3 实验现象

配置逻辑分析仪，检测 I2S 输出，观察波形。

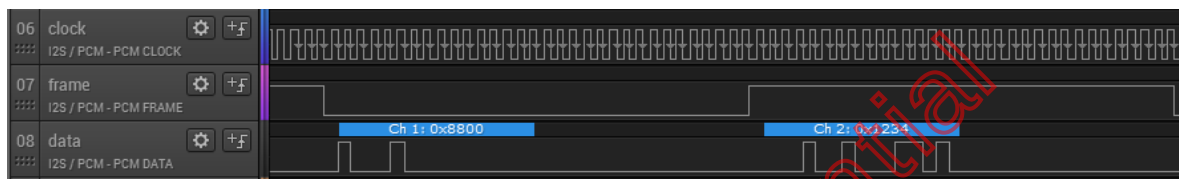


图 14-1 I2S 输出波形

14.2 从设备接收数据

使用 I2S0 作为从设备，检测到接收数据后进入中断处理函数中进行数据的接收。

工程目录：\HoneyComb\sdk\board\evb\io_sample\I2S\I2S_Slave_Recv

14.2.1 硬件设计

硬件连接：P3_2→Pin_frame，P3_3→Pin_clock，P4_0→Pin_data。

可外接一块 RTL87x2E 芯片作为 I2S 主设备发送数据，与此芯片进行通信。

14.2.2 软件流程

1. 引脚定义。

```
1.  #define I2S_LRCK_PIN      P3_2
2.  #define I2S_BCLK_PIN     P3_3
3.  #define I2S_DATA_TX_PIN  P4_0
```

2. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、无内部上拉、输出使能、输出低。

3. 配置 PINMUX：分配引脚分别为 I2S_BCLK_PINMUX、I2S_LRCK_PINMUX、I2S_DATA_TX_PINMUX 功能。

```
1.  void board_i2s_init(void)
2.  {
3.      Pad_Config(I2S_BCLK_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
4.                  PAD_PULL_NONE, PAD_OUT_ENABLE, PAD_OUT_LOW);
5.      Pad_Config(I2S_LRCK_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
6.                  PAD_PULL_NONE, PAD_OUT_ENABLE, PAD_OUT_LOW);
7.      Pad_Config(I2S_DATA_TX_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
8.                  PAD_PULL_NONE, PAD_OUT_ENABLE, PAD_OUT_LOW);
9.      Pinmux_Config(I2S_BCLK_PIN, I2S_BCLK_PINMUX);
10.     Pinmux_Config(I2S_LRCK_PIN, I2S_LRCK_PINMUX);
11.     Pinmux_Config(I2S_DATA_TX_PIN, I2S_DATA_TX_PINMUX);
12. }
```

4. 使能 I2S0 时钟。

5. 初始化 I2S0 外设。

- 1) I2S_ClockSource 设置为 I2S_CLK_40M，即 I2S 时钟源为 40M。
 - 2) I2S_BClockMi 设置为 0x271，I2S_BClockNi 设置为 0x10，即

$$BCLK = I2S_ClockSource * (I2S_BClockNi / I2S_BClockMi) = 1024K, LRCK = BCLK/64 = 16K$$
 - 3) I2S_DeviceMode 设置为 I2S_DeviceMode_Slave，即主设备模式。
 - 4) I2S_ChannelType 设置为 I2S_Channel_stereo，即传输通道类型为双声道输出。
 - 5) I2S_DataWidth 设置为 I2S_Width_16Bits，即数据宽度为 16bits 数据宽度。
 - 6) I2S_DataFormat 设置为 I2S_Mode，即数据格式为 I2S 格式。
 - 7) I2S_DMAMCmd 设置为 I2S_DMA_DISABLE，即失能 DMA 数据传输功能。
 - 8) 将 I2S 传输切换为外部接收模式。
6. 使能 I2S0 接收模式。
7. 开启 I2S RX 准备接收数据 (I2S_INT_RX_READY) 中断。

```

1.  void driver_i2s_init(void)
2.  {
3.      RCC_PeriphClockCmd(APB_I2S, APB_I2S_CLOCK, ENABLE);
4.      I2S_InitTypeDef I2S_InitStruct;
5.      I2S_StructInit(&I2S_InitStruct);
6.      I2S_InitStruct.I2S_ClockSource    = I2S_CLK_40M;
7.      I2S_InitStruct.I2S_BClockMi      = 0x271; /* <!LRCK = 16K */
8.      I2S_InitStruct.I2S_BClockNi      = 0x10; /* <!BCLK = 1024K */
9.      I2S_InitStruct.I2S_DeviceMode    = I2S_DeviceMode_Slave;
10.     I2S_InitStruct.I2S_ChannelType    = I2S_Channel_stereo;
11.     I2S_InitStruct.I2S_DataWidth      = I2S_Width_16Bits;
12.     I2S_InitStruct.I2S_DataFormat     = I2S_Mode;
13.     I2S_InitStruct.I2S_DMAMCmd       = I2S_DMA_DISABLE;
14.     I2S_Init(I2S_NUM, &I2S_InitStruct);
15.     I2S_Cmd(I2S_NUM, I2S_MODE_RX, ENABLE);
16.     I2S_INTConfig(I2S_NUM, I2S_INT_RX_READY, ENABLE);
17. }

```

8. 当接收到数据后，触发 I2S_INT_RX_READY，进入中断处理函数接收数据。

```

1.  void I2S0_RX_Handler (void)
2.  {
3.      if (I2S_GetINTStatus(I2S_NUM, I2S_INT_RX_READY))
4.      {
5.          uint8_t len = I2S_GetRxFIFOLen(I2S_NUM);
6.          DBG_DIRECT("length = %d", len);
7.          for (uint8_t i = 0; i < len; i++)
8.          {
9.              DBG_DIRECT("data[%d] = 0x%X", i, I2S_ReceiveData(I2S_NUM));

```

```
10.      }  
11.      I2S_ClearINTPendingBit(I2S_NUM, I2S_INT_RX_READY);  
12.      }  
13.      }
```

14.2.3 实验现象

当 I2S 开始接收数据时，在在 DebugAnalyser 工具上，打印接收数据信息。

Realtek Confidential

15 编解码器（CODEC）

专用硬件设备：FT232 usb 转串口、AMIC、DMIC

专用测试软件：串口调试工具、PCM_Data_Parser、Cool Edit Pro

15.1 模拟麦克风

实现 AMIC 采集模拟语音经 CODEC 编码的功能。AMIC 采集模拟语音数据，经 CODEC 编码，送到 I2S0 接收 FIFO，利用 GDMA 将数据搬运到 UART0，PC 端收到 UART 传输的数据，使用 PCM_Data_Parser 解析数据，使用 Cool Edit Pro 播放录制的语音。

工程目录：\HoneyComb\sdk\board\evb\io_sample\CODEC\AMIC

15.1.1 硬件设计

硬件连接：H0→MICBIAS，P2_6→MIC_P，P2_7→MIC_N，同时：P2_4→RX，P2_5→TX。

EVB 外接 AMIC 模块，连接 H0 和 MICBIAS，GND 和 GND，P2_6 和 MIC_P，P2_7 和 MIC_N；EVB 外接 FT232 模块，连接 P2_4 和 RX，P2_5 和 TX。

15.1.2 软件流程

1. 配置 PAD、PINMUX，设置 P2_4、P2_5 为 UART0 功能。
2. 配置 PAD：设置引脚，SW 模式、PowerOn、无内部上拉、输出使能或失能、输出高或低。

```

1. void board_codec_init(void)
2. {
3.     Pad_Config(H_0, PAD_SW_MODE, PAD_NOT_PWRON,
4.               PAD_PULL_NONE, PAD_OUT_ENABLE, PAD_OUT_HIGH);
5.     Pad_Config(P2_6, PAD_SW_MODE, PAD_NOT_PWRON,
6.               PAD_PULL_NONE, PAD_OUT_DISABLE, PAD_OUT_HIGH);
7.     Pad_Config(P2_7, PAD_SW_MODE, PAD_NOT_PWRON,
8.               PAD_PULL_NONE, PAD_OUT_DISABLE, PAD_OUT_LOW);
9. }
```

3. 初始化 UART 外设。

设置波特率为 3000000，使能 GDMA 发送功能等。

4. 初始化 I2S 外设。
设置 I2S 时钟源为 40M，位时钟为 16K，主设备模式，单声道输出，I2S 格式等。
5. 初始化 GDMA 外设。
 - 1) 设置为 GDMA 通道 0。
 - 2) 设置传输方向为外设到外设传输，源端地址为 I2S0 的 RX_DR，目的端地址为 UART 的 RB_THR。
 - 3) 使能总传输完成中断（GDMA_INT_Transfer）等。
6. 切换系统到 PWM 模式。
7. 初始化 CODEC 模拟寄存器。
8. 使能 CODEC 时钟。
9. 初始化 CODEC 外设。
 - 1) 初始化 CODEC 模拟寄存器。
 - 2) CODEC_MICBstMode 设置为 MICBST_Mode_Differential，即差分模式。
 - 3) CODEC_SampleRate 设置为 SAMPLE_RATE_16KHz，即采样频率为 16KHz。
 - 4) CODEC_I2SFormat 设置为 CODEC_I2S_DataFormat_I2S，即 I2S 格式。
 - 5) CODEC_I2SDataWidth 设置为 CODEC_I2S_DataWidth_16Bits，即数据宽度为 16bit。

```

1.  void driver_codec_init(void)
2.  {
3.      SystemCall(3, 1);
4.      CODEC_AnalogCircuitInit();
5.      RCC_PeriphClockCmd(APBPeriph_CODEC, APBPeriph_CODEC_CLOCK, ENABLE);
6.      CODEC_InitTypeDef CODEC_InitStruct;
7.      CODEC_StructInit(&CODEC_InitStruct);
8.      CODEC_InitStruct.CODEC_MicType      = CODEC_AMIC;
9.      CODEC_InitStruct.CODEC_MICBstMode   = MICBST_Mode_Differential;
10.     CODEC_InitStruct.CODEC_SampleRate    = SAMPLE_RATE_16KHz;
11.     CODEC_InitStruct.CODEC_I2SFormat     = CODEC_I2S_DataFormat_I2S;
12.     CODEC_InitStruct.CODEC_I2SDataWidth  = CODEC_I2S_DataWidth_16Bits;
13.     CODEC_Init(CODEC, &CODEC_InitStruct);
14. }

```

10. 使能 I2S 发送模式和接收模式；使能 GDMA 通道 0。

```

1.  void codec_demo(void)
2.  {
3.      .....

```

```

4.     I2S_Cmd(I2S0, I2S_MODE_TX | I2S_MODE_RX, ENABLE);
5.     GDMA_Cmd(GDMA_Channel_AMIC_NUM, ENABLE);
6.     }

```

11. AMIC 采集到模拟语音数据，经 CODEC 编码，送到 I2S0 接收 FIFO，利用 GDMA 将数据搬运到 UART0；当数据传输完成时，触发 GDMA_INT_Transfer 中断，进入中断处理函数 GDMA_Channel_AMIC_Handler。

- 1) 重新设置源端地址和目的端地址。
- 2) 重新设置 GDMA 传输数据大小。
- 3) 清除 GDMA 通道 0 GDMA_INT_Transfer 中断挂起位，使能 GDMA 通道 0。

```

1. void GDMA_Channel_AMIC_Handler(void)
2. {
3.     GDMA_SetSourceAddress(GDMA_Channel_AMIC, (uint32_t)&(I2S0->RX_DR));
4.     GDMA_SetDestinationAddress(GDMA_Channel_AMIC, (uint32_t)&(UART->RB_THR));
5.     GDMA_SetBufferSize(GDMA_Channel_AMIC, AUDIO_FRAME_SIZE / 4);
6.     GDMA_ClearINTPendingBit(GDMA_Channel_AMIC_NUM, GDMA_INT_Transfer);
7.     GDMA_Cmd(GDMA_Channel_AMIC_NUM, ENABLE);
8. }

```

15.1.3 实验现象

将语音数据导入到 PC 端。

- 使用 UART 工具接收数据。选择正确的端口，将波特率设置为 3M，打开端口；开始录音，RTL87x2E 进行语音采集、语音编码、发送编码数据；录音完成，将编码数据存储到 txt 文件中。
- 数据中会包含多余的空格，需要进一步处理。用 PCM_Data_Parser.exe 处理 txt 文件，输出 audio_data.data 即是编码后的语音数据。
- 使用 Cool Edit Pro 查看解码后的语音波形，或播放语音。

15.2 数字麦克

实现 DMIC 采集数字语音经 CODEC 编码的功能。DMIC 采集数字语音数据，经 CODEC 编码，送到 I2S0 接收 FIFO，利用 GDMA 将数据搬运到 UART0，PC 端收到 UART 传输的数据，使用 PCM_Data_Parser 解析数据，使用 Cool Edit Pro 播放录制的语音。

工程目录：\HoneyComb\sdk\board\evb\io_sample\CODEC\DMIC

15.2.1 硬件设计

硬件连接：P3_2→CLK，P3_3→DATA。同时：P2_4→RX，P2_5→TX。

EVB 外接 DMIC 模块，连接 P3_2 和 CLK，P3_3 和 DATA，GND 和 GND，VCC 和 VCC；EVB 外接 FT232 模块，连接 P2_4 和 RX，P2_5 和 TX。

15.2.2 软件流程

1. 配置 PAD、PINMUX，设置 P2_4、P2_5 为 UART0 功能。
2. 引脚定义。

```
1.  #define DMIC_MSBC_CLK_PIN      P3_2
2.  #define DMIC_MSBC_DAT_PIN      P3_3
```

3. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、无内部上拉、输出失能、输出低。
4. 配置 PINMUX：分配引脚分别为为 DMIC1_CLK、DMIC1_DAT 功能。

```
1.  void board_codec_init(void)
2.  {
3.      Pad_Config(DMIC_MSBC_CLK_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
                  PAD_PULL_NONE, PAD_OUT_DISABLE, PAD_OUT_LOW);
4.      Pad_Config(DMIC_MSBC_DAT_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
                  PAD_PULL_NONE, PAD_OUT_DISABLE, PAD_OUT_LOW);
5.      Pinmux_Config(DMIC_MSBC_CLK_PIN, DMIC1_CLK);
6.      Pinmux_Config(DMIC_MSBC_DAT_PIN, DMIC1_DAT);
7.  }
```

5. 初始化 UART 外设。

设置波特率为 3000000，使能 GDMA 发送功能等。

6. 初始化 I2S 外设。
设置 I2S 时钟源为 40M，位时钟为 16K，主设备模式，单声道输出，I2S 格式等。
7. 初始化 GDMA 外设。
 - 1) 设置为 GDMA 通道 0。
 - 2) 设置传输方向为外设到外设传输，源端地址为 I2S0 的 RX_DR，目的端地址为 UART 的 RB_THR。
 - 3) 使能总传输完成中断（GDMA_INT_Transfer）等。
8. 切换系统到 PWM 模式。
9. 初始化 CODEC 模拟寄存器。
10. 使能 CODEC 时钟。
11. 初始化 CODEC 外设。
 - 1) CODEC_MicType 设置为 CODEC_DMIC，即使用数字麦克风。
 - 2) CODEC_DmicClock 设置为 DMIC_Clock_2MHz，即设置 DMIC 的时钟频率为 2M。
 - 3) CODEC_DmicDataLatch 设置为 DMIC_Rising_Latch，即上升沿锁存。
 - 4) CODEC_SampleRate 设置为 SAMPLE_RATE_16KHz，即采样频率为 16KHz。
 - 5) CODEC_I2SFormat 设置为 CODEC_I2S_DataFormat_I2S，即 I2S 格式。
 - 6) CODEC_I2SDataWidth 设置为 CODEC_I2S_DataWidth_16Bits，即数据宽度为 16bit。

```

1. void driver_codec_init(void)
2. {
3.     SystemCall(3, 1);
4.     CODEC_AnalogCircuitInit();
5.     RCC_PeriphClockCmd(APBPeriph_CODEC, APBPeriph_CODEC_CLOCK, ENABLE);
6.     CODEC_InitTypeDef CODEC_InitStruct;
7.     CODEC_StructInit(&CODEC_InitStruct);
8.     CODEC_InitStruct.CODEC_MicType      = CODEC_DMIC;
9.     CODEC_InitStruct.CODEC_DmicClock    = DMIC_Clock_2MHz;
10.    CODEC_InitStruct.CODEC_DmicDataLatch = DMIC_Rising_Latch;
11.    CODEC_InitStruct.CODEC_SampleRate    = SAMPLE_RATE_16KHz;
12.    CODEC_InitStruct.CODEC_I2SFormat     = CODEC_I2S_DataFormat_I2S;
13.    CODEC_InitStruct.CODEC_I2SDataWidth  = CODEC_I2S_DataWidth_16Bits;
14.    CODEC_Init(CODEC, &CODEC_InitStruct);
15.    DBG_DIRECT("Use dmic test!");
16. }

```

12. 使能 I2S 发送模式和接受模式；使能 GDMA 通道 0。

```

1. void codec_demo(void)
2. {
3.     .....
4.     I2S_Cmd(I2S0, I2S_MODE_TX | I2S_MODE_RX, ENABLE);
5.     GDMA_Cmd(GDMA_Channel_DMIC_NUM, ENABLE);
6. }

```

13. DMIC 采集到数字语音数据，经 CODEC 编码，送到 I2S0 接收 FIFO，利用 GDMA 将数据搬运到 UART0；当数据传输完成时，触发 GDMA_INT_Transfer 中断，进入中断处理函数 GDMA_Channel_DMIC_Handler。

- 1) 重新设置源端地址和目的端地址。
- 2) 重新设置 GDMA 传输数据大小。
- 3) 清除 GDMA 通道 0 传输完成中断挂起位，使能 GDMA 通道 0。

```

1. void GDMA_Channel_DMIC_Handler(void)
2. {
3.     GDMA_SetSourceAddress(GDMA_Channel_DMIC, (uint32_t)&(I2S0->RX_DR));
4.     GDMA_SetDestinationAddress(GDMA_Channel_DMIC, (uint32_t)&(UART->RB_THR));
5.     GDMA_SetBufferSize(GDMA_Channel_DMIC, AUDIO_FRAME_SIZE / 4);
6.     GDMA_ClearINTPendingBit(GDMA_Channel_DMIC_NUM, GDMA_INT_Transfer);
7.     GDMA_Cmd(GDMA_Channel_DMIC_NUM, ENABLE);
8. }

```

15.2.3 实验现象

将语音数据导入到 PC 端。

- 使用 UART 工具接收数据。选择正确的端口，将波特率设置为 3M，打开端口；开始录音，RTL87x2E 进行语音采集、语音编码、发送编码数据；录音完成，将编码数据存储到 txt 文件中。
- 数据中会包含多余的空格，需要进一步处理。用 PCM_Data_Parser.exe 处理 txt 文件，输出 audio_data.data 即是编码后的语音数据。
- 使用 Cool Edit Pro 查看解码后的语音波形，或播放语音。

16 正交解调 (QDEC)

专用硬件设备：滚轮编码器

16.1 相位检测

使用 QDEC，实现检测转动传感器的运动状态的功能。滚动 Mouse 滚轮，在 DebugAnalyser 工具上，打印滚轮滚动信息。

Q-decode 用于检测如图 16-1 转动传感装置的运动状态。转动装置运动时会输出两路正交信号 PHA 和 PHB，Q-decode 通过检测 PHA 和 PHB 的相位变化来判断转动的方向、速度。相位分为：00 相位，01 相位，11 相位，10 相位。

如图 16-2 所示，正常情况相位状态变化有 2 种：00->10->11->01 和 01->11->10->00，PHA 和 PHB 每次只应有一个信号变化，如果 2 个信号同时变化，则认为状态出错。

工程目录：\HoneyComb\sdk\board\evb\io_sample\QDEC\QDEC

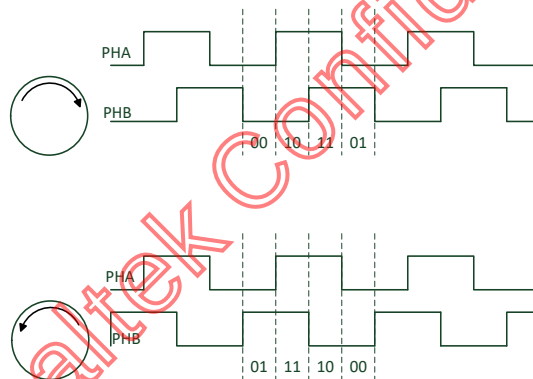


图 16-1 QDEC 连接图

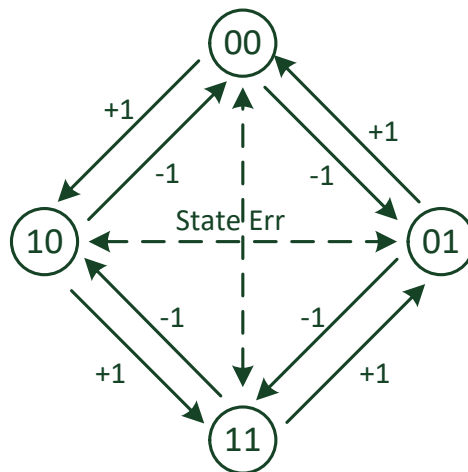


图 16-2 QDEC 相位转移图

16.1.1 硬件设计

硬件连接：P2_2→PhaseA，P2_3→PhaseB。

EVB 外接 Mouse 模块，连接 P2_2 和 PhaseA，P2_3 和 PhaseB，VCC 和 VCC，GND 和 GND。

16.1.2 软件流程

1. 引脚定义。

```
1.  #define QDEC_Y_PHA_PIN      P2_2 /* phase A */
2.  #define QDEC_Y_PHB_PIN      P2_3 /* phase B */
```

2. 配置 PAD：设置引脚、PINMUX 模式、PowerOn、内部上拉、输出失能、输出低。

3. 配置 PINMUX：分配引脚分别为 qdec_phase_a_y、qdec_phase_b_y 功能。

```
1.  void board_qdec_init(void)
2.  {
3.      Pad_Config(QDEC_Y_PHA_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
4.                  PAD_PULL_UP, PAD_OUT_DISABLE, PAD_OUT_LOW);
5.      Pad_Config(QDEC_Y_PHB_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
6.                  PAD_PULL_UP, PAD_OUT_DISABLE, PAD_OUT_LOW);
7.      Pinmux_Config(QDEC_Y_PHA_PIN, qdec_phase_a_y);
8.      Pinmux_Config(QDEC_Y_PHB_PIN, qdec_phase_b_y);
9.  }
```

4. app_task 初始化。

```
1.  os_task_create(&app_task_handle,"app",app_main_task,0,
2.                  APP_TASK_STACK_SIZE,APP_TASK_PRIORITY);
```

5. 在 app_main_task 中执行 driver_init 函数，执行 driver_qdec_init 函数，对 QDEC 外设进行初始化。

- 1) axisConfigY 设置为 ENABLE，即使能 Y 轴功能。
- 2) debounceEnableY 设置为 Debounce_Enable，即使能去抖动功能。

6. 使能 QDEC Y 轴新数据产生中断（QDEC_Y_INT_NEW_DATA）。

```
1.  void driver_qdec_init(void)
2.  {
3.      .....
4.      QDEC_InitTypeDef QDEC_InitStruct;
5.      QDEC_StructInit(&QDEC_InitStruct);
```

```

6.     QDEC_InitStruct.axisConfigY    = ENABLE;
7.     QDEC_InitStruct.debounceEnableY = Debounce_Enable;
8.     QDEC_INTConfig(QDEC, QDEC_Y_INT_NEW_DATA, ENABLE);
9.     .....
10.  }
```

7. 开始任务调度。

```

1.     os_sched_start();
```

8. 在 stack 准备好时，执行 app_handle_dev_state_evt 函数，执行 QDEC_Cmd 函数，使能 QDECY 轴通道。

```

1.     QDEC_Cmd(QDEC, QDEC_AXIS_Y, ENABLE);
```

9. 当 QDEC Y 轴检测到新数据时，触发 QDEC_Y_INT_NEW_DATA 中断，进入中断处理函数 Qdecode_Handler。

- 1) 屏蔽 QDEC_Y_INT_NEW_DATA 中断。
- 2) 读 Y 轴运动方向、计数值。
- 3) 定义消息类型 IO_MSG_TYPE_QDECODE，保存 Y 轴数据，发送 msg 到 task。
- 4) 清除 QDEC_Y_INT_NEW_DATA 中断挂起位。
- 5) 取消屏蔽 QDEC_Y_INT_NEW_DATA 中断。

```

1.  void Qdecode_Handler(void)
2.  {
3.      if (QDEC_GetFlagState(QDEC, QDEC_FLAG_NEW_CT_STATUS_Y) == SET)
4.      {
5.          QDEC_INTMask(QDEC, QDEC_Y_CT_INT_MASK, ENABLE);
6.          Y_Axis_Data.AxisDirection = QDEC_GetAxisDirection(QDEC, QDEC_AXIS_Y);
7.          Y_Axis_Data.AxisCount = QDEC_GetAxisCount(QDEC, QDEC_AXIS_Y);
8.          T_IO_MSG int_qdec_msg;
9.          int_qdec_msg.type = IO_MSG_TYPE_QDECODE;
10.         int_qdec_msg.u.buf = (void *)&Y_Axis_Data;
11.         if (false == app_send_msg_to_apptask(&int_qdec_msg))
12.             .....
13.         QDEC_ClearINTPendingBit(QDEC, QDEC_CLR_NEW_CT_Y);
14.         QDEC_INTMask(QDEC, QDEC_Y_CT_INT_MASK, DISABLE);
15.     }
16. }
```

10. app_main_task 循环检测 msg queue，当有 msg 时，执行 app_handle_io_msg(io_msg)函数。

11. 在 `app_handle_io_msg` 函数中, 判断消息类型为 `IO_MSG_TYPE_QDECODE`, 执行 `io_handle_qdec_msg`。打印 Y 轴数据, 即 Mouse 滚轮滚动的数据。

```
1. void io_handle_qdec_msg(T_IO_MSG *io_qdec_msg)
2. {
3.     QDEC_Data_TypeDef *p_buf = io_qdec_msg->u.buf;
4.     APP_PRINT_INFO2("[io_qdec]io_handle_qdec_msg:
        Y_Axis_Direction = %d,
        Y_Axis_Count = %d,",
        p_buf->AxisDirection,
        p_buf->AxisCount);
5. }
```

16.1.3 实验现象

滚动 Mouse 滚轮, 在 DebugAnalyser 工具上, 打印滚轮滚动信息。

Realtek Confidential

17 电容触摸感应按键（CTC）

电容触摸感应通过感应电容的变化，检测手指触摸表面或附近。当手指接触或接近传感表面时，它与传感器的电场耦合，这时一部分电荷通过手指流到地面，将导致传感器的电容下降。图 17-1 为其原理图。

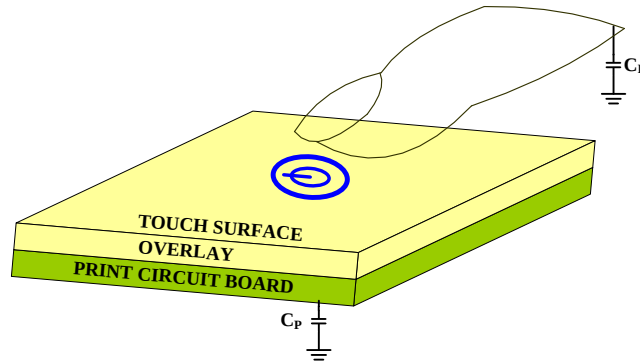


图 17-1 CTC 原理图

每个 CTC 感应通道都通过高灵敏度 ADC 进行过采样来实现的。ADC 将捕获参考电平(基线)和信号电平的电压，并将它们转换为数字数据。如果参考电平与信号电平的差值大于程序中指定的手指触摸阈值，则检测到触摸，同时发送触摸中断给 CPU。

17.1 按键感应

使用 CTC 实现电容触摸感应功能，当触摸 P2_2 和 P2_3 引脚后，进入中断，并在 DebugAnalyser 工具上，打印滚轮滚动信息。

工程目录：\HoneyComb\sdk\board\evb\io_sample\CAPTOUCH\Touch

17.1.1 硬件设计

硬件连接：P2_2→CTC 外设通道 0 检测引脚；P2_3→CTC 外设通道 1 检测引脚。

用手指触摸 P2_2 以及 P2_3 引脚，可触发 CTC 中断，读取状态。

17.1.2 软件流程

1. 开启 CTC 外设 RCC 时钟。
2. 初始化 CTC 外设，启用 ETC 功能，即开启差异阈值判断模式。
3. 开启 ADC 电源。
4. 初始化 CTC 通道。

- 1) CTC_DifferenceTouchThd 设置为 0x00，即通道触摸判定阈值为 0。
- 2) CTC_MBias 设置为 CTC_MBIAS_0p25uA，即通道灵敏度设置为 25uA。
- 3) CTC_ChannelEn 设置为 ENABLE，即使能 CTC 控制通道。
5. 初始化 CTC_Channel 0 和 CTC_Channel 1 外设。
6. 设置 NVIC 中断。
7. 开启通道 0 和通道 1 的触摸数值错误中断 (FALSE_TOUCH)。
8. 开启通道 0 和通道 1 的触摸释放中断 (TOUCH_RELEASE)。
9. 开启通道 0 和通道 1 的触摸按压感应中断 (TOUCH_PRESS)。

```

1.  void driver_ctc_init (void)
2.  {
3.      CTC_RCCConfig(CTC_CLOCK_SOURCE_1M);
4.      CTC_InitTypeDef CTC_InitStruct;
5.      CTC_StructInit(&CTC_InitStruct);
6.      CTC_InitStruct.CTC_ETCEn = ENABLE;
7.      CTC_Init(CTC, &CTC_InitStruct);
8.
9.      AUXADC_PowerOn(ENABLE);
10.     CTC_ChannelInitTypeDef CTC_ChannelInitStruct;
11.     CTC_ChannelStructInit(&CTC_ChannelInitStruct);
12.     CTC_ChannelInitStruct.CTC_DifferenceTouchThd = 0x00;
13.     CTC_ChannelInitStruct.CTC_MBias = CTC_MBIAS_0p25uA;
14.     CTC_ChannelInitStruct.CTC_ChannelEn = ENABLE;
15.     CTC_ChannelInit(CTC_Channel0, &CTC_ChannelInitStruct);
16.     CTC_ChannelInit(CTC_Channel1, &CTC_ChannelInitStruct);
17.     /* Enable IRQ */
18.     NVIC_InitTypeDef NVIC_InitStruct;
19.     NVIC_InitStruct.NVIC_IRQChannel = CAP_TOUCH_IRQn;
20.     NVIC_InitStruct.NVIC_IRQChannelPriority = 3;
21.     NVIC_InitStruct.NVIC_IRQChannelCmd = ENABLE;
22.     NVIC_Init(&NVIC_InitStruct);
23.
24.     CTC_INTConfig(CTC_INT_FALSE_TOUCH_CH0 |
25.                  CTC_INT_TOUCH_RELEASE_CH0 |
26.                  CTC_INT_TOUCH_PRESS_CH0, ENABLE);
27.     CTC_INTConfig(CTC_INT_FALSE_TOUCH_CH1 |
28.                  CTC_INT_TOUCH_RELEASE_CH1 |
29.                  CTC_INT_TOUCH_PRESS_CH1, ENABLE);
30. }

```

10. 使能电容触摸感应。
11. CTC 参考电平（基线）初始化完成后，打印初始化完成信息。
12. 用手指触摸 P2_2 或 P2_3 引脚，触发感应通道 1 或感应通道 2 按压中断，进入中断处理函数，可根据需求打印中断信息，通道 1 和通道 2 中断产生时的平均采样数据，与基线数据的差值以及触摸计数器的数值。

```

1.  void CAP_Touch_Handler(void)
2.  {
3.      /* Add corresponding interrupt processing according to application needs.*/
4.      .....
5.      if (CTC_GetINTStatus(CTC, CTC_INT_TOUCH_RELEASE_CH1))
6.      {
7.          //Add user code here.
8.          CTC_ClearINTPendingBit(CTC, CTC_INT_TOUCH_RELEASE_CH1);
9.          APP_PRINT_INFO0("CTC_INT_TOUCH_RELEASE_CH1\r\n");
10.     }
11.     if (CTC_GetINTStatus(CTC, CTC_INT_TOUCH_RELEASE_CH0))
12.     {
13.         //Add user code here.
14.         CTC_ClearINTPendingBit(CTC, CTC_INT_TOUCH_RELEASE_CH0);
15.         APP_PRINT_INFO0("CTC_INT_TOUCH_RELEASE_CH0\r\n");
16.     }
17.     if (CTC_GetINTStatus(CTC, CTC_INT_TOUCH_PRESS_CH1))
18.     {
19.         //Add user code here.
20.         CTC_ClearINTPendingBit(CTC, CTC_INT_TOUCH_PRESS_CH1);
21.         APP_PRINT_INFO0("CTC_INT_TOUCH_PRESS_CH1\r\n");
22.     }
23.     if (CTC_GetINTStatus(CTC, CTC_INT_TOUCH_PRESS_CH0))
24.     {
25.         //Add user code here.
26.         CTC_ClearINTPendingBit(CTC, CTC_INT_TOUCH_PRESS_CH0);
27.         APP_PRINT_INFO0("CTC_INT_TOUCH_PRESS_CH0\r\n");
28.     }
29.     .....
30. }

```

17.1.3 实验现象

触摸引脚 P2_2 或 P2_3，在 DebugAnalyser 工具上，打印触摸按键信息。

18 实时时钟（RTC）

18.1 滴答定时器

使用 RTC 实现 TICK 功能，在 DebugAnalyser 工具上，定时打印信息。

工程目录：\HoneyComb\sdk\board\evb\io_sample\RTC\Tick

18.1.1 软件流程

1. RTC 外设初始化。
 - 1) 复位 RTC 外设。
 - 2) RTC 的分频系数设置为（3200-1），RTC 时钟频率为 10Hz（32K/3200）。
 - 3) 使能 RTC 滴答中断。
 - 4) 使能 RTC 中断。
 - 5) 复位 RTC 计数值。
 - 6) 启动 RTC 外设。

```

1. void driver_rtc_init(void)
2. {
3.     RTC_DeInit();
4.     RTC_SetPrescaler(RTC_PRESCALER_VALUE);
5.     RTC_INTConfig(RTC_INT_TICK, ENABLE);
6.     .....
7.     RTC_NvCmd(ENABLE);
8.     RTC_ResetCounter();
9.     RTC_Cmd(ENABLE);
10. }

```

2. 设定时间到，触发中断，进入中断处理函数 RTC_Handler。
3. 判断 RTC 滴答中断状态为 SET，则
 - 1) 打印信息。
 - 2) 清除滴答中断。

```

1. void RTC_Handler(void)
2. {
3.     if (RTC_GetINTStatus(RTC_INT_TICK) == SET)
4.     {
5.         DBG_DIRECT("[main]RTC_Handler: RTC_INT_TICK");

```

```
6.         RTC_ClearTickINT();  
7.     }  
8. }
```

18.1.2 实验现象

在 DebugAnalyser 工具上，每 0.1s 打印信息。

Realtek Confidential

18.2 计数溢出

使用 RTC 实现 Overflow 功能，RTC 计数器溢出，在 DebugAnalyser 工具上，打印信息。

工程目录：\HoneyComb\sdk\board\evb\io_sample\RTC\Overflow

18.2.1 软件流程

1. RTC 外设初始化。
 - 1) 复位 RTC 外设。
 - 2) RTC 的分频系数设置为（0），RTC 时钟频率为 32000Hz。
 - 3) 开启 RTC 中断。
 - 4) 复位 RTC 计数值。
 - 5) 启动 RTC 外设。

```

1. void driver_rtc_init(void)
2. {
3.     RTC_DeInit();
4.     RTC_SetPrescaler(RTC_PRESCALER_VALUE);
5.     .....
6.     RTC_ResetCounter();
7.     RTC_Cmd(ENABLE);
8. }
```

2. 当计数器溢出时（计数大于 $2^{32}=4294967296$ ， $4294967296/32000 \approx 134217s$ ），触发中断，进入中断处理函数 RTC_Handler。
3. 判断 RTC 计数器溢出中断状态为 SET，则打印信息，并清除计数器溢出中断。

```

1. void RTC_Handler(void)
2. {
3.     if (RTC_GetINTStatus(RTC_INT_OVF) == SET)
4.     {
5.         DBG_DIRECT("[main]RTC_Handler: RTC_INT_OVF");
6.         RTC_ClearOverFlowINT();
7.     }
8. }
```

18.2.2 实验现象

在 DebugAnalyser 工具上，每 134217s 打印信息。

18.3 定时比较器（闹钟）

使用 RTC 实现 Comparator 功能，RTC 计数到，在 DebugAnalyser 工具上，打印信息。

工程目录：\HoneyComb\sdk\board\evb\io_sample\RTC\Comparator

18.3.1 软件流程

1. RTC 外设初始化。
 - 1) 复位 RTC 外设。
 - 2) 设置 RTC 的分频系数，RTC 时钟频率为 10Hz。
 - 3) 设置比较器通道 1 的比较值为 10。
 - 4) 使能 RTC 比较器通道 1 计数中断。
 - 5) 开启 RTC 中断。
 - 6) 复位 RTC 计数值。
 - 7) 启动 RTC 外设。

```

1. void driver_rtc_init(void)
2. {
3.     RTC_DeInit();
4.     RTC_SetPrescaler(RTC_PRESCALER_VALUE);
5.     RTC_SetComp(RTC_COMP_INDEX, RTC_COMP_VALUE);
6.
7.     RTC_CompINTConfig(RTC_INT_CMP_NUM, ENABLE);
8.     RTC_NvCmd(ENABLE);
9.     .....
10.    RTC_ResetCounter();
11.    RTC_Cmd(ENABLE);
12. }

```

2. 比较器设定时间（ $0.1s \times 10$ ）到，触发中断，进入中断处理函数 RTC_Handler。
3. 判断 RTC 比较器通道 1 计数中断状态为 SET，则
 - 1) 打印信息。
 - 2) 重新设置比较值。
 - 3) 清除比较器通道 1 计数中断。

```

1. void RTC_Handler(void)
2. {
3.     if (RTC_GetINTStatus(RTC_INT_CMP_NUM) == SET)
4.     {

```

```
5.         DBG_DIRECT("[main]RTC_Handler: RTC_INT_CMP_NUM");
6.         DBG_DIRECT("[main]RTC_Handler:
           RTC counter current value = %d",
           RTC_GetCounter());
7.         RTC_SetComp(RTC_COMP_INDEX, RTC_GetCounter() + RTC_COMP_VALUE);
8.         RTC_ClearCompINT(RTC_COMP_INDEX);
9.     }
10. }
```

18.3.2 实验现象

在 DebugAnalyser 工具上，每 1s 时，打印信息。

Realtek Confidential

18.4 分频比较器

使用 RTC 实现 Rrescaler + Comparator 功能，RTC 分频比较值以及 RTC 计数都满足中断条件时，在 DebugAnalyser 工具上，打印信息。

工程目录：\HoneyComb\sdk\board\evb\io_sample\RTC\Pre_comp3

18.4.1 软件流程

➤ 如果开启 RTC_PRECOMP3_INT 中断

1. RTC 外设初始化。
 - 1) 复位 RTC 外设。
 - 2) 设置 RTC 的分频系数，RTC 时钟频率为 10Hz。
 - 3) 设置 RTC 的分频比较值为 320（RTC 分频比较值 $\leq$ 分频系数）。
 - 4) 设置比较器通道 3 的比较值为 10。
 - 5) 使能 RTC 分频比较器通道 3 中断。
 - 6) 开启 RCT 中断。
 - 7) 复位 RTC 计数值。
 - 8) 启动 RTC 外设。

```

1.  void driver_rtc_init(void)
2.  {
3.      RTC_DeInit();
4.      RTC_SetPrescaler(RTC_PRESCALER_VALUE);
5.      RTC_SetPreCompValue(RTC_PRECOMP_VALUE);
6.      RTC_SetCompValue(RTC_COMP3, RTC_COMP3_VALUE);
7.
8.      RTC_INTConfig(RTC_INT_PRE_COMP3, ENABLE);
9.      RTC_NvCmd(ENABLE);
10.     .....
11.     RTC_ResetCounter();
12.     RTC_Cmd(ENABLE);
13. }
```

2. RTC 分频比较器值到达过设定值，且比较器设定时间（ $0.1s \times 10$ ）到，触发中断，进入中断处理函数 RTC_Handler。
3. 判断 RTC 分频比较器通道 3 中断状态为 SET，则

- 1) 打印信息。
- 2) 重新设置比较值。
- 3) 清除分频比较器通道 3 计数中断。

```

1.  void RTC_Handler(void)
2.  {
3.      if (RTC_GetINTStatus(RTC_INT_PRE_COMP3) == SET)
4.      {
5.          DBG_DIRECT("RTC_INT_PRE_COMP3");
6.          RTC_SetCompValue(RTC_COMP3, counter + RTC_COMP3_VALUE);
7.          RTC_ClearCompINT(RTC_INT_PRE_COMP3);
8.      }
9.  }

```

➤ 如果未开启 RTC_PRECOMP3_INT 中断

1. RTC 外设初始化。
 - 1) 复位 RTC 外设。
 - 2) 设置 RTC 的分频系数，RTC 时钟频率为 10Hz。
 - 3) 设置 RTC 的分频比较值为 320（RTC 分频比较值≤分频系数）。
 - 4) 使能 RTC 分频比较器中断。
 - 5) 开启 RTC 中断。
 - 6) 复位 RTC 计数值。
 - 7) 启动 RTC 外设。

```

1.  void driver_rtc_init(void)
2.  {
3.      RTC_DeInit();
4.      RTC_SetPrescaler(RTC_PRESCALER_VALUE);
5.      RTC_SetPreCompValue(RTC_PRECOMP_VALUE);
6.
7.      RTC_MaskINTConfig(RTC_INT_PRE_COMP, DISABLE);
8.      RTC_INTConfig(RTC_INT_PRE_COMP, ENABLE);
9.      .....
10.     RTC_ResetCounter();
11.     RTC_Cmd(ENABLE);
12. }

```

2. RTC 分频比较器值到达过设定值，且设定时间(0.1s)到，触发中断，进入中断处理函数 RTC_Handler。

3. 判断 RTC 分频比较器中断状态为 SET，则
 - 1) 打印信息。
 - 2) 清除分频比较器中断。

```
1. void RTC_Handler(void)
2. {
3.     if (RTC_GetINTStatus(RTC_INT_PRE_COMP) == SET)
4.     {
5.         DBG_DIRECT("RTC_INT_PRE_COMP");
6.         RTC_ClearCompINT(RTC_INT_PRE_COMP);
7.     }
8. }
```

18.4.2 实验现象

在 DebugAnalyser 工具上，设定时间到，打印中断信息。

Realtek Confidential

19 低功耗比较器（LPC）

RTL87x2E 芯片可作为电压检测作用的引脚为 P2_2、P2_3、P2_6、P2_7、Vbat，检测电压阈值范围：80mV~3200mV。

专用硬件设备：直流稳压源

19.1 电压检测

验证 LPC 电压检测功能，P2_2 检测输入电压，当电压低于设定阈值时触发 LPC 中断，在 DebugAnalyser 工具上，打印信息。

工程目录：\HoneyComb\sdk\board\evb\io_sample\LPC\VoltageDetection

19.1.1 硬件设计

硬件连接：P2_2→直流稳压源。

P2_2 外接直流稳压源，调节直流稳压源的电压值，P2_2 检测电压。

19.1.2 软件流程

1. 引脚定义。

```
1. #define LPC_CAPTURE_PIN P2_2
```

2. 配置 PAD：设置引脚 PINMUX 模式、PowerOn、无内部上拉、输出失能、输出高。

```
1. void board_lpc_init(void)
2. {
3.     Pad_Config(LPC_CAPTURE_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
4.               PAD_PULL_NONE, PAD_OUT_DISABLE, PAD_OUT_HIGH);
5.     Pinmux_Config(LPC_CAPTURE_PIN, IDLE_MODE);
6. }
```

3. app_task 初始化。

```
1. os_task_create(&app_task_handle,"app",app_main_task,0,
2.               APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);
```

4. 在 `app_main_task` 中执行 `driver_init` 函数，执行 `driver_lpc_init` 函数，对 LPC 外设进行初始化。
 - 1) `LPC_Channel` 设置为 `LPC_CAPTURE_CHANNEL`，即 P2_2 引脚。
 - 2) `LPC_Edge` 设置为 `LPC_VOLTAGE_DETECT_EDGE`，即 LPC 比较器输出极性为低于设置的电压阈值（`LPC_Vin_Below_Vth`）。
 - 3) `LPC_Threshold` 设置为 `LPC_VOLTAGE_DETECT_THRESHOLD`，即电压阈值为 2000mV。
5. 使能 LPC 功能，使能电压符合 LPC 比较器输出极性中断（`LPC_INT_LPCOMP_NV`）。

```

1.  void driver_lpc_init(void)
2.  {
3.      LPC_InitTypeDef LPC_InitStruct;
4.      LPC_StructInit(&LPC_InitStruct);
5.      LPC_InitStruct.LPC_Channel = LPC_CAPTURE_CHANNEL;
6.      LPC_InitStruct.LPC_Edge = LPC_VOLTAGE_DETECT_EDGE;
7.      LPC_InitStruct.LPC_Threshold = LPC_VOLTAGE_DETECT_THRESHOLD;
8.      LPC_Init(&LPC_InitStruct);
9.      LPC_Cmd(ENABLE);
10. }

```

6. 开始任务调度。

```

1.  os_sched_start();

```

7. 在 `stack` 准备好时，执行 `app_handle_dev_state_evt` 函数，执行 `nvic_lpc_config` 函数，配置 NVIC 中断，开启 LPC 中断。

```

1.  void nvic_lpc_config(void)
2.  {
3.      .....
4.      LPC_INTCmd(ENABLE);
5.  }

```

8. 调节直流稳压源电压值，当 P2_2 检测电压低于 2000mV 时，触发 `LPC_INT_LPCOMP_VOL` 中断，进入中断处理函数 `LPCOMP_Handler`。
 - 1) 失能 `LPC_INT_LPCOMP_VOL` 中断。
 - 2) 打印电压检测中断信息。

```

1.  void LPCOMP_Handler (void)
2.  {
3.      LPC_INTConfig(LPC_INT_LPCOMP_VOL, DISABLE);
4.      APP_PRINT_INFO0("LPCOMP_Handler\r\n");
5.      //Add application code here

```

```
6.     LPC_INTConfig(LPC_INT_LPCOMP_VOL, ENABLE);  
7.     }
```

19.1.3 实验现象

调节直流稳压源，当电压低于 2000mV 时，在 DebugAnalyser 工具上，打印信息。

Realtek Confidential

19.2 电压检测唤醒低功耗状态（DLPS）

验证 LPC 中断唤醒深度低功耗功能，P2_2 检测输入电压，当电压高于设定阈值时，系统自动进入 DLPS；当电压低于设定阈值时触发 LPC 中断，唤醒系统，退出 DLPS，在 DebugAnalyser 工具上，打印电压检测信息。

工程目录：\HoneyComb\sdk\board\evb\io_sample\LPC\DLPS

19.2.1 硬件设计

硬件连接：P2_2→直流稳压源。

P2_2 外接直流稳压源，调节直流稳压源的电压值，P2_2 检测电压。

19.2.2 软件流程

1. 开启 DLPS 功能。

```
1.  #define DLPS_EN 1
2.  #define USE_LPC_DLPS 1
3.  #define USE_USER_DEFINE_DLPS_EXIT_CB 1
4.  #define USE_USER_DEFINE_DLPS_ENTER_CB 1
```

2. 初始化 LPC 全局数据。

```
1.  void global_data_lpc_init(void)
2.  {
3.      IO_LPC_DLPS_Enter_Allowed = true;
4.  }
```

3. 配置 PAD、PINMUX 模式：设置 P2_2 为电压检测功能。

（详情参考：LPC→VoltageDetection）

4. 执行 pwr_mgr_init 函数，初始化电源管理。

- 1) 注册用户进入 DLPS 回调函数 app_enter_dlps_config，注册用户退出 DLPS 回调函数 app_exit_dlps_config。
- 2) 注册硬件控制回调函数 DLPS_IO_EnterDlpsCb 和 DLPS_IO_ExitDlpsCb，进入 DLPS 会保存 CPU、PINMUX、Peripheral 等，退出 DLPS 会恢复 CPU、PINMUX、Peripheral 等。
- 3) 设置 DLPS 模式。

```

1.  void pwr_mgr_init(void)
2.  {
3.      #if DLPS_EN
4.          if (false == dlps_check_cb_reg(app_dlps_check_cb))
5.              .....
6.          DLPS_IORegUserDlpsEnterCb(app_enter_dlps_config);
7.          DLPS_IORegUserDlpsExitCb(app_exit_dlps_config);
8.          DLPS_IORegister();
9.          lps_mode_set(LPM_DLPS_MODE);
10.         .....
11.     #endif
12. }

```

5. 在 app_enter_dlps_config 中，执行 io_lpc_dlps_enter 函数，打印进入 DLPS 信息。

```

1.  void io_uart_dlps_enter(void)
2.  {
3.      DBG_DIRECT("DLPS ENTER");
4.  }

```

6. 在 app_exit_dlps_config 中，执行 io_lpc_dlps_exit 函数，打印退出 DLPS 信息。

```

1.  void io_uart_dlps_exit(void)
2.  {
3.      DBG_DIRECT("DLPS EXIT");
4.  }

```

7. app_task 初始化。

```

1.  os_task_create(&app_task_handle,"app",app_main_task,0,
                APP_TASK_STACK_SIZE, APP_TASK_PRIORITY);

```

8. 在 app_main_task 中执行 driver_init 函数，执行 driver_lpc_init 函数，对 LPC 外设进行初始化。

- 1) LPC_Channel 设置为 LPC_CAPTURE_CHANNEL，即 P2_5 引脚。
- 2) LPC_Edge 设置为 LPC_VOLTAGE_DETECT_EDGE，即 LPC 比较器输出极性为低于设置的电压阈值（LPC_Vin_Below_Vth）。
- 3) LPC_Threshold 设置为 LPC_VOLTAGE_DETECT_THRESHOLD，即电压阈值为 2000mV。

9. 使能 LPC 功能。

10. 使能 LPC 电压比较极性中断。

11. 使能 LPC 唤醒。

12. 使能 RTC 系统唤醒功能。

```

1.  void driver_lpc_init(void)
2.  {
3.      LPC_InitTypeDef LPC_InitStruct;
4.      LPC_StructInit(&LPC_InitStruct);
5.      LPC_InitStruct.LPC_Channel  = LPC_CAPTURE_CHANNEL;
6.      LPC_InitStruct.LPC_Edge    = LPC_VOLTAGE_DETECT_EDGE;
7.      LPC_InitStruct.LPC_Threshold = LPC_VOLTAGE_DETECT_THRESHOLD;
8.      LPC_Init(&LPC_InitStruct);  LPC_Cmd(ENABLE);
9.
10.     /* Config LPC voltage detection interrupt */
11.     LPC_INTConfig(LPC_INT_LPCOMP_VOL, ENABLE);
12.
13.     /* Enable LPC DLPS Wakeup Config */
14.     LPC_WKCmd(ENABLE);
15.     RTC_SystemWakeupConfig(ENABLE);
16. }

```

13. 开始任务调度。

14. 在 stack 准备好时，执行 app_handle_dev_state_evt 函数，执行 nvic_lpc_config 函数。

- 1) 初始化 NVIC 配置。
- 2) 使能 LPC 中断。

```

1.  void nvic_lpc_config(void)
2.  {
3.      .....
4.      NVIC_Init(&NVIC_InitStruct);
5.      LPC_INTCmd(ENABLE);
6.  }

```

15. 调节直流稳压源电压值，当 P2_2 检测电压低于 2000mV 时，触发 LPC_INT_LPCOMP_VOL 中断，进入中断处理函数 LPCOMP_Handler。

- 1) 失能 LPC_INT_LPCOMP_VOL 中断。
- 2) 定义消息类型 IO_MSG_TYPE_BAT_LPC，发送 msg 给 task。

```

1.  void LPCOMP_Handler(void)
2.  {
3.      LPC_INTConfig(LPC_INT_LPCOMP_VOL, DISABLE);
4.      T_IO_MSG int_lpc_msg;

```



```

5.     int_lpc_msg.type = IO_MSG_TYPE_BAT_LPC;
6.     if (false == app_send_msg_to_apptask(&int_lpc_msg))
7.     {
8.         .....
9.     }
10. }

```

16. app_main_task 循环检测 msg queue，当有 msg 时，执行 app_handle_io_msg(io_msg)函数。

17. 在 app_handle_io_msg 函数中，判断消息类型为 IO_MSG_TYPE_BAT_LPC，打印电压检测信息，执行 io_lpc_handle_msg。

18. 判断类型为 IO_MSG_TYPE_BAT_LPC 时，则

- 1) 打印电压检测完成信息。
- 2) 使能 LPC_INT_VOLTAGE_VOL 中断。
- 3) IO_LPC_DLPS_Enter_Allowed 标志设置为 TRUE。

```

1. void io_lpc_handle_msg(T_IO_MSG *io_lpc_msg)
2. {
3.     .....
4.     if (IO_MSG_TYPE_BAT_LPC == type)
5.     {
6.         APP_PRINT_INFO0("io_handle_lpc_msg: VoltageDetection done ");
7.         LPC_INTConfig(LPC_INT_LPCOMP_NV, ENABLE);
8.         IO_LPC_DLPS_Enter_Allowed = true;
9.     }
10. }

```

19.2.3 实验现象

调节直流稳压源，当接入 P2_2 引脚的电压高于 2000mV，会自动进入 DLPS；当电压低于 2000mV 时，会退出 DLPS，并在 DebugAnalyser 工具上循环打印电压检测信息。

20 三线 SPI（SPI3WIRE）

20.1 读取鼠标 ID

使用三线 SPI 读取鼠标 ID。

工程目录：\HoneyComb\sdk\board\evb\io_sample\SPI3WIRE\Polling。

20.1.1 硬件设计

硬件连接：P2_2→CLK，P2_3→SDIO，P2_4→CS#。

EVB 外接 PWM3610DM-SUDU 鼠标，连接 P2_2 和 CLK，P2_3 和 SDIO，P2_4 和 CS#，同时连接 GND 和 VDD。

20.1.2 软件流程

1. 引脚定义。

```
1.  #define SPI_3WIRE_CLK_PIN    P2_2
2.  #define SPI_3WIRE_DATA_PIN   P2_3
3.  #define SPI_3WIRE_CS_PIN     P2_4
```

2. 配置 PAD 和 PINMUX。

```
1.  void board_3wire_spi_init(void)
2.  {
3.      Pad_Config(SPI_3WIRE_CLK_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
4.                  PAD_PULL_NONE, PAD_OUT_ENABLE, PAD_OUT_HIGH);
5.      Pad_Config(SPI_3WIRE_DATA_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
6.                  PAD_PULL_NONE, PAD_OUT_ENABLE, PAD_OUT_HIGH);
7.      Pad_Config(SPI_3WIRE_CS_PIN, PAD_PINMUX_MODE, PAD_IS_PWRON,
8.                  PAD_PULL_NONE, PAD_OUT_ENABLE, PAD_OUT_HIGH);
9.
10.     Pinmux_Config(SPI_3WIRE_CLK_PIN, SPI2W_CLK);
11.     Pinmux_Config(SPI_3WIRE_DATA_PIN, SPI2W_DATA);
12.     Pinmux_Config(SPI_3WIRE_CS_PIN, SPI2W_CS);
13. }
```

3. 执行 driver_3wire_spi_init 函数，对 SPI3WIRE 外设进行初始化：

- 1) 设置系统时钟为 20M，SPI3WIRE 输出时钟 800k。
- 2) 设置工作模式为三线 SPI。
- 3) 设置读取数据的延时时间。
- 4) 设置输出不产生延时。
- 5) 设置 SPI3WIRE 为普通模式。

```

1.  void driver_3wire_spi_init(void)
2.  {
3.      /* Enable SPI3W clock */
4.      RCC_PeriphClockCmd(APBPeriph_SPI3W, APBPeriph_SPI3W_CLOCK, ENABLE);
5.
6.      /* Initialize SPI3W */
7.      SPI3W_InitTypeDef SPI3W_InitStruct;
8.      SPI3W_StructInit(&SPI3W_InitStruct);
9.
10.     SPI3W_InitStruct.SPI3W_SysClock    = 20000000;
11.     SPI3W_InitStruct.SPI3W_Speed      = 800000;
12.     SPI3W_InitStruct.SPI3W_Mode       = SPI3W_3WIRE_MODE;
13.     /* The delay time from the end of address phase to the start of read data phase.
14.      * delay time = (SPI3W_ReadDelay + 1)/(2*SPI3W_Speed).
15.      * delay time = (0x03 + 1)/(2 * speed) = 2.5us
16.      */
17.     SPI3W_InitStruct.SPI3W_ReadDelay   = 0x3;
18.     SPI3W_InitStruct.SPI3W_OutputDelay = SPI3W_OE_DELAY_NONE;
19.     SPI3W_InitStruct.SPI3W_ExtMode     = SPI3W_NORMAL_MODE;
20.     SPI3W_Init(&SPI3W_InitStruct);
21. }

```

4. 设置 resync 信号的持续时间，使能 resync 信号输出，等待 resync 信号输出完毕后失能 resync 信号。

```

1.  void spi3wire_demo(void)
2.  {
3.      .....
4.
5.      /* Send resync time. Resync signal time = 2*1/(2*SPI3W_Speed) = 1.25us
6.       * This parameter can be only be the following value: 0x0 to 0xf.
7.       */
8.      SPI3W_SetResyncTime(2);
9.      SPI3W_ResyncSignalCmd(ENABLE);
10.     while (SPI3W_GetFlagStatus(SPI3W_FLAG_RESYNC_BUSY) == SET);
11.     SPI3W_ResyncSignalCmd(DISABLE);
12. }

```

```

13.      .....
14.    }

```

5. 执行 SPI3WIRE_Cmd 函数，使能三线 SPI。
6. 执行 mouse_reset 函数，向 mouse 指定地址写入指令，重置鼠标状态。

```

1.  bool spi3wire_writebyte(uint8_t address, uint8_t data)
2.  {
3.      uint32_t timeout = 0;
4.
5.      /* Check 3wire spi busy or not */
6.      while (SPI3WIRE_GetFlagStatus(SPI3WIRE_FLAG_BUSY) == SET)
7.      {
8.          timeout++;
9.          if (timeout > 0x1ffff)
10.         {
11.             return false;
12.         }
13.     }
14.     /* Write data */
15.     SPI3WIRE_StartWrite(address, data);
16.
17.     timeout = 0;
18.     while (SPI3WIRE_GetFlagStatus(SPI3WIRE_FLAG_BUSY) == SET)
19.     {
20.         timeout++;
21.         if (timeout > 0x1ffff)
22.         {
23.             return false;
24.         }
25.     }
26.     return true;
27. }

```

7. 执行 mouse_getproductid 函数读取鼠标的 ID。执行 spi3wire_readbyte 函数，读取 0x00 和 0x01 地址下的 ID。
 - 1) 等待 busy 状态标志位消除。
 - 2) 清除 SPI3WIRE 的接收 FIFO 中数据。
 - 3) 通过 SPI3WIRE 读取传入参数地址下的一个数据，等待 busy 状态标志位消除。
 - 4) 读取 SPI3WIRE 接收 FIFO 中数据
 - 5) 返回读取的数据并打印。

```

1.  uint8_t spi3wire_readbyte(uint8_t address)

```

```

2.  {
3.      uint8_t reg_value = 0;
4.      uint32_t timeout = 0;
5.
6.      /* Check spi busy or not */
7.      while (SPI3W_GetFlagStatus(SPI3W_FLAG_BUSY) == SET)
8.      {
9.          timeout++;
10.         if (timeout > 0xffff)
11.         {
12.             break;
13.         }
14.     }
15.
16.     /* Clear Receive data length */
17.     SPI3W_ClearRxDataLen();
18.
19.     SPI3W_StartRead(address, 1);
20.
21.     timeout = 0;
22.     while (SPI3W_GetFlagStatus(SPI3W_FLAG_BUSY) == SET)
23.     {
24.         timeout++;
25.         if (timeout > 0xffff)
26.         {
27.             break;
28.         }
29.     }
30.
31.     /* Get the length of received data */
32.     while (SPI3W_GetRxDataLen() == 0);
33.     /* Read data */
34.     SPI3W_ReadBuf(&reg_value, 1);
35.
36.     return reg_value;
37. }

```

20.1.3 实验现象

在 DebugAnalyzer 工具上，打印读取到的内部数据 ID。若鼠标型号为 PWM3610DM-SUDU，读取到的 ID 为 0x3e/0x01。

参考文献

- [1]. RTL8762E Evaluation Board User Manual
- [2]. RTL8762E SDK User Guide
- [3]. RTL8762x MP Tool User Guide
- [4]. DebugAnalyzer User Guide
- [5]. RTL8762E ADC 应用实例

Realtek Confidential